Improving Retrieval Quality in Arabic RAG Systems via Corpus-Steered Query Enhancement

A thesis submitted in partial fulfillment of the requirements of the B.Sc. degree in Electrical and Electronic Engineering

Submitted by:

Mohammed Elhaj Sami (INDEX)

Osman Bashir (INDEX)

Supervised by:

Dr. Tahani

To

Department of Electrical and Electronic Engineering

Faculty of Engineering

University of Khartoum

August 2026

Declaration of Authorship

We declare that this thesis, submitted in partial fulfilment of the requirements for the degree of Bachelor of Science in Electrical and Electronic Engineering, is our own work. It contains no material previously published or written by another person, nor material which to a substantial extent has been accepted for the award of any other degree at the University of Khartoum or any other institution, except where due acknowledgement is made in the text. Any contribution made to this research by others is explicitly acknowledged.

Name: Mohammed Elhaj Sami Signature: . . . . . Date: . . . . . .

Name: Osman Bashir Signature: . . . . . Date: . . . . . .

Dedication

Acknowledgments

Abstract

Retrieval-Augmented Generation (RAG) grounds Large Language Model (LLM) outputs in external knowledge, but Arabic information retrieval fails on many queries because of morphological richness, orthographic variation, and vocabulary mismatch. Query enhancement (QE)—modifying a query before retrieval—offers a modular remedy, but its techniques were developed for English with far larger proprietary models, and neither their transfer to Arabic nor their placement in a hybrid architecture is established. This thesis asks to what extent LLM-based QE—blind or steered by the target corpus—can improve Arabic information retrieval across sparse, dense, and hybrid paradigms, using only openly available models. Its objectives were to establish baselines, adapt generative QE for Arabic, compare generators, and locate the expansion within a hybrid pipeline. Experiments used the Arabic subset of the Multilingual Information Retrieval Across a Continuum of Languages (MIRACL) benchmark (2,896 queries, 2.06 million passages), with the Multilingual Dense Passage Retriever (mDPR) and the Best Matching 25 (BM25) implementation BM25S. An error analysis identifying a 34% query failure rate motivated adapting Query2Doc for Arabic. Ten LLMs were compared as generators, of which nine proved viable; query repetition, hybrid fusion, and Corpus-Steered Query Expansion (CSQE) followed. CSQE instead retrieves top-ranked documents from the collection and builds the expansion from the wording found in them, not from the model's own memory. QE improved dense retrieval for all nine but degraded sparse retrieval for most of them through term dilution, which query repetition removed. Placement then proved decisive: expanding the sparse retriever alone outperformed expanding both retrievers or the dense retriever alone, because withholding the expansion from the dense retriever preserved the complementarity on which fusion depends. The final system raised Normalised Discounted Cumulative Gain at rank 10 (NDCG@10) from 0.4621 for BM25 alone to 0.7137, a relative gain of 54.5%, and exceeded the strongest hybrid baseline without QE by 13.9%, using an openly available 8-billion-parameter generator. LLM-based QE is therefore effective across all three paradigms, provided that its form and placement are adapted to the retriever rather than applied uniformly.

المستخلص

يربط التوليد المعزَّز بالاسترجاع (RAG) مخرجات النماذج اللغوية الكبيرة (LLM) بمعرفة خارجية، غير أن استرجاع المعلومات العربية يُخفق في كثير من الاستعلامات بسبب الثراء الصرفي والتباين الإملائي وعدم التطابق المعجمي بين الاستعلام والمستند. ويُقدِّم تحسين الاستعلام ،(QE) أي تعديل الاستعلام قبل الاسترجاع، حلاً يمكن إضافته دون تغيير بقية النظام، إلا أن تقنياته وُضعت أساساً للغة الإنجليزية معتمدةً على نماذج احتكارية أكبر بكثير، ولم تثبت بعدُ فاعلية تطبيقها على العربية ولا موضعها الأمثل داخل بنية هجينة. تبحث هذه الأطروحة مدى قدرة تحسين الاستعلام المعتمد على النماذج اللغوية الكبيرة، بنوعيه الأعمى والموجَّه بالمدوَّنة المستهدفة، على الارتقاء باسترجاع المعلومات العربية في أنماط الاسترجاع الثلاثة: المتناثر والكثيف والهجين، مع الاقتصار على نماذج متاحة للعموم. وتمثلت أهدافها في وضع خطوط أساس، وتكييف التحسين التوليدي للعربية، والمقارنة بين النماذج التوليدية، وتحديد موضع التوسيع داخل المسار الهجين. أُجريت التجارب على الجزء العربي من معيار ،MIRACL الذي يضم 2,896 استعلاماً و06.2 مليون مقطع نصي، باستخدام mDPR للاسترجاع الكثيف وBM25S، وهو تطبيق لخوارزمية ،BM25 للاسترجاع المتناثر. وأظهر تحليل الأخطاء إخفاق 34% من الاستعلامات، فدفع ذلك إلى تكييف تقنية Query2Doc للعربية. وقورنت عشرة نماذج لغوية توليدية، وأثبتت التجارب صلاحية تسعة منها، ثم اختُبرت طرائق تكرار الاستعلام والدمج الهجين والتوسيع الموجَّه بالمدوَّنة ،(CSQE) الذي يسترجع أولاً عدداً قليلاً من المستندات الأعلى ترتيباً من المدوَّنة نفسها، ثم يبني التوسيع من ألفاظها لا من المعرفة الكامنة في النموذج. وقد حسَّن توسيع الاستعلام الاسترجاعَ الكثيف في النماذج التسعة جميعها، غير أنه أضعف الاسترجاع المتناثر في معظمها لإضعافه أوزان مصطلحات الاستعلام الأصلية، وهو أثر أزاله تكرار الاستعلام. وتبيَّن أن موضع تطبيق التوسيع عامل حاسم؛ إذ تفوَّق تطبيقه على المسترجع المتناثر وحده على تطبيقه على المسترجعَين معاً أو على المسترجع الكثيف وحده، لأن حجب التوسيع عن المسترجع الكثيف يحفظ التكامل بين المسترجعَين الذي يقوم عليه الدمج. وبذلك رفع النظام النهائي الكسب التراكمي المخصوم المُطبَّع عند الرتبة 10 (NDCG@10) من 4621.0 باستخدام BM25 وحده إلى 7137.0، بزيادة نسبية بلغت 5%.54، وتجاوز أقوى نظام هجين بلا تحسين للاستعلام بنسبة 9%.13، باستخدام نموذج توليدي متاح للعموم يضم 8 مليارات معلمة. ومن ثَمَّ فإن تحسين الاستعلام المعتمد على النماذج اللغوية الكبيرة فعّال في أنماط الاسترجاع الثلاثة جميعها، شريطة مواءمة صيغته وموضعه لطبيعة المسترجع.

Contents

Contents vii

List of Figures xii

List of Tables xiii

List of Abbreviations xv

1 Introduction 1

1.1 Problem Definition . . . 2

1.2 Objectives . . . 3

1.3 Thesis Layout . . . 4

2 Theoretical Background and Literature Review 6

2.1 Theoretical Background . . . 6

2.1.1 LLMs and the Transformer Architecture . . . 6

2.1.2 RAG . . . 7

2.1.3 Information Retrieval Methods . . . 9

2.1.4 QE Techniques . . . 10

2.1.5 Arabic Language Processing Challenges . . . 13

2.2 Mathematical Models . . . 15

2.2.1 BM25 Scoring Function . . . 15

2.2.2 Dense Retrieval and Cosine Similarity . . . 15

2.2.3 Hybrid Fusion Functions . . . 16

2.2.4 Evaluation Metrics . . . 17

2.3 Evaluation Dataset Selection . . . 18

2.3.1 Selected Benchmark . . . 19

2.3.2 Alternatives Considered . . . 19

2.3.3 Limitation 20
2.4 Models Used in Experiments 20
2.4.1 Retrieval Models 21
2.5 Related Work 22
2.5.1 Foundational QE Research (2022–2023) 22
2.5.2 Modern LLM-Based QE (2024–2025) 23
2.5.3 Arabic Information Retrieval and RAG 25
2.5.4 Research Gap 26

3 Methodology **29**
3.1 Dataset and Experimental Setup 29
3.1.1 Dataset: MIRACL Arabic 29
3.1.2 Hardware and Software Environment 30
3.1.3 Evaluation Metrics 30
3.1.4 Experimental Pipeline Overview 31
3.2 Baseline Implementation 32
3.2.1 Dense Retrieval Baseline (mDPR) 32
3.2.2 Sparse Retrieval Baseline (BM25S) 33
3.2.3 Baseline Comparison Rationale 34
3.3 Error Analysis Methodology 34
3.3.1 Quantitative Analysis Framework 35
3.3.2 Query Length Bucketing 36
3.3.3 Failed Query Inspection 36
3.4 Query2Doc Implementation 36
3.4.1 Query2Doc Technique 37
3.4.2 Modifications from the Original Paper 38
3.4.3 LLM Configuration 38
3.4.4 Batch Processing Optimisation 39
3.4.5 Dense Retrieval with Query2Doc 39
3.4.6 BM25 Retrieval with Query2Doc 40
3.5 Model Comparison Methodology 40
3.5.1 Model Selection Criteria 41
3.5.2 Standardised Evaluation Protocol 41

3.5.3 Temperature Selection . . . 42
3.5.4 Model-Specific Technical Issues . . . 43
3.5.5 Quantisation Strategy . . . 43
3.6 Query Repetition for Sparse Retrieval . . . 43
3.6.1 Fixed Repetition (Query2Doc-style) . . . 44
3.6.2 Adaptive Repetition (MuGI-style) . . . 44
3.6.3 Motivation for the Adaptive Variant . . . 45
3.6.4 Sweep Design . . . 45
3.7 Hybrid Retrieval Fusion . . . 45
3.8 CSQE . . . 46
3.8.1 The CSQE Pipeline . . . 47
3.8.2 Component Ablation Design . . . 49
3.8.3 Retriever-Specific Application . . . 49
3.9 Per-Query Error Analysis . . . 50
3.9.1 Per-Query Metric Computation . . . 51
3.9.2 Classification Thresholds . . . 51
3.9.3 First-Pass Quality Split . . . 51
3.9.4 Regression Classification . . . 51

4 Results and Discussion 53
4.1 Baseline Results and Comparison . . . 53
4.1.1 Complementary Strengths . . . 54
4.2 Error Analysis Findings . . . 55
4.2.1 Overall Failure Rate . . . 55
4.2.2 Short Query Performance Gap . . . 56
4.2.3 Retrieval Coverage Analysis . . . 57
4.2.4 Technique Selection Rationale . . . 58
4.3 Query2Doc Results . . . 58
4.3.1 Dense Retrieval . . . 59
4.3.2 BM25 Retrieval . . . 60
4.3.3 Comparison with the Original Query2Doc Paper . . . 61
4.4 Model Comparison Results . . . 61
4.4.1 Dense Retrieval Leaderboard . . . 62

4.4.2 BM25 Retrieval Results . . . 63
4.4.3 Dropped Models Analysis . . . 64
4.5 Cross-Cutting Findings from the Model Comparison . . . 65
4.5.1 Model Size Correlates with Dense Retrieval Performance . . . 65
4.5.2 Generational Improvement Within Model Families . . . 66
4.5.3 Arabic Specialisation vs. Multilingual Training . . . 67
4.5.4 Dense vs. BM25 Behaviour with QE . . . 68
4.5.5 Model Selection Summary . . . 68
4.6 BM25 Query Repetition Results . . . 69
4.7 Hybrid Retrieval Fusion Results . . . 72
4.8 CSQE Results . . . 74
4.8.1 Main CSQE Results . . . 75
4.8.2 Component Ablation . . . 75
4.9 CSQE with Hybrid Fusion . . . 76
4.9.1 Fusion Configuration Comparison . . . 77
4.9.2 Overall System Progression . . . 79
4.10 Per-Query Error Analysis . . . 81
4.10.1 Win/Loss Distribution . . . 81
4.10.2 First-Pass Quality as the Largest Modulator . . . 83
4.10.3 Big Wins: The Corpus Grounding Effect . . . 85
4.10.4 Regression Analysis . . . 85

5 Conclusion and Recommendations 88
5.1 Conclusions . . . 88
5.2 Challenges . . . 93
5.3 Recommendations for Future Work . . . 94

Bibliography 97

A Model Details A-1
A.1 Summary of Language Models . . . A-1
A.2 Arabic-Specialized Models . . . A-2
A.2.1 Falcon-H1-Arabic-3B . . . A-2
A.2.2 Jais-2-8B . . . A-2

A.2.3 ALLaM-7B A-3
A.2.4 SILMA Kashif-2B A-4
A.3 Multilingual Models A-4
A.3.1 Qwen 2.5 3B A-4
A.3.2 Qwen 2.5 7B A-5
A.3.3 Qwen3-4B A-5
A.3.4 Qwen3-8B A-5
A.3.5 Gemma 3 4B-IT A-6
A.3.6 Aya Expanse 8B A-6
A.4 Model-Specific Technical Issues A-7

B Extended Result Tables **B-1**
B.1 BM25 Query Repetition: Full Sweep B-1
B.2 Hybrid Fusion: Full Convex-Combination Sweep B-2
B.3 Summary of All Experiments B-2
B.4 Representative Big-Win Queries B-3

C Implementation Code **C-1**
C.1 Query Expansion System Prompts C-1
C.2 Query Repetition and Expansion Assembly C-3
C.3 CSQE Configuration C-4
C.4 Source Code Repository C-5

List of Figures

2.1 RAG system architecture 8

3.1 Experimental pipeline overview 31
3.2 mDPR encoding flow 33
3.3 BM25S indexing flow 34
3.4 Query2Doc generation 37
3.5 Hybrid fusion: CC and RRF 46
3.6 CSQE pipeline 48
3.7 Best system architecture 50

4.1 Per-query NDCG@10 distribution 55
4.2 NDCG@10 by query length 57
4.3 Dense retrieval NDCG@10 across models 63
4.4 Model size vs Query2Doc gain 66
4.5 BM25 query-repetition sweep across nine models 71
4.6 Dense vs BM25 gain per model 72
4.7 Hybrid CC α sweep, all metrics 74
4.8 System progression to the best system 80
4.9 Per-query Δ NDCG@10 histogram 82
4.10 NDCG@10 by first-pass relevance 83
4.11 Per-bucket comparison by query length 84

List of Tables

2.1 Query-enhancement papers reviewed 13
2.2 Arabic retrieval and QA datasets surveyed 19
2.3 LLMs evaluated for Arabic QE 21

3.1 Initial Query2Doc generation parameters 39
3.2 Model configurations for the Query2Doc comparison 41

4.1 Baseline retrieval results 53
4.2 Baseline query performance segmentation 56
4.3 Retrieval performance by query length 56
4.4 Relevant-passage coverage by retrieval depth 58
4.5 Query2Doc results with dense retrieval 59
4.6 Query2Doc results with BM25 retrieval 60
4.7 Query2Doc: original paper versus this implementation 61
4.8 Dense retrieval results for all models 62
4.9 BM25 retrieval results for all models 64
4.10 Generational comparison within the Qwen family 67
4.11 Models improving over baseline, by retriever 68
4.12 BM25 metrics at each model's best repetition setting 70
4.13 Hybrid retrieval fusion results 73
4.14 CSQE retrieval results 75
4.15 CSQE corpus-only, blind-only and combined variants 76
4.16 Effect of repetition factor α on CSQE BM25 76
4.17 CSQE application strategies in hybrid fusion 77
4.18 BM25-expanded RRF component ablation 78
4.19 Effect of α on the best BM25-expanded RRF system 78
4.20 NDCG@10 gains of BM25-expanded RRF over prior systems 79

4.21 Per-query comparison of CSQE+Hybrid against the blind baseline 81
4.22 Classification of regression queries 86

A.1 Summary of language models used A-1

B.1 BM25 NDCG@10 under query repetition, full sweep B-1
B.2 Hybrid fusion, full convex-combination sweep B-2
B.3 Summary of all experiments B-3
B.4 Representative big-win queries B-4

List of Abbreviations

AI	**A**rtificial **I**ntelligence
API	**A**pplication **P**rogramming **I**nterface
ARCD	**A**rabic **R**eading **C**omprehension **D**ataset
BERT	**B**idirectional **E**ncoder **R**epresentations from **T**ransformers
BF16	16-bit Brain Floating Point (*bfloat16*) precision
BM25	**B**est **M**atching **25**
CC	**C**onvex **C**ombination
CSQE	**C**orpus-**S**teered **Q**uery **E**xpansion
DCG	**D**iscounted **C**umulative **G**ain
DPO	**D**irect **P**reference **O**ptimization
DPR	**D**ense **P**assage **R**etrieval
FAISS	**F**acebook **AI** **S**imilarity **S**earch
FP16	16-bit **F**loating-**P**oint (half) precision
GPT	**G**enerative **P**re-trained **T**ransformer
GPU	**G**raphics **P**rocessing **U**nit
GQA	**G**rouped **Q**uery **A**ttention
GRF	**G**enerative **R**elevance **F**eedback
GRPO	**G**roup **R**elative **P**olicy **O**ptimization
HyDE	**Hy**pothetical **D**ocument **E**mbeddings
IDCG	**I**deal **D**iscounted **C**umulative **G**ain
IDF	**I**nverse **D**ocument **F**requency
LLM	**L**arge **L**anguage **M**odel

MAP	**M**ean **A**verage **P**recision
mDPR	**M**ultilingual **D**ense **P**assage **R**etriever
MIPS	**M**aximum **I**nner **P**roduct **S**earch
MIRACL	**M**ultilingual **I**nformation **R**etrieval **A**cross a **C**ontinuum of **L**anguages
MMLU	**M**assive **M**ultitask **L**anguage **U**nderstanding
MMMLU	**M**ultilingual **M**assive **M**ultitask **L**anguage **U**nderstanding
MMTEB	**M**assive **M**ultilingual **T**ext **E**mbedding **B**enchmark
MRR	**M**ean **R**eciprocal **R**ank
MS MARCO	**M**icro**s**oft **MA**chine **R**eading **CO**mprehension
MSA	**M**odern **S**tandard **A**rabic
MTEB	**M**assive **T**ext **E**mbedding **B**enchmark
MuGI	**Mu**lti-Text **G**eneration **I**ntegration
NCAI	**N**ational **C**enter for **A**rtificial **I**ntelligence
NDCG	**N**ormalised **D**iscounted **C**umulative **G**ain
NF4	4-bit **N**ormal**F**loat quantisation format
NLP	**N**atural **L**anguage **P**rocessing
NLTK	**N**atural **L**anguage **T**ool**k**it
OALL	**O**pen **A**rabic **LL**M Leaderboard
QA	**Q**uestion **A**nswering
QE	**Q**uery **E**nhancement
QLoRA	**Q**uantised **Lo**w-**R**ank **A**daptation
RAG	**R**etrieval-**A**ugmented **G**eneration
RAGQA	**RAG Q**uestion **A**nswering
ReLU	**Re**ctified **L**inear **U**nit
RLHF	**R**einforcement **L**earning from **H**uman **F**eedback
RM3	**R**elevance **M**odel **3**
RMSNorm	**R**oot **M**ean **S**quare **Norm**alization

RoPE **Ro**tary **P**osition **E**mbeddings

RQ-RAG **R**efine **Q**ueries for **R**etrieval-**A**ugmented **G**eneration

RRF **R**eciprocal **R**ank **F**usion

SFT **S**upervised **F**ine-**T**uning

SiLU **Si**gmoid **L**inear **U**nit

SSM **S**tate **S**pace **M**odel

STS **S**emantic **T**extual **S**imilarity

SwiGLU **Swi**sh-**G**ated **L**inear **U**nit

TII **T**echnology **I**nnovation **I**nstitute

TyDi QA **Ty**pologically **Di**verse **Q**uestion **A**nswering

VRAM **V**ideo **R**andom-**A**ccess **M**emory

Chapter 1

Introduction

The rapid growth of digital Arabic content has intensified the need for intelligent information retrieval systems capable of accurately locating relevant information within large corpora. Retrieval-Augmented Generation (RAG) has emerged as the dominant paradigm for grounding Large Language Model (LLM) outputs in external knowledge, mitigating hallucination by retrieving relevant passages before generation. However, the quality of a RAG system's final output is fundamentally bounded by the quality of its retrieval component: if the retriever fails to surface relevant documents, the generator cannot compensate for this deficiency regardless of its capabilities.

Arabic information retrieval faces challenges that compound this retrieval bottleneck. The Arabic language employs a root-and-pattern morphological system in which a single root can yield dozens of surface forms, creating severe vocabulary mismatch between queries and documents. Orthographic variations in characters such as Hamza and Alif introduce further inconsistencies, while the diglossic nature of Arabic—where Modern Standard Arabic (MSA) is used in formal text but regional dialects dominate informal communication—creates a systematic gap between how users formulate queries and how knowledge bases are written. These linguistic characteristics amplify the retrieval failure rate beyond what is observed for English and other morphologically simpler languages.

Query enhancement (QE)—the modification of a user's query before it is submitted to the retriever—offers a promising pre-retrieval intervention. Rather than modifying the retriever or reindexing the knowledge base, QE operates as a modular layer that enriches the query with additional vocabulary, context, and semantic information. This approach is retriever-agnostic, requires no changes to the existing infrastructure, and can be deployed incrementally. Recent work has demonstrated that LLMs can generate effective query expansions by producing pseudo-documents that bridge the vocabulary gap between queries and relevant passages.

However, existing QE research is overwhelmingly English-centric. The foundational techniques—

Query2Doc, Hypothetical Document Embeddings (HyDE), and Generative Relevance Feedback (GRF)—were developed and validated on English benchmarks using proprietary models with 175 billion or more parameters, such as the Generative Pre-trained Transformer 3 (GPT-3), and although they adjust the surface form of the expanded query between sparse and dense retrieval, the expansion itself is generated without regard to the retrieval paradigm, and no study establishes which retriever within a hybrid architecture should receive it. Whether these techniques transfer to Arabic, how LLM-generated expansions interact with each retrieval paradigm in the Arabic setting, and whether expansions can be grounded in the target corpus rather than generated blindly, are questions the literature leaves open—particularly under the practical constraint of openly available models of modest scale rather than commercial application programming interfaces (APIs).

1.1 Problem Definition

Arabic information retrieval systems fail on a substantial proportion of individual queries even on standard benchmarks: aggregate scores conceal frequent cases in which no relevant passage is retrieved at a usable rank, and the generation stage of a RAG system cannot recover from such failures. These failures are concentrated where the query itself carries too little information—short queries suffer from information poverty, providing too few terms to discriminate among millions of passages, while the terms they do provide are subject to the vocabulary mismatch created by Arabic's root-and-pattern morphology, orthographic variation, and diglossia. Because this deficiency originates in the query rather than in the retriever or the index, it cannot be resolved by strengthening the retriever alone; it calls for a pre-retrieval intervention that enriches the query before it is submitted. LLM-based QE offers exactly such an intervention, yet its established techniques were developed for English using proprietary models of 175 billion or more parameters, leaving unanswered the questions on which Arabic deployment depends: whether generated expansions bridge or aggravate the Arabic vocabulary gap; whether an expansion that benefits dense semantic retrieval also benefits—or instead degrades, through dilution of the original query terms—sparse lexical retrieval, and whether that degradation can be remedied; and whether grounding the expansion in evidence retrieved from the target corpus produces more reliable enhancement than generating it from the model's parametric knowledge

alone. The specific problem addressed in this thesis is therefore how LLM-based QE should be constructed and deployed for Arabic information retrieval—how expansions are generated (blind or corpus-steered) and to which retriever within a hybrid sparse–dense architecture they are applied—so that retrieval effectiveness improves substantially over strong unenhanced baselines while relying only on openly available models of practical scale, among which the choice of generator is itself an open question. This problem motivates the central research question of this thesis: *To what extent can LLM-based QE—blind and corpus-steered—improve Arabic information retrieval across sparse, dense, and hybrid retrieval paradigms?* In answering it, this thesis develops and validates a pipeline that couples corpus-steered query expansion with asymmetric hybrid sparse–dense fusion, in which the expansion is applied to the sparse retriever only.

1.2 Objectives

In pursuit of the central research question stated in Section 1.1, the objectives of this thesis are as follows:

1. To establish independent dense and sparse retrieval baselines on the Arabic subset of the Multilingual Information Retrieval Across a Continuum of Languages (MIRACL) benchmark, quantifying the effectiveness of each paradigm and the complementarity between them using standard retrieval metrics—Normalised Discounted Cumulative Gain (NDCG), Recall, and Mean Reciprocal Rank (MRR).

2. To conduct a systematic error analysis of the baseline results—quantifying the overall failure rate, the effect of query length, and retrieval coverage, and characterising the linguistic failure patterns arising from Arabic morphological, orthographic and lexical variation—in order to identify the failure modes that QE must address and to guide the selection of the enhancement technique.

3. To adapt the Query2Doc technique for Arabic zero-shot application using openly available LLMs, including the engineering optimisations required for practical execution on freely available cloud graphics processing units (GPUs).

4. To conduct a standardised comparative evaluation of openly available LLMs spanning 2–8 billion parameters as Arabic query-expansion generators, identifying the most effective model under each retrieval paradigm, establishing whether Arabic-specialised models outperform multilingual models of comparable scale, and characterising the performance patterns observed across model families.

5. To analyse how LLM-generated query expansions interact with sparse versus dense retrieval, establishing whether QE strategies must be adapted to the retrieval paradigm.

6. To investigate query repetition as a remedy for the term-dilution degradation that pseudo-document concatenation induces in Best Matching 25 (BM25) sparse retrieval, determining the optimal repetition factor across the evaluated models.

7. To establish a hybrid sparse–dense fusion baseline without QE, quantifying the performance ceiling that any enhancement-based system must exceed.

8. To adapt and evaluate Corpus-Steered Query Expansion (CSQE) for Arabic retrieval, isolating the contributions of its corpus-grounded and blind expansion components through controlled ablation.

9. To determine the optimal placement of query expansion within the hybrid pipeline by evaluating retriever-specific application strategies—expanding the sparse retriever, the dense retriever, or both—and to explain the observed behaviour through a per-query error analysis of the final system.

1.3 Thesis Layout

The remainder of this thesis is organised as follows. Chapter 2 establishes the theoretical foundations required to understand the research. It introduces the core concepts of LLMs, RAG, information retrieval methods (sparse, dense, and hybrid), and QE techniques. The mathematical formulations underlying the retrieval and evaluation methods are presented. The selection of the evaluation dataset from among candidate Arabic retrieval and question-answering benchmarks is then justified, followed by detailed descriptions of all models used in the experimental work. The chapter closes with a review of related literature, the identification of the research

gap addressed by this thesis, and a summary of the chapter. Chapter 3 presents the methodology employed throughout the research. It describes the dataset—the Arabic subset of the MIRACL benchmark—together with the hardware and software environment and the evaluation metrics. The baseline implementation for both dense (Multilingual Dense Passage Retriever, mDPR) and sparse (BM25S) retrieval is detailed, followed by the error analysis methodology that guided technique selection. The adaptation of Query2Doc for Arabic zero-shot application is described, including modifications from the original paper and engineering optimisations. The methodology of the comparative model evaluation is then described, covering the model selection criteria, the standardised generation protocol, the temperature setting, the model-specific technical issues encountered, and the quantisation strategy adopted for models exceeding the available memory. The chapter then presents the query repetition methodology for resolving BM25 term-dilution, the hybrid BM25–Dense fusion methodology, and the CSQE pipeline including its component ablation design and retriever-specific application strategy. The chapter concludes with the per-query error analysis methodology. Chapter 4 reports the results and discussion in zigzag correspondence with the methodology. Baseline performance is analysed, error analysis findings are presented, and Query2Doc results are reported for both dense and sparse retrieval. The comprehensive model comparison leaderboards are presented and analysed, including the examination of dropped models, followed by the cross-cutting findings drawn from that comparison. The chapter then reports the expanded experimental results: the query repetition sweep across nine models, hybrid retrieval fusion configurations, the main CSQE results for both retrievers together with the component ablation and alpha sweep, CSQE combined with hybrid fusion across three retriever assignment configurations, the overall system progression table, and a per-query error analysis decomposing the sources of improvement and regression. The chapter closes with a consolidated summary of all experiments. Chapter 5 presents the overall conclusions, discusses the challenges encountered during the research, and provides recommendations for future work in Arabic QE for RAG systems.

Chapter 2

Theoretical Background and Literature Review

This chapter establishes the theoretical foundations and reviews the current state-of-the-art relevant to the research presented in this thesis. Section 2.1 introduces the core concepts of LLMs, RAG systems, information retrieval methods, QE techniques, and the specific challenges posed by the Arabic language. Section 2.2 presents the mathematical formulations underlying the retrieval and evaluation methods employed. Section 2.3 surveys candidate Arabic retrieval benchmarks and motivates the selection of the MIRACL Arabic dataset. Section 2.4 introduces the models utilized in the experimental work, with their full descriptions given in Appendix A. Finally, Section 2.5 surveys the related literature and identifies the research gap addressed by this thesis.

2.1 Theoretical Background

2.1.1 LLMs and the Transformer Architecture

LLMs are deep neural networks trained on vast corpora of text data to model the statistical properties of natural language. The modern era of LLMs was initiated by the introduction of the Transformer architecture by Vaswani et al. [1], which replaced recurrent processing with a self-attention mechanism that computes relationships between all positions in a sequence simultaneously.

The Transformer architecture consists of two main components: an encoder that processes input sequences into contextual representations, and a decoder that generates output sequences autoregressively. The core innovation is the *scaled dot-product attention* mechanism, which computes attention weights between query (Q), key (K), and value (V) matrices as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{2.1}$$

where d_k is the dimensionality of the key vectors. Multiple attention heads are employed in parallel, allowing the model to attend to information from different representation subspaces.

Two dominant architectural paradigms have emerged from the original Transformer design. *Encoder-only* models such as Bidirectional Encoder Representations from Transformers (BERT) process entire sequences bidirectionally and are well-suited for classification and retrieval tasks. *Decoder-only* models, which underpin most modern LLMs, generate text autoregressively by predicting each token conditioned on all preceding tokens.

The development of LLMs typically involves two stages. During *pre-training*, the model is trained on trillions of tokens from diverse text sources using self-supervised objectives such as next-token prediction. This stage imbues the model with broad linguistic and world knowledge. During *instruction tuning* (also called Supervised Fine-Tuning, or SFT), the pre-trained model is further trained on curated prompt-response pairs to follow user instructions. Alignment techniques such as Direct Preference Optimization (DPO) and Reinforcement Learning from Human Feedback (RLHF) are subsequently applied to improve helpfulness and safety [2].

A fundamental limitation of LLMs is that their knowledge is *parametric*—encoded statically in model weights during training. This makes the knowledge prone to becoming outdated, incomplete for specialized domains, and susceptible to *hallucination*: the generation of fluent but factually incorrect text. The model cannot distinguish between information it has reliably learned and information it is confabulating, nor can its knowledge be updated without costly retraining [3].

2.1.2 RAG

RAG was introduced by Lewis et al. [3] to address the limitations of purely parametric language models. RAG augments an LLM with access to an external knowledge base by incorporating a retrieval step before generation, thereby grounding the model's outputs in verifiable sources.

The standard RAG architecture consists of two primary components:

- **The Retriever:** Given a user query q, the retriever searches a knowledge base $\mathcal{D} = \{d_1, d_2, \ldots, d_N\}$ to identify a set of relevant passages $\{d_1, d_2, \ldots, d_k\}$, ranked by relevance to q. Retrieval methods are discussed in detail in Section 2.1.3.
- **The Generator:** An LLM that produces an answer a conditioned on both the original query q and the retrieved passages. The retrieved context provides factual grounding, reducing the likelihood of hallucination and enabling the model to cite sources.

The RAG pipeline operates sequentially: (1) the user submits a query q; (2) the retriever identifies the top-k relevant passages from the knowledge base; (3) the generator produces an answer conditioned on the query and retrieved passages; and (4) the system returns the generated answer, optionally with source citations. This architecture effectively separates knowledge storage (in the external corpus) from language generation (in the LLM), allowing the knowledge base to be updated independently without retraining the model.

The quality of the final generated answer is fundamentally bounded by the quality of the retrieved passages. If the retriever fails to surface relevant documents, the generator cannot compensate for this deficiency regardless of its capabilities. This *retrieval bottleneck* motivates the investigation of techniques to improve retrieval effectiveness, which is the central focus of this thesis. Figure 2.1 illustrates the architecture of the standard RAG framework described above.

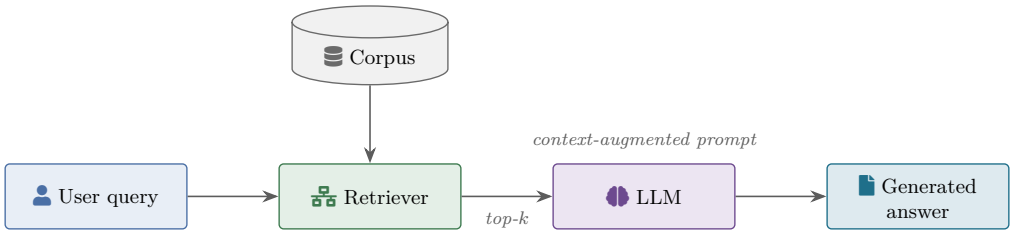

Figure 2.1: High-level architecture of a RAG system. A user query is sent to a retriever that consults the corpus and returns the top-k passages, which are passed alongside the query to an LLM to produce a context-augmented answer.

2.1.3 Information Retrieval Methods

Information retrieval methods can be broadly categorized into sparse, dense, and hybrid approaches, each with distinct strengths and limitations.

2.1.3.1 Sparse Retrieval

Sparse retrieval methods represent queries and documents as high-dimensional sparse vectors, where each dimension corresponds to a term in the vocabulary and the value reflects the term's importance. The most widely used sparse retrieval method is BM25, a probabilistic ranking function based on the bag-of-words model [4]. BM25 computes relevance scores using term frequency, inverse document frequency, and document length normalization. The mathematical formulation of BM25 is presented in Section 2.2.1.

Sparse methods excel at exact term matching and are computationally efficient, requiring no neural network inference. However, they suffer from the *vocabulary mismatch problem*: if the query and document use different terms to express the same concept, sparse methods fail to identify the relevance. This limitation is particularly acute for morphologically rich languages such as Arabic, where a single concept may be expressed through numerous surface forms.

2.1.3.2 Dense Retrieval

Dense retrieval methods address the vocabulary mismatch problem by encoding queries and documents into continuous vector representations (embeddings) within a shared semantic space. Karpukhin et al. [5] introduced Dense Passage Retrieval (DPR), which employs a bi-encoder architecture: two independent BERT-based encoders produce fixed-size dense vectors for queries and passages respectively. Relevance is computed via vector similarity (typically dot product or cosine similarity, as formulated in Section 2.2.2) between the query embedding and passage embeddings.

Dense retrievers are trained on relevance-annotated data using contrastive learning objectives, where the model learns to place relevant query-passage pairs closer together in the embed-

ding space while pushing irrelevant pairs apart. At inference time, approximate nearest neighbor search algorithms (e.g., Facebook AI Similarity Search, FAISS) enable efficient retrieval over millions of passages.

While dense retrieval captures semantic relationships that sparse methods miss, it can struggle with rare terms, entity names, and highly specific queries where exact lexical matching is important [6].

2.1.3.3 Hybrid Retrieval

Hybrid retrieval combines sparse and dense methods to leverage their complementary strengths. Sparse retrieval excels at exact matching and handles rare terms well, while dense retrieval captures semantic similarity and handles paraphrasing. Two fusion strategies are employed in this thesis: Reciprocal Rank Fusion (RRF) [7], which combines ranked lists by summing inverse rank scores and requires no score normalisation, and score-level interpolation (Convex Combination, CC), which computes a weighted sum of min-max normalised sparse and dense scores. The formal definitions of both fusion functions are given with the other scoring functions in Section 2.2.3.

2.1.4 QE Techniques

QE refers to a family of techniques that modify the user's query before it is submitted to the retriever, with the goal of improving retrieval effectiveness. Song and Zheng [2] categorize query optimization methods into four atomic operations: *expansion*, *decomposition*, *disambiguation*, and *abstraction*. Each operation is described below; this thesis focuses on query expansion, the family most directly suited to the vocabulary-mismatch and information-poverty problems that dominate Arabic retrieval.

2.1.4.1 Query Expansion

Query expansion broadens the scope of a query by adding related terms, context, or generated text. Traditional query expansion methods such as Relevance Model 3 (RM3) extract expansion

terms from pseudo-relevant documents retrieved in a first pass. Modern approaches leverage LLMs to generate expansion content, which can take several forms:

- **HyDE:** Proposed by Gao et al. [8], HyDE instructs an LLM to generate a hypothetical document that would answer the query. This generated document is then encoded using an unsupervised contrastive encoder, and the resulting embedding is used for retrieval. The encoder acts as a lossy compressor that filters out hallucinated details while retaining relevant semantic features.
- **Query2Doc:** Introduced by Wang et al. [6], this method generates a pseudo-document using an LLM and concatenates it with the original query to form an enriched query representation. For sparse retrieval, the original query is repeated multiple times before concatenation to prevent term dilution by the longer pseudo-document.
- **GRF:** Proposed by Mackie et al. [9], GRF uses an LLM to generate diverse text (keywords, essays, facts) relevant to the query. The generated text is treated as a feedback set from which expansion terms are extracted using a probabilistic model.

2.1.4.2 Query Decomposition

Query decomposition breaks a complex or multi-hop query into simpler sub-queries that can be retrieved separately and recombined. Chan et al. [10] train a language model (RQ-RAG) to decompose, rewrite, and disambiguate queries on demand, improving retrieval for multi-hop question answering (QA) where a single retrieval pass is insufficient.

2.1.4.3 Query Disambiguation

Query disambiguation reformulates an ambiguous or poorly phrased query into a clearer, more retrievable form without necessarily adding new terms. The Rewrite-Retrieve-Read framework [11] employs a trainable language model to rewrite queries before retrieval, optimising the rewriter using reinforcement-learning signals from the downstream reader's performance.

2.1.4.4 Query Abstraction

Query abstraction steps back from a specific query to a more general question whose answer is easier to retrieve and which then guides retrieval for the original query. Zheng et al. [12] introduced *step-back prompting*, in which the model first derives a higher-level concept or principle before answering, improving retrieval and reasoning on knowledge-intensive tasks.

Among these four families, query expansion is adopted in this thesis because it directly addresses the dominant Arabic retrieval failure modes (vocabulary mismatch and short, information-poor queries) while remaining model-agnostic and requiring no task-specific training.

These QE techniques operate as modular layers that can be added to existing RAG systems without modifying the retriever index or the generator, making them particularly attractive for practical deployment. Table 2.1 summarises the query-enhancement literature reviewed for this thesis, organised by method family.

Table 2.1: Query-enhancement papers reviewed for this thesis, with method, evaluation dataset, and headline result.

Paper	Year	Method	Dataset	Key result
Query2Doc (Wang et al.)	2023	LLM pseudo-document concatenation	MS MARCO + BEIR	+8 NDCG@10 on MS MARCO with GPT-3
HyDE (Gao et al.)	2022	LLM hypothetical document	TREC-DL / BEIR	strong zero-shot improvements
GAR (Mao et al.)	2021	LLM generates answer/context for query rewriting	open-domain QA	competitive with supervised reranking
GRF (Mackie et al.)	2023	Generative Relevance Feedback for dense	TREC-DL	~+10 MAP; fusion best
CSQE (Lei et al.)	2024	Corpus-Steered Query2Doc	TREC-DL	+30% MAP over BM25
MuGI (Zhang et al.)	2024	Multi-Text Generation Integration	BEIR	best QE practices study
ThinkQE (Lei et al.)	2025	Iterative chain-of-thought expansion	TREC-DL	state of the art on multi-step queries
Exp4Fuse (Liu et al.)	2025	Original + expanded query via modified RRF	multiple	nearest prior art to thesis
DAPR (Wang et al.)	2024	Dense Approach via Pre-trained Retrievers	academic IR	context-rich expansion
AQE (Yang et al.)	2025	Aligned Query Expansion via LLM alignment	academic IR	efficient at scale
KAR (Xia et al.)	2025	Knowledge-Aware Query Expansion	textual + relational retrieval	strong on relational
Song & Zheng (survey)	2024	Query optimisation taxonomy (4 operations)	—	defines expansion, decomposition, disambiguation, abstraction
QE survey (arXiv 2509.07794)	2025	Comprehensive QE survey in the LLM era	—	recent QE families

2.1.5 Arabic Language Processing Challenges

The Arabic language presents several characteristics that compound the retrieval challenges described in the preceding sections.

2.1.5.1 Morphological Richness

Arabic employs a root-and-pattern morphological system in which words are derived from typically tri-consonantal roots through templatic patterns and extensive affixation. A single root can yield dozens of surface forms through prefixes, suffixes, infixes, and clitics. For example, the root *k-t-b* (ك-ت-ب, "to write") generates forms such as *kitaab* (book), *kaatib* (writer), *maktaba*

(library), and *maktuub* (written). This morphological productivity creates a severe vocabulary mismatch problem, as queries and relevant documents may use different inflected forms of the same root.

2.1.5.2 Diglossia

Arabic exhibits a diglossic situation in which MSA is used in formal writing, education, and media, while various regional dialects (Egyptian, Levantine, Gulf, Maghrebi, and others) are used in daily speech and informal written communication. Knowledge bases are predominantly composed of MSA text, whereas user queries—particularly in conversational settings—may be formulated in dialectal Arabic, creating a systematic gap between query and document vocabularies [13].

2.1.5.3 Orthographic Variations

Arabic script permits multiple orthographic representations for certain characters, leading to inconsistencies even within MSA. Common variations include the different forms of Hamza (ء، أ، إ، ؤ، ئ), Alif (ا، آ، أ), and Ya/Alif Maqsura (ى، ي). These variations can cause both sparse and dense retrieval methods to treat orthographic variants as distinct terms.

2.1.5.4 Diacritical Marks

Arabic diacritics (*tashkeel*) disambiguate pronunciation and meaning but are typically omitted in modern texts. This omission increases lexical ambiguity, as identical consonantal skeletons may correspond to multiple words with different meanings.

These characteristics collectively create what may be termed a “morphological gap” between user queries and knowledge base documents, which is distinct from and often more severe than the vocabulary mismatch observed in English information retrieval.

2.2 Mathematical Models

This section presents the mathematical formulations of the retrieval scoring functions and evaluation metrics employed in this thesis.

2.2.1 BM25 Scoring Function

The BM25 ranking function [4] computes the relevance score of a document d with respect to a query $q = \{q_1, q_2, \ldots, q_n\}$ consisting of n terms:

$$\text{BM25}(q, d) = \sum_{i=1}^{n} \text{IDF}(q_i) \cdot \frac{f(q_i, d) \cdot (k_1 + 1)}{f(q_i, d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{\text{avgdl}}\right)} \tag{2.2}$$

where $f(q_i, d)$ is the term frequency of query term q_i in document d, $|d|$ is the length of document d in tokens, avgdl is the average document length across the collection, k_1 controls the term frequency saturation (typically $k_1 = 1.2$), and b controls the degree of document length normalization (typically $b = 0.75$).

The Inverse Document Frequency (IDF) component is computed as:

$$\text{IDF}(q_i) = \ln\left(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5} + 1\right) \tag{2.3}$$

where N is the total number of documents in the collection and $n(q_i)$ is the number of documents containing term q_i. The IDF assigns higher weight to rarer terms, which are generally more discriminative for retrieval.

2.2.2 Dense Retrieval and Cosine Similarity

In dense retrieval, queries and documents are encoded into dense vector representations by neural encoder models. Given a query encoder E_q and a document encoder E_d, the relevance score

between query q and document d is computed as the cosine similarity between their embedding vectors:

$$\text{sim}(q, d) = \cos(\mathbf{e}_q, \mathbf{e}_d) = \frac{\mathbf{e}_q \cdot \mathbf{e}_d}{\|\mathbf{e}_q\| \, \|\mathbf{e}_d\|} \tag{2.4}$$

where $\mathbf{e}_q = E_q(q)$ and $\mathbf{e}_d = E_d(d)$ are the embedding vectors for the query and document respectively, $\mathbf{e}_q \cdot \mathbf{e}_d$ denotes the dot product, and $\| \cdot \|$ denotes the Euclidean norm. When embedding vectors are L_2-normalized, cosine similarity is equivalent to the dot product, and retrieval can be performed efficiently using Maximum Inner Product Search (MIPS) algorithms.

2.2.3 Hybrid Fusion Functions

Hybrid retrieval (Section 2.1.3.3) combines the ranked lists of a sparse and a dense retriever using one of two fusion functions. RRF [7] combines ranked lists by summing inverse rank scores:

$$s_{\text{RRF}}(d) = \sum_{r \in \mathcal{R}} \frac{1}{k + \text{rank}_r(d)} \tag{2.5}$$

where $\mathcal{R}$ is the set of ranked lists being fused and k is a smoothing constant (typically $k = 20$) that reduces the impact of top-ranked documents. RRF is parameter-free in the sense that it does not require score normalisation, making it robust to score distribution differences between sparse and dense retrievers.

Score-level interpolation via CC computes a weighted sum of normalised retriever scores:

$$s_{\text{CC}}(d, q) = \alpha \cdot \hat{s}_{\text{Dense}}(d, q) + (1 - \alpha) \cdot \hat{s}_{\text{BM25}}(d, q) \tag{2.6}$$

where $\hat{s}$ denotes min-max normalised scores and $\alpha \in [0, 1]$ controls the relative contribution of the dense retriever. CC is sensitive to the normalisation strategy and score scale differences between the two systems.

2.2.4 Evaluation Metrics

Three standard information retrieval metrics are used throughout this thesis to evaluate retrieval performance.

2.2.4.1 Recall at k (Recall@k)

Recall at k measures the proportion of all relevant documents that appear within the top-k retrieved results:

$$\text{Recall@}k = \frac{|\{\text{relevant documents}\} \cap \{\text{top-}k \text{ retrieved}\}|}{|\{\text{relevant documents}\}|} \tag{2.7}$$

Recall@k captures the retriever's ability to surface relevant documents within the top-k positions. In RAG systems, Recall@k at the retrieval stage directly bounds the information available to the generator.

2.2.4.2 NDCG@k

NDCG measures ranking quality by considering both the relevance of retrieved documents and their positions. Documents at higher ranks contribute more to the score, reflecting the intuition that users examine top-ranked results more carefully.

The Discounted Cumulative Gain (DCG) at position k is defined as:

$$\text{DCG@}k = \sum_{i=1}^{k} \frac{2^{\text{rel}_i} - 1}{\log_2(i+1)} \tag{2.8}$$

where rel_i is the relevance grade of the document at position i. The ideal DCG (IDCG@k) is computed by sorting all relevant documents by decreasing relevance. NDCG@k is the ratio:

$$\text{NDCG@}k = \frac{\text{DCG@}k}{\text{IDCG@}k} \tag{2.9}$$

NDCG@k ranges from 0 to 1, where 1 indicates a perfect ranking.

2.2.4.3 MRR

MRR measures the average of the reciprocal ranks of the first relevant document across a set of queries Q:

$$\text{MRR} = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i} \tag{2.10}$$

where rank_i is the position of the first relevant document for the i-th query. MRR emphasizes the ability to place at least one relevant document at the top of the ranked list.

2.3 Evaluation Dataset Selection

The choice of evaluation benchmark is central to a QE study, because QE is only meaningful where the benchmark exposes a genuine query–document matching problem. Several Arabic and multilingual benchmarks were therefore surveyed and assessed against five criteria: (1) a *retrieval-first* design with explicit relevance judgments, rather than a reading-comprehension task in which retrieval is incidental; (2) *natural, native-authored* queries rather than machine-translated or template-generated ones, so that the query–document mismatch is linguistically authentic; (3) coverage of *MSA* over an open-domain corpus; (4) sufficient *scale* for stable evaluation while remaining tractable on the available compute; and (5) *open availability* for reproducibility. Table 2.2 summarises the principal candidates.

Table 2.2: Arabic retrieval and QA datasets surveyed for benchmark selection. "Suitability" refers to fitness for studying QE for retrieval.

Dataset	Ar. queries	Variety	Query origin	Primary challenge	Suitability
MIRACL [14]	∼8,700	MSA	Native (ad-hoc)	Retrieval recall	High
ArabicaQA [15]	∼89,000	MSA	Crowd (paraphrased)	Scale / difficulty	High
TyDi QA [16]	∼23,000	MSA	Native (info-seeking)	Morphology / typology	Medium–High
MultiNativQA [17]	∼12,000	MSA + dialect	Native (cultural)	Dialect / cultural gap	Medium–High
Mintaka [18]	∼20,000	MSA (transl.)	Translated	Multi-hop reasoning	Medium
ARCD [19]	∼1,400	MSA	Crowd	Small scale	Low

2.3.1 Selected Benchmark

MIRACL [14] was selected as the primary evaluation benchmark. Unlike reading-comprehension datasets, it is purpose-built for ad-hoc retrieval and provides human relevance judgments produced by native-speaker annotators, including *hard negatives*—passages that share surface terms with the query but are not relevant. This makes it a direct test of whether an enhanced query can separate the truly relevant passage from lexically similar distractors, which is precisely the behaviour QE aims to improve. Its queries are written by native speakers from genuine information needs, avoiding the priming and translation artefacts that affect translated datasets; its corpus is the standard open-domain Arabic Wikipedia; and its development set of 2,896 queries is large enough for stable evaluation yet tractable under the compute constraints of this work.

2.3.2 Alternatives Considered

ArabicaQA [15] offers an order of magnitude more queries and an explicit "easy"/"difficult" split by retrieval rank, making it attractive for stress-testing enhancement on hard queries; TyDi QA [16], whose information-seeking annotation methodology MIRACL builds upon, contributes highly natural queries; and MultiNativQA [17] uniquely includes regional Arabic dialects, making it the natural vehicle for evaluating dialect-aware query rewriting. Mintaka [18] targets multi-hop reasoning suited to query decomposition but is professionally *translated* into Arabic, introducing translationese. Machine-translated (Arabic-SQuAD) and template-

generated (ACQAD) resources were excluded outright, as their query–document mismatch is an artefact of translation or generation rather than authentic Arabic phrasing, which would reward enhancement methods for the wrong reasons.

2.3.3 Limitation

MIRACL covers only MSA over Wikipedia and therefore does not exercise dialectal QE. This scoping choice is revisited as a direction for future work in Chapter 5, where dialect-rich resources such as MultiNativQA are proposed for evaluating dialect-aware enhancement.

2.4 Models Used in Experiments

This section describes the language models employed in the experimental work of this thesis. The models were selected based on three primary criteria: (1) strong Arabic language support, either through specialized Arabic training or robust multilingual coverage; (2) compatibility with consumer-grade GPU hardware, specifically the NVIDIA T4 (15 GB of video random-access memory, VRAM) available through Google Colab and the NVIDIA A100 (40 GB VRAM); and (3) open weights under a licence permitting research use, whether permissive or non-commercial, to ensure reproducibility [2]. Table 2.3 summarises the ten LLMs evaluated.

The ten models divide into two groups. The **Arabic-specialized** group—Falcon-H1-Arabic-3B, Jais-2-8B, ALLaM-7B and SILMA Kashif-2B—comprises models whose developers list Arabic as a primary training language, whether through an Arabic-centric vocabulary, Arabic-weighted pre-training, or task-specific optimisation for Arabic retrieval. The **multilingual** group—Qwen 2.5 3B and 7B, Qwen3-4B and 8B, Gemma 3 4B-IT and Aya Expanse 8B—comprises general-purpose models that support Arabic among many other languages without special emphasis on it. Including both groups allows the thesis to test whether Arabic specialisation confers an advantage for QE over broad multilingual coverage, a question addressed in Section 4.5.3. Full descriptions of each model—architecture, training data, benchmark scores, licence and VRAM footprint—are given in Appendix A, and Table A.1 in Appendix A.1 sets the ten models side by side with their developer, architecture and licence.

Table 2.3: Open-source LLMs evaluated for Arabic QE in this thesis, ordered by parameter count.

Model	Family	Params (B)	Multilingual	Arabic-specialised	Quantisation
SILMA Kashif-2B	Silma AI	2	No	Yes (RAG)	FP16
Qwen 2.5 3B	Alibaba	3	Yes (29 langs)	No	FP16
Falcon-H1-3B	TII	3	Yes	Yes	BF16
Qwen3-4B	Alibaba	4	Yes (119 langs)	No	FP16
Gemma 3 4B	Google	4	Yes	No	FP16/BF16
Qwen 2.5-7B	Alibaba	7	Yes (29 langs)	No	FP16
ALLaM-7B (DROPPED)	NCAI	7	Yes	Yes	FP16
Jais-2-8B	G42	8	Yes	Yes	BF16
Qwen3-8B	Alibaba	8	Yes (119 langs)	No	FP16
Aya Expanse 8B	Cohere	8	Yes (101 langs)	No	4-bit NF4

2.4.1 Retrieval Models

2.4.1.1 mDPR

mDPR [5] is a bi-encoder dense retrieval model based on multilingual BERT. In this thesis, the pre-built mDPR index for MIRACL Arabic was used, which encodes the 2.1 million Arabic Wikipedia passages into dense vectors. mDPR was selected as the dense baseline because it was *not* fine-tuned on MIRACL training data, unlike models such as BGE-M3 and E5 which were partially trained on MIRACL. This choice provides a more realistic evaluation of QE effectiveness, as improvements cannot be attributed to the retriever having memorized the evaluation queries.

2.4.1.2 BM25S

BM25S [20] is a Python-native implementation of the BM25 algorithm that eliminates the Java dependencies required by traditional implementations such as Pyserini. In this thesis, BM25S was configured with $k_1 = 0.9$ and $b = 0.4$, matching the Pyserini default configuration used in the official MIRACL benchmark to ensure comparability. (The BM25S library defaults are

$k_1 = 1.5$, $b = 0.75$.) BM25S was selected over Pyserini for its simpler deployment pipeline and faster iteration speed, achieving 96% of the Pyserini BM25 baseline performance on the MIRACL Arabic evaluation set.

Throughout the remainder of this thesis, the term "BM25" refers to the BM25S implementation with the configuration above ($k_1 = 0.9$, $b = 0.4$), unless explicitly stated otherwise.

2.5 Related Work

This section reviews the published literature on QE for information retrieval and RAG systems, organised chronologically and thematically. The review progresses from foundational work on generative query expansion, which relied on large proprietary models, through modern approaches that demonstrate effectiveness at far smaller scale and ground the expansion in the target corpus, and concludes with Arabic-specific information retrieval research.

2.5.1 Foundational QE Research (2022–2023)

The foundational work on LLM-based QE was conducted using large proprietary models. Gao et al. [8] introduced HyDE, which demonstrated that an instruction-following LLM (InstructGPT, 175B parameters) could generate hypothetical documents for zero-shot dense retrieval, achieving performance comparable to supervised models without requiring any relevance labels. HyDE's key insight was that document-to-document similarity is more reliable than query-to-document similarity, and that even hallucinated documents capture useful relevance patterns when encoded through a dense bottleneck.

Wang et al. [6] extended this concept with Query2Doc, which generates pseudo-documents and concatenates them with the original query rather than replacing it. Using `text-davinci-003` (175B parameters) with few-shot prompting, Query2Doc achieved 3–15% improvement over BM25 and made sparse retrieval competitive with state-of-the-art dense methods. Critically, the authors found smaller models insufficient for quality query expansion at the time—a finding that has since been challenged by newer, more capable small models. For sparse retrieval,

the paper introduced query repetition (repeating the original query 5 times before concatenation) to prevent term dilution.

Mackie et al. [9] proposed GRF, which uses GPT-3 to generate diverse content types (keywords, essays, facts, news articles) as feedback for query expansion. GRF achieved 5–19% improvement in mean average precision (MAP) and 17–24% in NDCG@10 over BM25+RM3 baselines. In subsequent work [21], the authors extended GRF to dense and learned sparse retrieval models, demonstrating approximately 9–10% improvement in NDCG@20. A key finding was that GRF and traditional pseudo-relevance feedback are complementary: their fusion achieved the best Recall@1000 in 17 out of 18 experiments.

Ma et al. [11] introduced the Rewrite-Retrieve-Read framework, which employs a trainable T5-large model as a query rewriter optimized via reinforcement learning from the downstream reader's performance. This work demonstrated that small, trainable models can effectively improve the performance of large, frozen LLMs by operating at the query formulation stage.

A common characteristic of all foundational work is the reliance on 175B-parameter models (the GPT-3 family) for query expansion, with smaller models either untested or found inadequate.

2.5.2 Modern LLM-Based QE (2024–2025)

The period from 2024 to 2025 saw significant progress in demonstrating that moderately-sized open-source LLMs (7–8B parameters) can perform effective query expansion, overturning the assumption that 175B-scale models are necessary.

Lei et al. [22] proposed CSQE, which uses the target corpus to guide LLM-generated expansions. The CSQE pipeline operates in two stages. First, a first-pass sparse retrieval step retrieves a small set of top-k documents from the target corpus using the original query. These retrieved documents are then provided to the LLM alongside the query, instructing it to extract and synthesise topically relevant vocabulary and context grounded in the actual corpus content. This corpus-grounded expansion is combined with a blind expansion (standard Query2Doc) to produce the final enriched query. The key motivation is that corpus grounding anchors the

LLM's output to attested Wikipedia vocabulary, preventing the hallucination of plausible but factually incorrect entities that characterises blind generation for niche or ambiguous queries. Lei et al. reported that even a 7B-parameter model achieves a 30% improvement in MAP over BM25 on English benchmarks [22].

Zhang et al. [23] studied best practices for LLM-based query expansion and proposed MuGI (Multi-Text Generation Integration), which prompts an LLM to generate multiple pseudo-documents and integrates them with the original query. To prevent the lengthy generated text from overwhelming the original query terms in sparse retrieval, MuGI employs an adaptive query-repetition strategy that scales the number of query repetitions to the length of the generated text—the basis for the adaptive repetition parameter (β) adopted in this thesis (Section 3.6).

Several further studies have extended LLM-based QE along complementary axes. Xia et al. [24] proposed knowledge-aware query expansion, augmenting LLM-generated expansions with structured knowledge to improve both textual and relational retrieval. Yang et al. [25] demonstrated aligned query expansion, using LLM alignment to make query expansions more effective and efficient for information retrieval. Lei et al. [26] proposed ThinkQE, which refines query expansions through an evolving, reasoning-driven generation process. Zhang et al. [27] explored personalized, user-centric query expansion, reporting retrieval improvements on a personalization benchmark. Not all findings have been uniformly positive, however: Yoon et al. [28] caution that a substantial portion of the apparent gains from LLM-based query expansion on common benchmarks may stem from benchmark contamination ("knowledge leakage") rather than genuine retrieval improvement, underscoring the importance of careful evaluation.

Research on knowledge distillation further reduced the model size requirements. Young et al. [29] proposed GaQR, which distills query rewriting capabilities from GPT-3.5 into Llama2-7B via a three-phase pipeline (generation, verification, distillation), achieving substantially faster inference than chain-of-thought generation methods. Chan et al. [10] introduced RQ-RAG, which trains Llama2-7B to autonomously refine queries through rewriting, decomposition, and disambiguation, achieving state-of-the-art performance for 7B-parameter models on multi-hop QA tasks. Bhusal [30] further demonstrated that knowledge distillation from GPT-3.5 to Flan-T5-base (approximately 250M parameters) can produce effective query augmentation, suggesting a path toward extremely efficient deployment.

Zhang et al. [31] addressed query robustness with QE-RAG, a benchmark specifically designed to evaluate RAG system performance under query entry errors (typos, misspellings, visual similarities). Their findings demonstrated that QE techniques significantly improve retrieval robustness against noisy inputs—a characteristic analogous to the morphological and orthographic variations present in Arabic queries.

Collectively, this body of work demonstrates that the parameter requirement for effective query expansion has fallen sharply: from 175 billion in 2022–2023 to 7–8 billion in 2024–2025, with further compression possible via knowledge distillation. However, a critical gap remains: *LLM-based QE has not been evaluated for monolingual Arabic retrieval, and the question of which retriever within a heterogeneous sparse–dense hybrid should receive the expansion has not been addressed in any language*. The closest concurrent work, by Macmillan-Scott et al. [32], applies sub-7B models (including Gemma 3 4B) to the distinct *cross-lingual* Arabic retrieval setting; notably, it independently corroborates several observations of this thesis, including that reasoning-style meta-text can pollute sparse retrieval and that zero-shot prompting is preferable for short queries.

2.5.3 Arabic Information Retrieval and RAG

Research on Arabic-specific RAG systems has begun to establish baselines, though work on QE for Arabic remains scarce.

Zhang et al. [14] introduced MIRACL, a large-scale multilingual retrieval benchmark covering 18 languages including Arabic. The Arabic subset contains over 2.1 million Wikipedia passages with human-annotated relevance judgments for 2,896 development queries and 799 test queries, making it the primary evaluation resource for Arabic retrieval research.

Alsubhi et al. [33] conducted the first systematic analysis of RAG pipeline components for Arabic. Testing chunking strategies, embedding models, rerankers, and generators, they found that sentence-aware chunking outperformed fixed-size and semantic chunking for Arabic text, and that multilingual embedding models (BGE-M3, multilingual E5-large) outperformed Arabic-specific models for retrieval. They also identified Aya-8B as a superior generator for semantically dense Arabic content compared to StableLM-1.6B.

El-Beltagy and Abdallah [13] explored RAG applications for Arabic, benchmarking twelve embedding models and five LLMs. Their work revealed that E5 and BGE models exhibited remarkable robustness to Arabic dialectal variations in queries, and that open-source models (Llama 3, Mistral 7B) performed comparably to GPT-3.5 Turbo for Arabic text generation in RAG contexts.

These studies establish that Arabic RAG systems can achieve reasonable performance with existing multilingual tools. However, they focus primarily on the retrieval and generation components of the pipeline. The query formulation stage—specifically, whether LLM-based QE techniques developed for English transfer effectively to Arabic—remains uninvestigated.

2.5.4 Research Gap

Taken together, the literature reviewed above leaves four gaps at the intersection of LLM-based QE and Arabic information retrieval.

a. **Language.** LLM-based QE has been developed and validated almost exclusively on English benchmarks, with recent work establishing that openly available LLMs of 7–8 billion parameters are capable expansion generators (Section 2.5.2). None of these studies addresses *monolingual* Arabic retrieval; the sole concurrent exception [32] addresses the distinct *cross-lingual* setting. Conversely, Arabic RAG research has established retrieval and generation baselines (Section 2.5.3) but has left the query-formulation stage—which operates upstream of both—uninvestigated.

b. **Retriever interaction.** The foundational techniques adapt the expansion to the retriever only at the surface: Query2Doc repeats the original query for sparse retrieval and concatenates it once for dense retrieval, but the expansion itself is generated identically for every paradigm, and no study establishes which retriever within a hybrid should receive it. Whether an expansion that benefits dense semantic matching also benefits sparse lexical matching, and whether the remedy proposed for sparse retrieval—repetition of the original query—generalises across generator models, has not been established for Arabic.

c. **Corpus grounding.** The original CSQE work [22] was evaluated exclusively on English benchmarks using English-centric models. Whether corpus grounding confers the same

benefit for Arabic—where BM25 homonym sensitivity may corrupt the first pass and misdirect the expansion—has not been investigated; moreover, although the blind component has been isolated in English, the corpus-grounded component has never been evaluated on its own in any language.

d. **Placement within a hybrid pipeline.** The interaction between corpus-steered expansion and hybrid BM25–Dense fusion remains underexplored. While Exp4Fuse [34] shows that fusing the original- and expanded-query result lists from a *single sparse* retriever outperforms using the expansion alone, the asymmetric assignment of expansion across retriever *types* in a heterogeneous dense–sparse hybrid—and its behaviour for Arabic—has not been studied. Whether applying query expansion to only one retriever in such a hybrid can outperform applying it to both is therefore an open question with practical implications for retrieval pipeline design.

Specifically, the following questions remain unanswered:

- To what extent can LLM-based QE—blind and corpus-steered—improve Arabic information retrieval across sparse, dense, and hybrid retrieval paradigms, when the expansion is generated zero-shot by openly available LLMs of 2–8 billion parameters?
- Does the Query2Doc technique, originally validated with proprietary 175-billion-parameter models on English text, transfer to Arabic when applied zero-shot?
- Which openly available LLMs (2–8 billion parameters) are the most effective Arabic expansion generators under each retrieval paradigm, and do Arabic-specialised models outperform multilingual models of comparable scale?
- How do LLM-generated Arabic expansions interact with sparse as opposed to dense retrieval, and must the expansion strategy therefore be adapted to the retrieval paradigm?
- Does the query-repetition remedy proposed for English transfer to Arabic, at what repetition factor, and does the optimal factor hold across generator models?
- Does grounding the expansion in documents retrieved from the target corpus yield more reliable enhancement for Arabic than blind expansion, and how do the corpus-grounded and blind components contribute individually?

- Within a hybrid sparse–dense architecture, to which retriever should the expansion be applied?

These questions are addressed in this thesis through a staged experimental programme on the MIRACL Arabic benchmark, progressing from independent sparse and dense baselines and their error analysis, through a standardised comparison of openly available LLMs (2–8 billion parameters) as expansion generators and a query-repetition sweep for sparse retrieval, to a hybrid sparse–dense fusion baseline and finally the adaptation of CSQE to Arabic and its placement within the hybrid pipeline, as described in the following chapter. Answering these questions additionally requires independent sparse and dense baselines, a diagnostic error analysis of their failures, and a hybrid fusion reference established without enhancement; these are treated as prerequisite objectives rather than as gaps in the literature.

Chapter 3

Methodology

This chapter presents the methodology employed throughout this research. The experimental work was conducted in a systematic, phased approach: establishing retrieval baselines (Section 3.2), performing error analysis to identify failure patterns (Section 3.3), implementing and adapting the Query2Doc QE technique (Section 3.4), and conducting a comprehensive model comparison across ten open-source LLMs (Section 3.5). The chapter then presents the query repetition methodology for resolving BM25 term-dilution (Section 3.6), the hybrid BM25–Dense fusion methodology (Section 3.7), the CSQE pipeline including its component ablation design and retriever-specific application strategy (Section 3.8), and the per-query error analysis methodology (Section 3.9). All experimental setup details, including dataset characteristics and hardware configurations, are described in Section 3.1. The results corresponding to each methodological phase are presented in Chapter 4.

3.1 Dataset and Experimental Setup

3.1.1 Dataset: MIRACL Arabic

All experiments in this thesis were conducted on the Arabic subset of the MIRACL benchmark [14]. MIRACL was selected as the primary dataset for three reasons: (1) it is retrieval-focused with human-annotated relevance judgements, (2) it contains natural query-document mismatch characteristic of real-world Arabic information needs, and (3) it is the standard benchmark for multilingual retrieval research, enabling comparison with published results.

The MIRACL Arabic corpus consists of 2,061,414 passages derived from 656,982 Arabic Wikipedia articles. Each passage is identified by a document identifier in the format `X#Y`, where `X` denotes the article and `Y` the passage number within that article. The development set, which

was used for all experiments reported in this thesis, contains 2,896 queries with 29,197 human-annotated relevance judgements. Each judgement assigns a relevance grade ranging from 0 (not relevant) to 3 (highly relevant), provided by native Arabic speakers. The corpus is written exclusively in MSA; consequently, the evaluation of dialectal Arabic queries falls outside the scope of this work.

The dataset was accessed directly from HuggingFace[1] without local download, as both the corpus and pre-built retrieval indices were available through the Pyserini toolkit [35].

3.1.2 Hardware and Software Environment

All experiments were conducted on Google Colab, utilising two GPU configurations depending on availability and model requirements:

- **NVIDIA T4 (15 GB VRAM):** Used for baseline experiments and models with fewer than 4 billion parameters in FP16 precision.
- **NVIDIA A100 (40 GB VRAM):** Used for models requiring 4-bit quantisation or exceeding 7 billion parameters, and for batch sizes larger than 8.

The software environment consisted of Python 3.10, PyTorch 2.0+, the Hugging Face Transformers library (v4.30+), Pyserini (v0.22.1) for index access, and FAISS for dense vector search. The complete implementation, including the experiment notebooks and the retrieval, fusion and evaluation modules, is publicly available; the repository is given in Appendix C.4, and the key components of the CSQE pipeline are reproduced in Appendix C. For models exceeding GPU memory in native precision, NF4 quantisation was applied using the `bitsandbytes` library [36], following the Quantised Low-Rank Adaptation (QLoRA) quantisation scheme.

3.1.3 Evaluation Metrics

Three standard information retrieval metrics were employed for evaluation, as defined in Section 2.2:

[1] `https://huggingface.co/datasets/miracl/miracl`

- **NDCG@10** (Equation 2.9): Measures the quality of ranking within the top-10 retrieved passages. This was treated as the primary metric throughout all experiments.
- **Recall@10** (Equation 2.7): Measures the proportion of all relevant passages that appear in the top-10 results.
- **MRR** (Equation 2.10): Measures how quickly the first relevant passage is retrieved.

Additionally, Recall@100 was computed to assess overall retrieval coverage. All metrics were calculated using the `pytrec_eval` library over the full development set of 2,896 queries.

3.1.4 Experimental Pipeline Overview

The experimental pipeline follows a two-phase architecture, as illustrated in Figure 3.1. In the first phase (QE), each query from the MIRACL development set is processed by a QE module, which may leave the query unchanged (baseline) or augment it using an LLM-generated pseudo-document. In the second phase (retrieval and evaluation), the enhanced query is passed to a retrieval system, and the top-100 results are evaluated against the MIRACL relevance judgements.

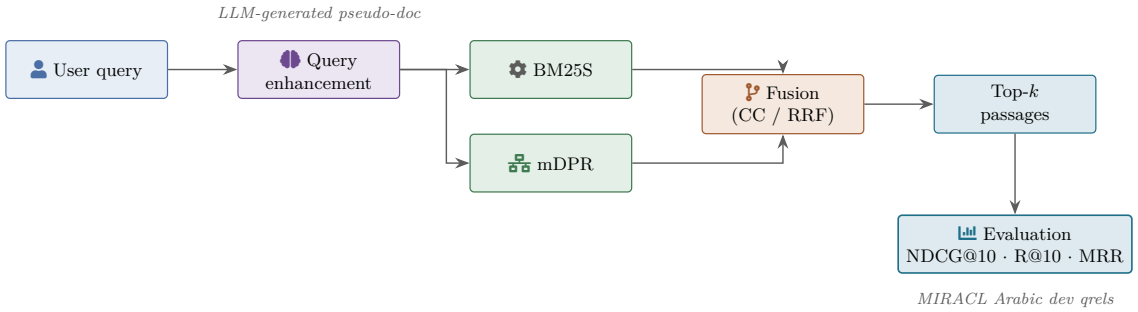

Figure 3.1: Overview of the experimental pipeline. A user query passes through the QE layer; BM25S and mDPR retrieve independently; their ranked lists are fused (CC or RRF) and the top-k passages are scored against the MIRACL Arabic dev qrels. The QE module is the variable component across experiments; the retrieval and evaluation components remain fixed.

A key design decision was to separate query generation from retrieval evaluation into two independent notebooks. The QE phase (which requires GPU for LLM inference) saves enhanced queries to disk as serialised Python objects. The evaluation phase then loads these pre-generated

queries and performs retrieval and metric computation. This two-notebook workflow enabled efficient reuse of enhanced queries across multiple retrieval configurations (Dense and BM25) without regenerating them.

3.2 Baseline Implementation

Two retrieval baselines were established to provide reference performance levels against which all QE experiments would be compared: a dense retrieval baseline using mDPR and a sparse retrieval baseline using BM25S. Both baselines used an `IdentityEnhancer` that returned queries unchanged.

3.2.1 Dense Retrieval Baseline (mDPR)

The dense retrieval baseline employed mDPR, described in Section 2.4.1.1. The mDPR encoder (`castorini/mdpr-tied-pft-msmarco`) was loaded on GPU, and queries were encoded in batches of 64. The pre-built FAISS index for MIRACL Arabic (5.47 GB, 768-dimensional embeddings) was accessed via Pyserini. For each query, the top-100 passages were retrieved using cosine similarity.

mDPR was pre-trained on the English Microsoft Machine Reading Comprehension (MS MARCO) dataset and was *not* fine-tuned on the MIRACL Arabic corpus, unlike models such as BGE-M3 and E5 which were partially trained on MIRACL. This choice mirrors a realistic deployment scenario in which an off-the-shelf multilingual retriever is improved through query-side interventions rather than corpus-specific fine-tuning, and ensures that observed improvements reflect QE rather than retriever memorisation of evaluation queries. Figure 3.2 illustrates the mDPR query-encoding pipeline.

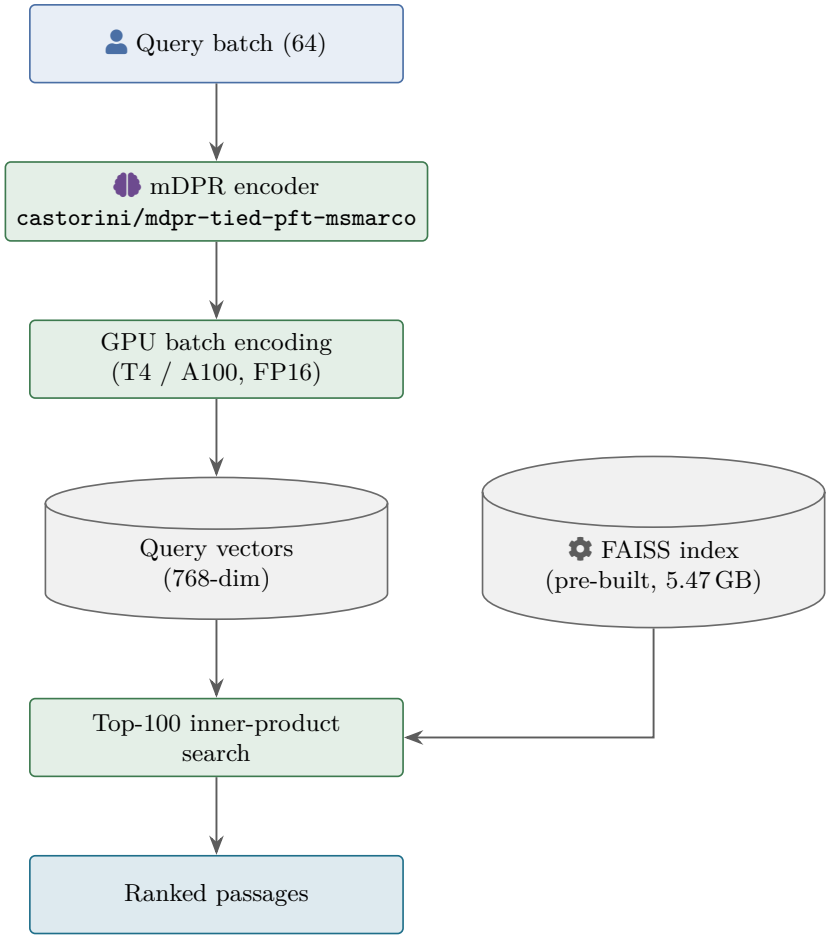

Figure 3.2: Dense retrieval pipeline using mDPR. Queries are encoded in batches on a single A100 GPU under FP16; the resulting 768-dimensional vectors are searched against a pre-built FAISS index of the corpus.

3.2.2 Sparse Retrieval Baseline (BM25S)

The sparse retrieval baseline employed BM25S [20], a pure-Python implementation of the BM25 algorithm (described in Section 2.1.3.1). BM25S was selected over the Pyserini-based BM25 implementation after encountering a blocking dependency conflict between Java 21 and Java 11 in the Google Colab environment. The BM25S library eliminates all Java dependencies while maintaining competitive performance.

The BM25S index was built from the MIRACL Arabic corpus with the following configuration:

- **Tokenisation:** Arabic stemming using PyStemmer with Natural Language Toolkit (NLTK)

Arabic stopwords (245+ words).

- **BM25 variant:** Lucene-style scoring with parameters $k_1 = 0.9$ and $b = 0.4$.
- **Index storage:** Google Drive (∼0.5 GB), loaded via symbolic links in Colab.

This configuration achieved 96% of the official MIRACL Pyserini baseline performance, with the 4% difference deemed acceptable given the benefits of a pure-Python pipeline that integrates directly with the QE modules. Figure 3.3 summarises the BM25S indexing pipeline.

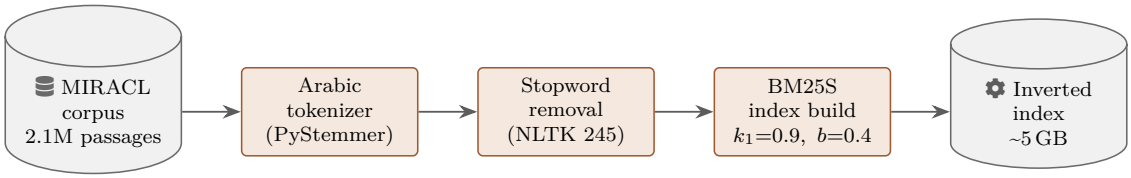

Figure 3.3: BM25S indexing pipeline applied to the MIRACL Arabic corpus. Passages are tokenised with PyStemmer, stopword-removed using 245 Arabic stopwords, then indexed under BM25S with $k_1 = 0.9$ and $b = 0.4$.

3.2.3 Baseline Comparison Rationale

Dense and sparse retrieval were tested *separately* rather than combined into a hybrid system. This decision, confirmed during the project planning phase, was made to provide clearer insight into where QE improvements originate. As discussed in Section 2.1.3, dense retrieval captures semantic similarity while sparse retrieval relies on exact term matching. Testing both baselines independently enables analysis of whether a given QE technique benefits semantic matching, lexical matching, or both.

3.3 Error Analysis Methodology

Before implementing QE techniques, a systematic error analysis was conducted on the dense retrieval baseline to identify the primary failure patterns and guide technique selection.

3.3.1 Quantitative Analysis Framework

A quantitative analysis was performed over all 2,896 queries in the MIRACL Arabic development set. Queries were segmented into three performance categories based on their individual NDCG@10 scores:

- **Failed:** NDCG@10 < 0.3
- **Mediocre:** $0.3 \leq$ NDCG@10 < 0.7
- **Successful:** NDCG@10 ≥ 0.7

These three categories describe each query's *absolute* retrieval quality in isolation. They should be distinguished from the *pairwise* classification scheme introduced later for the CSQE analysis (Section 3.9), which measures the *change* in NDCG@10 between two systems rather than absolute success, together with the regression sub-typing built upon it. Each scheme answers a different question—an absolute success rating versus the magnitude and direction of a system-to-system change. The 0.3 and 0.7 cut-offs adopted here are pragmatic boundaries chosen for interpretability rather than derived from a formal statistical criterion.

For each query, the following features were computed:

a. **Query length** (in words), measured after whitespace tokenisation.

b. **Score gap** between the top-ranked passage and the second-ranked passage, as a proxy for retriever confidence.

c. **First relevant document rank**, indicating how deep the retriever must search before finding a relevant passage.

d. **Coverage at multiple cut-offs** (top-10, top-20, top-50, top-100), measuring the proportion of queries with at least one relevant passage at each depth.

The Pearson correlation coefficient was computed between query length and NDCG@10 to quantify the relationship between query verbosity and retrieval performance.

3.3.2 Query Length Bucketing

Queries were grouped into three length buckets to analyse the effect of query length on retrieval performance:

- **Short:** 1–3 words (5.1% of queries)
- **Medium:** 4–8 words (86.2% of queries)
- **Long:** 9+ words (8.8% of queries)

The mean NDCG@10 was computed for each bucket, and the performance gap between short and long queries was used to quantify the impact of information poverty on retrieval effectiveness.

3.3.3 Failed Query Inspection

The 20 worst-performing queries (NDCG@10 = 0.000, indicating zero relevant passages in the top-10) were manually inspected to identify recurring linguistic patterns. Each query was categorised by question type (factoid, definition, temporal), the presence of named entities, diacritics, technical terminology, and spelling variations. These observations informed the selection of the first QE technique.

3.4 Query2Doc Implementation

Based on the error analysis findings (presented in Section 4.2), Query2Doc [6] was selected as the first QE technique. This section describes the Query2Doc approach, the modifications made for Arabic zero-shot application, and the implementation details.

3.4.1 Query2Doc Technique

Query2Doc, introduced by Wang et al. [6], uses an LLM to generate a pseudo-document—a short passage that attempts to answer the query. This pseudo-document is then concatenated with the original query to form an enhanced query. The rationale is that the pseudo-document introduces additional vocabulary, context, and semantic information that bridges the gap between the query and relevant passages.

As described in Section 2.1.4, Query2Doc falls within the category of LLM-based generative QE techniques. It differs from HyDE [8] in a critical way: while HyDE uses the generated document *alone* as the retrieval query, Query2Doc *concatenates* the generated document with the original query, preserving the original query terms. The Query2Doc generation pipeline used in this thesis is illustrated in Figure 3.4.

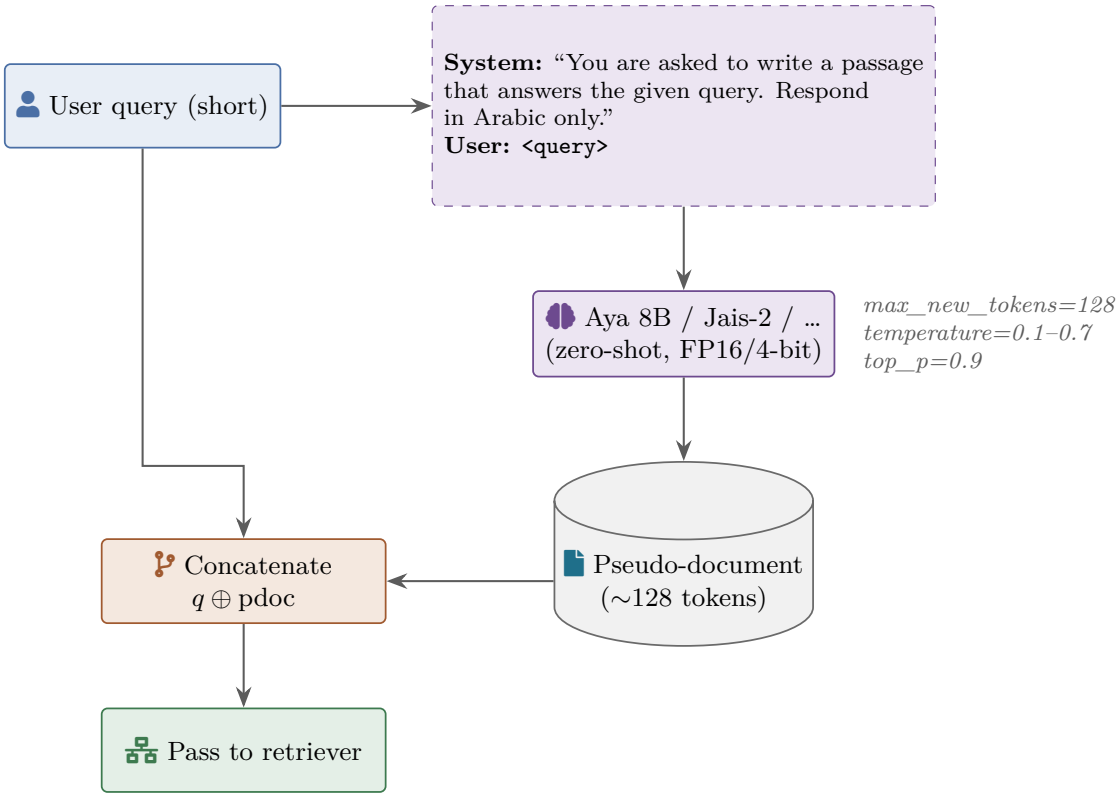

Figure 3.4: Query2Doc generation pipeline: the original query is wrapped in a fixed prompt template; an open-source LLM generates a short pseudo-document; the pseudo-document is concatenated with the query before being passed to the retriever.

3.4.2 Modifications from the Original Paper

Several modifications were made to adapt Query2Doc for this research context:

a. **Zero-shot prompting:** The original Query2Doc paper employed few-shot prompting with 4 demonstration examples using GPT-3 (`text-davinci-003`, 175B parameters). In this work, pseudo-document generation was performed zero-shot. The demonstrations of the original method are English query–passage pairs, and no curated set of Arabic demonstrations was available; constructing one would have introduced a design variable absent from the original method, and it is therefore left as a direction for future work (Section 5.3). Zero-shot prompting also keeps every prompt short, which matters when generation is batched over 2,896 queries for each of ten models within the available GPU budget (Section 3.4.4), and it allows each model's Arabic expansion capability to be assessed without task-specific demonstrations.

b. **Openly available models of 2–8 billion parameters:** Rather than using proprietary API-based models, all experiments used openly available models that can run on Google Colab free-tier GPUs. This approach ensures reproducibility and eliminates API costs.

c. **Arabic-only generation:** The system prompt explicitly instructs the model to respond in Arabic only, whereas the original paper targeted English queries.

d. **Retriever-specific concatenation:** For dense retrieval, simple concatenation was used: $q_{\text{enhanced}} = q \oplus d_{\text{pseudo}}$, where $\oplus$ denotes string concatenation. The original paper recommends repeating the query $n = 5$ times before concatenation for sparse retrieval to prevent term dilution, as discussed in Section 3.4.6.

3.4.3 LLM Configuration

The initial Query2Doc experiment used Qwen 2.5 3B Instruct (described in Appendix A) as the generation model. A single system prompt was used across all experiments: it instructs the model to write a passage answering the given query, forbids it from asking the user for clarification, and requires the response to be in Arabic. The prompt is reproduced verbatim in Appendix C.1.

The generation parameters were configured as summarised in Table 3.1.

Table 3.1: LLM generation parameters for the initial Query2Doc experiment.

Parameter	**Value**
Maximum new tokens	128
Temperature	0.7
Top-p (nucleus sampling)	0.9
Batch size	8
Precision	FP16

The maximum generation length was set to 128 tokens ($\sim$100 Arabic words), providing sufficient context for generating informative pseudo-documents while keeping per-query generation time practical.

3.4.4 Batch Processing Optimisation

A critical engineering challenge was generation throughput. Naïve sequential processing ($\sim$10 seconds per query) would require approximately 8 hours for the full development set. Three optimisations were implemented to achieve practical runtimes:

a. **Batch processing:** Multiple queries were tokenised with left-padding and processed simultaneously through the LLM, substantially reducing per-query generation time.

b. **Reduced token generation:** The maximum generation length was set to 128 tokens, keeping generation time practical while retaining expansion quality.

c. **Inference optimisations:** The model was set to evaluation mode (`model.eval()`), half-precision (`float16`) was used, and gradient computation was disabled (`torch.no_grad()`).

Together, these optimisations reduced full-set generation time to approximately 40 minutes per model on a T4 GPU, enabling practical iteration across multiple models and configurations.

3.4.5 Dense Retrieval with Query2Doc

For dense retrieval experiments, the enhanced query was formed by simple concatenation:

$$q_{\text{enhanced}} = q_{\text{original}} \oplus d_{\text{pseudo}} \tag{3.1}$$

The enhanced query was then encoded by the mDPR encoder and used for FAISS similarity search, following the same procedure as the baseline (Section 3.2.1). This approach leverages the semantic content of the pseudo-document to improve the query's embedding representation.

3.4.6 BM25 Retrieval with Query2Doc

For sparse retrieval, the same enhanced queries were evaluated against the BM25S index. Notably, the initial BM25 experiment used simple concatenation identical to the dense configuration, *without* implementing the query repetition strategy recommended by Wang et al. [6]. The original paper recommends:

$$q_{\text{enhanced}}^{\text{BM25}} = \underbrace{q \oplus q \oplus \cdots \oplus q}_{n \text{ times}} \oplus d_{\text{pseudo}} \tag{3.2}$$

where $n = 5$. This repetition preserves the importance of original query terms in the BM25 scoring function (Equation 2.2), preventing the pseudo-document from diluting the term weights of the original query. The omission of this strategy in the initial experiment, and its subsequent impact, is analysed in Section 4.3.2.

3.5 Model Comparison Methodology

Following the initial Query2Doc results with Qwen 2.5 3B, a comprehensive model comparison was conducted to identify the optimal LLM for Arabic query expansion. Ten openly available models were evaluated using the identical Query2Doc pipeline.

3.5.1 Model Selection Criteria

Models were selected based on three criteria: (1) Arabic language support, either through dedicated Arabic training or strong multilingual capabilities; (2) parameter count within the 2–8 billion range to remain executable on Google Colab GPUs; and (3) open weights under a licence permitting research use, whether permissive or non-commercial. The selected models span two categories, both of which are described in detail in Section 2.4:

- **Arabic-specialised:** Falcon-H1-3B (Section A.2.1), Jais-2-8B (Section A.2.2), ALLaM-7B (Section A.2.3), and SILMA Kashif-2B (Section A.2.4).
- **Multilingual:** Qwen 2.5 3B (Section A.3.1), Qwen 2.5 7B (Section A.3.2), Qwen3-4B (Section A.3.3), Qwen3-8B (Section A.3.4), Gemma 3 4B (Section A.3.5), and Aya Expanse 8B (Section A.3.6).

Table 3.2 summarises the configuration used for each model.

Table 3.2: Model configurations for the Query2Doc comparison experiments. All models used `max_new_tokens` = 128 and `top_p` = 0.9, with the exception of Qwen3-4B, which used the Qwen3 non-thinking sampling recommendation of `top_p` = 0.8 and `top_k` = 20.

Model	Params	Precision	Temp.	Batch	GPU
Qwen 2.5 3B	3B	FP16	0.7	8	T4
SILMA 2B	2B	FP16	0.1	16	A100
Falcon-H1 3B	3.15B	BF16	0.1	1*	A100
Qwen3-4B	4B	FP16	0.7	32	A100
Gemma 3 4B	4B	FP16	0.1	16	A100
Qwen 2.5 7B	7B	FP16	0.1	16	A100
ALLaM 7B	7B	FP16	0.7	—	A100
Jais-2 8B	8B	BF16	0.7	8	A100
Qwen3-8B	8B	FP16	0.1	16	A100
Aya Expanse 8B	8B	NF4	0.1	8	A100

*Forced to batch size 1 due to architecture-specific batching bug (see Section 3.5.4).

3.5.2 Standardised Evaluation Protocol

To ensure fair comparison, all models were evaluated using an identical protocol:

a. **Sanity check:** The first 5 queries were processed and the outputs were manually inspected to verify that the model (a) generates text in Arabic, (b) produces relevant content, and (c) does not exhibit degenerate behaviour (repetition, code output, or empty responses).

b. **Full generation:** All 2,896 queries were processed using the Query2Doc pipeline, and the enhanced queries were saved to disk.

c. **Dense evaluation:** Enhanced queries were evaluated against the mDPR dense retrieval baseline.

d. **BM25 evaluation:** The same enhanced queries were evaluated against the BM25S sparse retrieval baseline.

The work was divided between the two team members: one researcher tested Falcon-H1-3B, Jais-2-8B, ALLaM-7B, and Qwen3-4B, while the other tested SILMA Kashif-2B, Qwen 2.5-7B, Qwen3-8B, Gemma 3 4B, and Aya Expanse 8B. All experiments used the same codebase, system prompt, and evaluation pipeline to minimise confounding variables.

3.5.3 Temperature Selection

Temperature was not fixed at a single value across all models. Two considerations guided temperature selection:

a. **Model-recommended values:** Some models specify recommended generation parameters. Falcon-H1 recommends temperature 0.1, noting that higher values "largely drop performance." Qwen3 models require sampling (`do_sample=True`) to avoid infinite repetitions with greedy decoding.

b. **Empirical testing:** For SILMA Kashif-2B, two temperature values were compared (0.7 and 0.1). Temperature 0.1 yielded a 2.5% improvement in NDCG@10 over 0.7, leading to its adoption as the default for subsequent experiments where no model-specific recommendation existed.

3.5.4 Model-Specific Technical Issues

Four of the ten models required model-specific engineering workarounds before they could be run under the standardised protocol: a batching bug in Falcon-H1-3B's hybrid architecture, a precision and input-signature constraint in Jais-2-8B, reasoning-token suppression in the Qwen3 models, and a tokenizer artefact in ALLaM-7B. Each workaround, and the reason it was necessary, is documented in Appendix A.4; the ALLaM tokenizer artefact in particular is revisited in Section 4.4.3, where its effect on retrieval performance is quantified.

3.5.5 Quantisation Strategy

Models exceeding the available GPU memory in native precision were quantised using NF4 quantisation via the `bitsandbytes` library [36]. The quantisation configuration was:

- **Quantisation type:** NF4
- **Compute dtype:** BF16 (for matrix multiplication)
- **Double quantisation:** Enabled (quantises the quantisation constants for further memory savings)

This configuration was applied to Jais-2-8B (on T4 only; BF16 was used on A100) and Aya Expanse 8B (ALLaM-7B was run in full FP16 precision). Notably, Aya Expanse 8B achieved the best overall performance despite being quantised to 4-bit, suggesting that model architecture and training data quality are more significant factors than numerical precision for this task.

3.6 Query Repetition for Sparse Retrieval

The model comparison experiments (Section 4.4.2) revealed that single-pass Query2Doc ($n = 1$) degraded BM25 performance for six of nine models due to term dilution (Section 4.3.2.1).

The original Query2Doc paper [6] recommended concatenating $n = 5$ pseudo-documents with equal weighting, but did not investigate the sensitivity of the repetition factor to model size or pseudo-document length. Two solution families were evaluated to address this limitation.

3.6.1 Fixed Repetition (Query2Doc-style)

The original query is prepended n times before the pseudo-document, where n is a global hyperparameter applied identically to every query regardless of its length or the pseudo-document length. The enhanced query is assembled as:

$$q_{\text{rep}} = \underbrace{q \circ q \circ \cdots \circ q}_{n \text{ times}} \circ d_1 \circ d_2 \circ \cdots \circ d_k \tag{3.3}$$

where $\circ$ denotes string concatenation with a space separator and k is the number of pseudo-documents (for single-pass Query2Doc, $k = 1$). Fixed repetition was swept over $n \in \{1, 3, 5, 7, 10\}$.

3.6.2 Adaptive Repetition (MuGI-style)

Zhang et al. [23] proposed computing n per-query from the ratio of pseudo-document length to query length, controlled by a parameter β:

$$n(q, d, \beta) = \max\left(1, \left\lfloor \frac{|d|}{|q| \cdot \beta} \right\rfloor\right) \tag{3.4}$$

where $|q|$ and $|d|$ are the token lengths of the query and pseudo-document respectively. Larger β yields fewer repetitions; smaller β yields more. Adaptive repetition was swept over $\beta \in \{2, 4, 6\}$.

3.6.3 Motivation for the Adaptive Variant

The optimal number of repetitions depends on the length ratio between query and pseudo-document. A 3-word query paired with a 200-token pseudo-document requires more repetition than a 15-word query paired with a 100-token pseudo-document. The MuGI per-query adaptation removes this length sensitivity and, as reported in Section 4.6, produced the best results for the two largest models (Aya Expanse 8B and Jais-2 8B), where $\beta = 2$ was the winning configuration.

3.6.4 Sweep Design

Nine models were evaluated across eight configurations ($n \in \{1, 3, 5, 7, 10\}$ and $\beta \in \{2, 4, 6\}$), yielding 72 BM25 evaluations. All pseudo-document expansions had been generated during the model comparison phase (Section 3.5); no new LLM inference was required. The stored expansion files (serialised Python objects) were loaded and combined with varying repetition factors, then evaluated using `pytrec_eval` on the full MIRACL Arabic development set (2,896 queries).

3.7 Hybrid Retrieval Fusion

The baseline experiments (Section 4.1.1) confirmed that BM25 and mDPR exhibit complementary strengths: mDPR achieves superior ranking quality while BM25 provides broader lexical coverage. Hybrid retrieval fusion combines the ranked lists from both retrievers into a single ranking to exploit this complementarity. Two fusion methods, introduced in Section 2.1.3.3, were evaluated.

RRF (Equation 2.5) combines ranked lists by summing inverse rank scores with a smoothing constant k. Two values were tested: $k = 20$, following the standard practice established by Bruch et al. [7], and $k = 60$.

CC (Equation 2.6) computes a weighted sum of normalised retriever scores, where α con-

trols the relative contribution of the dense retriever. Before combination, the scores from each retriever were min-max normalised to $[0, 1]$ on a per-query basis to account for the different score scales of BM25 and mDPR. The weight α was swept over $\{0.1, 0.2, \ldots, 0.9\}$.

Both BM25S and mDPR retrieved the top-100 candidates independently using the original (unenhanced) queries. Fusion and re-ranking were performed in Python without additional GPU inference. The resulting hybrid baseline establishes the performance ceiling that all subsequent QE methods must surpass. The two fusion methods are illustrated side by side in Figure 3.5.

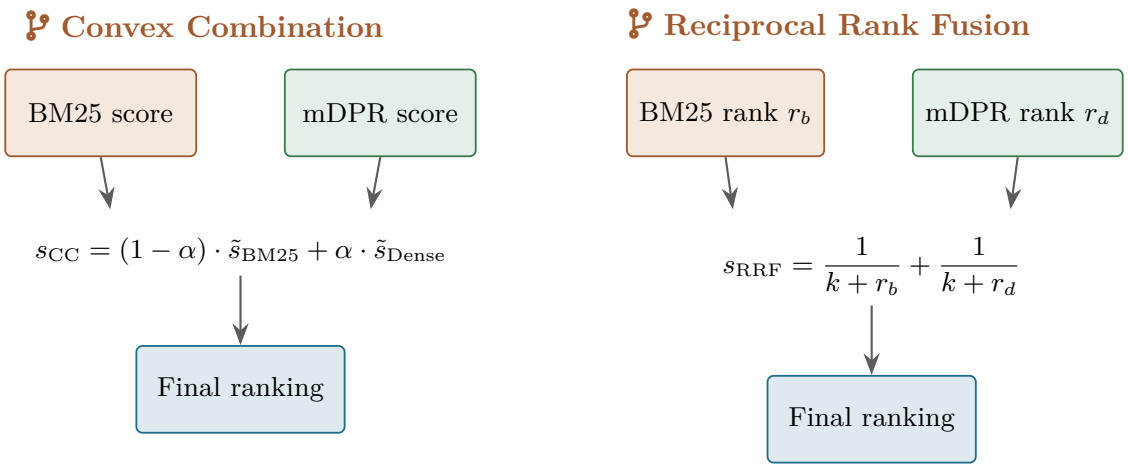

Figure 3.5: Two retriever-fusion methods evaluated in this thesis. CC (left) computes a weighted sum of normalised retrieval scores; RRF (right) sums the reciprocal of each retriever's rank for every document.

3.8 CSQE

CSQE, introduced by Lei et al. [22] and described in Section 2.5.2, extends blind Query2Doc by grounding LLM-generated expansions in first-pass retrieved documents. This section describes the CSQE pipeline as adapted for Arabic retrieval, the component ablation design, and the retriever-specific application strategy.

3.8.1 The CSQE Pipeline

The CSQE pipeline distinguishes itself from standard (blind) Query2Doc through a two-stage design:

a. **First-pass retrieval.** BM25S retrieves the top-$k = 5$ documents using the original query q.

b. **Corpus-grounded expansion.** Each retrieved document d_i is concatenated with q and passed to the LLM with a CSQE-specific system prompt instructing the extraction of topically relevant Arabic vocabulary grounded in the retrieved passage.

c. **Blind expansion.** The same query q is passed to the LLM without any retrieved context, following the standard Query2Doc procedure.

d. **Expansion assembly.** The corpus-grounded and blind samples are combined with a repeated original query:

$$q_{\text{CSQE}} = \underbrace{q \circ q \circ \cdots \circ q}_{\alpha \text{ times}} \circ c_1 \circ c_2 \circ b_1 \circ b_2 \tag{3.5}$$

where c_i denotes corpus-grounded expansion samples, b_i denotes blind expansion samples, α is the query repetition factor (Equation 3.3), and each expansion sample appears exactly once ($k = 1$ in MuGI notation).

The routine that assembles the expanded query from the four generated samples is reproduced in Appendix C.2, and the full set of hyperparameters is listed in Appendix C.3. The configuration used for the primary CSQE experiment was: $k = 5$ first-pass documents, 2 corpus-grounded samples, 2 blind samples, $\alpha = 4$, temperature 1.0, `max_new_tokens` = 128 per sample, using Aya Expanse 8B (Section A.3.6) in BF16 precision on an A100 GPU. Figure 3.6 illustrates the three-stage CSQE pipeline alongside a worked example.

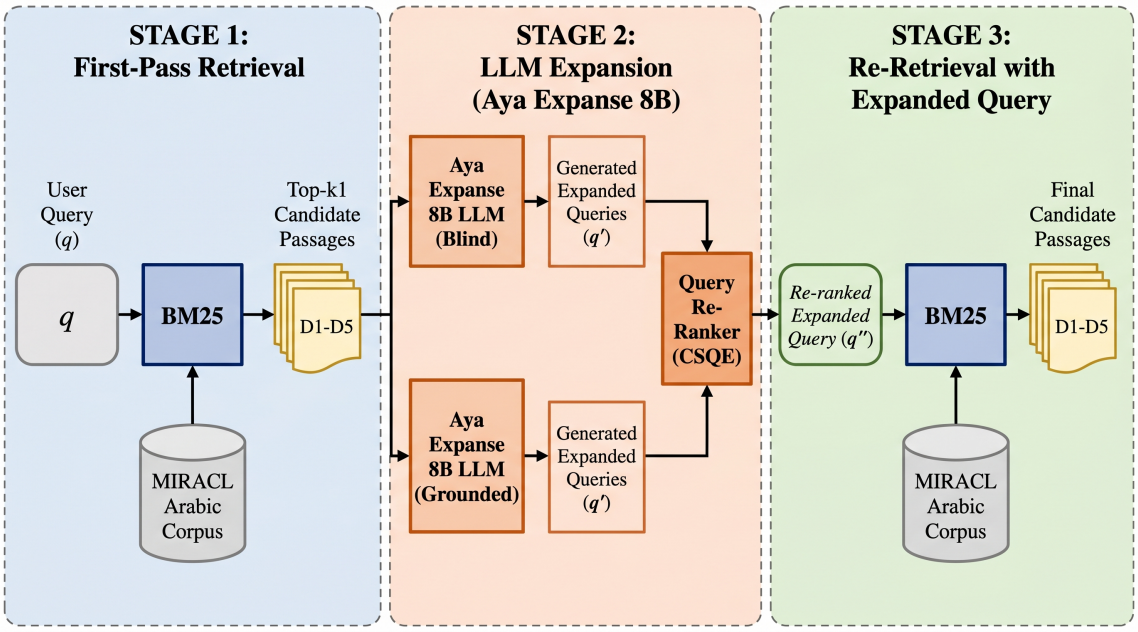

Figure 3.6: CSQE pipeline. Stage 1: BM25 retrieves the top-$k_1 = 5$ first-pass passages. Stage 2: Aya Expanse 8B is invoked four times ($\alpha = 4$, temp $= 1.0$): two corpus-grounded calls (c_1, c_2) extract key sentences from the retrieved passages; two blind calls (b_1, b_2) generate from parametric knowledge. Stage 3: the original query (repeated α times) is concatenated with all four pseudo-documents and fed to a BM25 second pass, yielding NDCG@10 $= 0.616$.

For corpus-grounded generation, the model was given a system prompt distinct from the blind Query2Doc prompt of Section 3.4.3. Where the blind prompt asks the model to *write* a passage from its own parametric knowledge, the corpus-grounded prompt casts it as an information-retrieval assistant whose task is to *examine* the retrieved documents and *extract* the key sentences relevant to the query, responding in Arabic. This extract-rather-than-generate framing is what ties the expansion to the corpus. Both prompts are reproduced verbatim in Appendix C.1.

The accompanying user message presented the original query followed by the numbered first-pass documents and a closing instruction to "begin by examining the initially retrieved documents and identifying the ones that are relevant, even partially, to the query … then extract the key sentences from each document that contribute to their relevance." A single English worked example (a query about warm-blooded sharks, taken from the original CSQE paper [22]) was prepended as a one-shot demonstration—the sole exception to the zero-shot protocol of Section 3.4.2; Aya Expanse, being multilingual, transfers this pattern to Arabic queries while still producing Arabic output. The blind-expansion branch reused the Query2Doc system prompt of Section 3.4.3 verbatim.

Corpus-grounded samples provide attested Arabic vocabulary anchored to the actual Wikipedia passages retrieved in the first pass. Blind samples provide diverse answer-space vocabulary unconstrained by the corpus. The two sources are hypothesised to be complementary: corpus samples prevent hallucination of incorrect entities, while blind samples expand vocabulary coverage beyond the retrieved passages. This hypothesis is validated through the ablation study presented in Section 4.8.2.

3.8.2 Component Ablation Design

To isolate the individual contributions of corpus-grounded and blind expansion, three ablation variants were defined:

- **Corpus-only ablation:** `num_corpus_samples` = 4, `num_blind_samples` = 0. This variant isolates the contribution of first-pass grounding by using only corpus-grounded expansions.
- **Blind-only ablation:** `num_corpus_samples` = 0, `num_blind_samples` = 4. This variant isolates the contribution of blind generation, equivalent to a 4-sample Query2Doc with repetition.
- **Full CSQE configuration:** `num_corpus_samples` = 2, `num_blind_samples` = 2. The combined system as described above.

All three variants used $\alpha = 4$ and the same Aya Expanse 8B model. Additionally, a query repetition factor sweep ($\alpha \in \{1, 2, 3, 4\}$) was conducted on the full CSQE variant. This sweep was reconstructed from stored expansion files (serialised Python objects) without requiring new LLM inference—only the number of query prepends was varied before re-running BM25 evaluation.

3.8.3 Retriever-Specific Application

The CSQE-expanded query (q_{CSQE}) is substantially longer than the original query. Three fusion configurations were tested to determine the optimal retriever–query assignment; whether

expansion helps or hurts each retriever was left as an empirical question, with the causal interpretation reserved for the results discussion (Section 4.9.1).

- **BM25-expanded:** BM25 receives the CSQE-expanded query; Dense receives the original query.
- **Dense-expanded:** BM25 receives the original query; Dense receives the CSQE-expanded query.
- **Both-expanded:** Both BM25 and Dense receive the CSQE-expanded query.

For each configuration, both RRF ($k = 20$) and CC (α swept over $\{0.1, \ldots, 0.9\}$) were evaluated. This design isolates the effect of retriever-specific query representation and determines whether asymmetric expansion outperforms symmetric application. The best-performing configuration, identified empirically in Chapter 4, is illustrated in Figure 3.7.

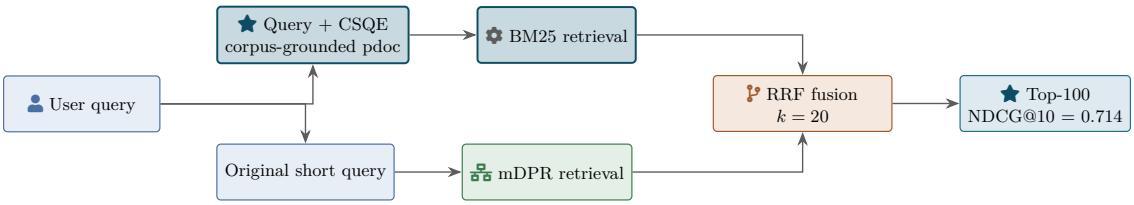

Figure 3.7: Architecture of the thesis's best-performing system. CSQE-expanded queries are fed to BM25 while the original short queries are fed to mDPR; the two ranked lists are fused with RRF ($k = 20$), yielding the headline NDCG@10 of 0.714.

3.9 Per-Query Error Analysis

Aggregate metrics such as NDCG@10 can mask substantial per-query variation: a system with a high average may still regress on a significant minority of queries. A per-query error analysis was conducted to decompose the overall CSQE improvement and identify the conditions under which corpus-steered expansion helps or hinders retrieval.

3.9.1 Per-Query Metric Computation

The `pytrec_eval` library was used to compute NDCG@10 for each of the 2,896 MIRACL Arabic development queries under both the CSQE+Hybrid system and the Aya Blind BM25 $n = 1$ baseline. The per-query delta was defined as $\Delta_i = \text{NDCG@10}_{\text{CSQE},i} - \text{NDCG@10}_{\text{Blind},i}$.

3.9.2 Classification Thresholds

Three categories were defined based on the per-query scores and deltas:

- **Failure:** CSQE NDCG@10 < 0.1. Queries where the system fails to retrieve any relevant document in the top-10, regardless of the baseline performance.
- **Big win:** $\Delta_i > 0.3$. Queries where CSQE achieves a large improvement over the blind baseline.
- **Regression:** $\Delta_i < -0.1$. Queries where CSQE performs worse than the blind baseline by a non-trivial margin.

3.9.3 First-Pass Quality Split

To assess the role of BM25 first-pass quality in CSQE effectiveness, queries were split into two groups based on whether the BM25 first-pass retrieval returned a relevant document at *rank 1* (a positive relevance judgement, rel > 0, for the top-1 retrieved document in the MIRACL qrels). Although CSQE grounding draws on the top-$k = 5$ first-pass documents, the rank-1 result is used as the indicator of first-pass quality because it most directly reflects whether the retriever has correctly identified the query topic. This split enables analysis of whether CSQE gains are driven primarily by successful corpus grounding or by the blind expansion component.

3.9.4 Regression Classification

Regression queries ($\Delta_i < -0.1$) were manually inspected and classified into three types:

- **Type A (Strong BM25 hurt):** The BM25 baseline already achieved NDCG@10 ≥ 0.3 for the query; the CSQE expansion introduced off-topic vocabulary that diluted the effective query terms.
- **Type B (Poisoned first-pass):** The BM25 baseline achieved NDCG@10 < 0.1 for the query; the first-pass retrieval returned irrelevant documents, causing the LLM to ground its expansion on incorrect content.
- **Type C (Partial BM25):** The BM25 baseline achieved NDCG@10 between 0.1 and 0.3; a mixed-quality scenario where the expansion may have helped or hurt depending on specific query characteristics.

This classification, together with the aggregate comparison and the query-length and first-pass splits defined above, forms the interpretive framework applied to the final system in Chapter 4.

Chapter 4

Results and Discussion

This chapter presents the results of all experiments conducted in this research, following the zigzag structure with the methodology described in Chapter 3. Section 4.1 reports the baseline retrieval performance. Section 4.2 presents the error analysis findings that guided technique selection. Section 4.3 reports the initial Query2Doc results with both dense and sparse retrieval. Section 4.4 presents the comprehensive model comparison across ten LLMs. Section 4.5 synthesises the key findings from the model comparison phase. The chapter then reports the expanded experimental results: the BM25 query repetition sweep (Section 4.6), hybrid retrieval fusion (Section 4.7), CSQE (Section 4.8), CSQE combined with hybrid fusion (Section 4.9), and a per-query error analysis decomposing the sources of improvement and regression (Section 4.10). Where a result is reported here in summary form, the corresponding full table is given in Appendix B.

4.1 Baseline Results and Comparison

The two retrieval baselines described in Section 3.2 were evaluated on the full MIRACL Arabic development set (2,896 queries). Table 4.1 presents the results.

Table 4.1: Baseline retrieval results on MIRACL Arabic dev set (2,896 queries). Bold marks the higher score between the two single-retriever baselines; the no-QE hybrid fusion result is included as the strongest non-enhancement reference (detailed in Section 4.7).

Retriever	**NDCG@10**	**Recall@10**	**Recall@100**	**MRR**
mDPR (Dense)	**0.4993**	**0.6156**	0.8407	**0.5328**
BM25S (Sparse)	0.4621	0.5964	**0.8577**	0.4836
Hybrid RRF ($k = 20$, no QE)	0.6267	0.7597	0.9466	0.6517

The mDPR dense baseline reproduced the official MIRACL result exactly: this work measured NDCG@10 = 0.4993, and the published value of 0.499 is the same number rounded to

three decimal places. This confirmed the correctness of the implementation. The BM25S sparse baseline achieved 96% of the official Pyserini BM25 performance (NDCG@10 = 0.4621 vs. target 0.481), with the 4% difference attributable to the use of BM25S rather than Pyserini's Java-based implementation, as discussed in Section 3.2.2.

4.1.1 Complementary Strengths

The two baselines exhibited complementary performance characteristics. mDPR achieved higher NDCG@10 (+8.1%) and MRR (+10.2%), indicating superior ranking quality—relevant passages were placed higher in the result list. BM25S achieved higher Recall@100 (+2.0%), indicating broader retrieval coverage—more relevant passages were retrieved overall, though ranked less effectively.

This complementary behaviour aligns with the theoretical properties of dense and sparse retrieval discussed in Section 2.1.3: dense retrieval captures semantic similarity and ranks well, while sparse retrieval excels at broad lexical matching. This finding confirmed the decision to test QE techniques on both retrievers independently. It also motivates a third reference point: fusing the two retrievers without any QE (RRF, $k = 20$) already raises NDCG@10 to 0.6267 (Table 4.1). This no-QE hybrid is reported here, alongside the single-retriever baselines, as the strongest non-enhancement reference; it is analysed in detail in Section 4.7, where it serves as the ceiling that all QE methods must surpass. The per-query NDCG@10 distribution for both single-retriever baselines is shown in Figure 4.1.

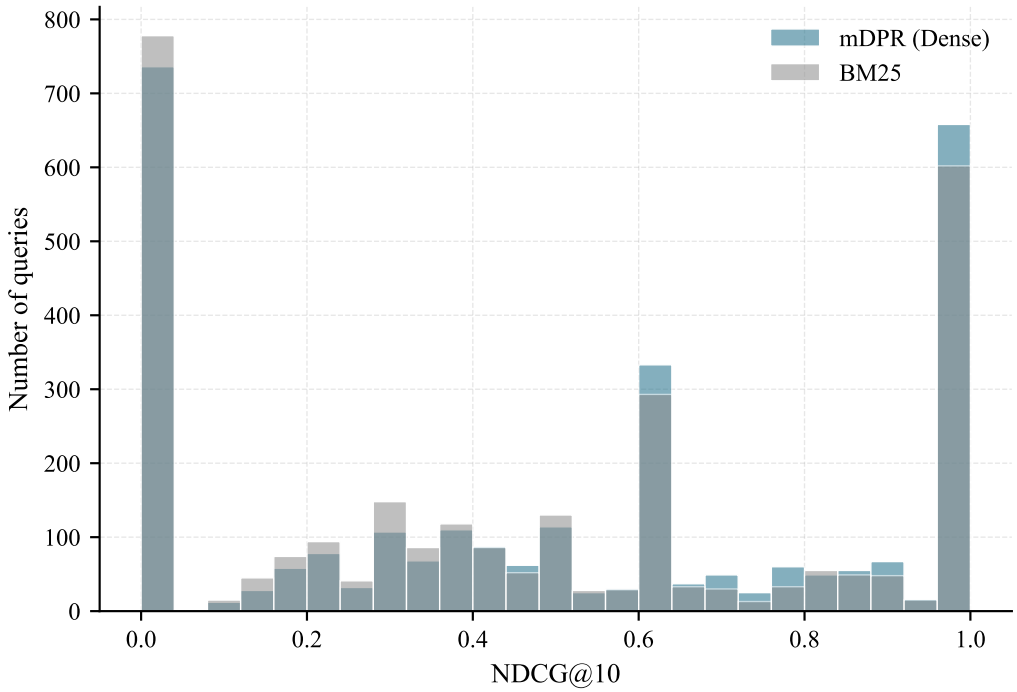

Figure 4.1: Per-query NDCG@10 distribution for the two baseline retrievers on the MIRACL Arabic dev set. Both exhibit a bimodal distribution: a large mass near zero (poorly served queries) and another mass near one (perfectly served), with relatively few in between.

4.2 Error Analysis Findings

The error analysis methodology described in Section 3.3 was applied to the dense baseline. The findings guided the selection of the first QE technique.

4.2.1 Overall Failure Rate

Before an enhancement technique could be chosen, it was necessary to establish how much room for improvement actually existed and where it was concentrated. A single aggregate score conceals this: NDCG@10 = 0.4993 is equally consistent with uniformly mediocre retrieval across every query and with a mixture of near-perfect successes and outright failures, and the two situations call for very different remedies. Segmenting the 2,896 development queries by their individual NDCG@10 scores separates the two cases and quantifies the population that QE could realistically address. Table 4.2 reports this segmentation.

Table 4.2: Performance segmentation of baseline queries by NDCG@10 score.

Category	Count	% of Total	Avg NDCG@10
Failed (NDCG@10 < 0.3)	982	33.9%	0.053
Mediocre ($0.3 \leq$ NDCG@10 < 0.7)	962	33.2%	0.511
Successful (NDCG@10 ≥ 0.7)	952	32.9%	0.948

A notable finding was that 34% of queries (982 out of 2,896) were classified as failed, of which 736 retrieved no relevant passage at all within the top-10. The remaining queries split almost evenly between the mediocre (33%) and successful (33%) bands. While the substantial failed and mediocre fractions establish a clear opportunity for QE, the analysis also shows that roughly a third of queries were already well served by the dense baseline. This bimodal split—visible directly in the per-query distribution of Figure 4.1—carries a design implication that recurs throughout this chapter: any technique applied indiscriminately to all queries risks degrading the third that already succeed in order to rescue the third that fail.

4.2.2 Short Query Performance Gap

With the size of the failure population established, the next question was whether those failures cluster around any identifiable property of the query itself. Query length is the most direct candidate. A query of two or three words offers a lexical retriever very few terms to match and a dense encoder very little context to embed, and the QE literature reviewed in Chapter 2 treats precisely this information poverty as the condition expansion is designed to relieve. Grouping the development set into the three length bands used throughout this thesis—1–3, 4–8, and 9+ words—exposes a clear gap, reported in Table 4.3.

Table 4.3: Retrieval performance by query length bucket.

Length Bucket	Avg NDCG@10	Count	% of Total
Short (1–3 words)	0.345	147	5.1%
Medium (4–8 words)	0.511	2,495	86.2%
Long (9+ words)	0.476	254	8.8%

Short queries achieved only 72% of the performance of long queries (NDCG@10 = 0.345 vs. 0.476), a 28% gap. Performance did not, however, rise monotonically with length: medium-length queries (0.511) outscored long queries, and the Pearson correlation between query length and NDCG@10 was negligible ($r \approx -0.01$), indicating no linear length–performance trend.

The salient pattern is therefore not that longer queries are better, but that the *shortest* queries specifically underperform—consistent with information poverty in queries of only one to three words. This motivated the selection of query expansion as the first enhancement technique: by generating pseudo-documents that add vocabulary and context, the effective information content of short queries is increased. The per-bucket distribution is shown in Figure 4.2.

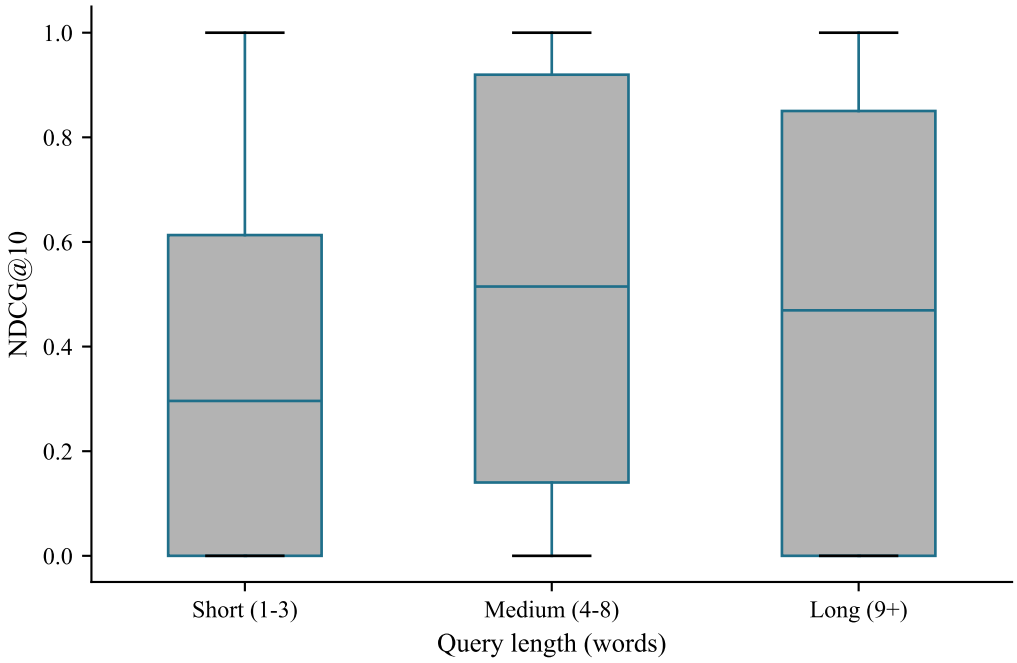

Figure 4.2: Distribution of mDPR NDCG@10 within each query-length bucket. Short queries clearly underperform, motivating QE as a remedy for information poverty in short queries.

4.2.3 Retrieval Coverage Analysis

A low NDCG@10 can arise from two quite different underlying failures, and the distinction determines which remedy is appropriate. Either the retriever never surfaces a relevant passage at all, which is a recall failure that no amount of reordering can repair, or it does surface one but places it too low to be counted, which is a ranking failure that reordering can fix. Measuring the proportion of queries with at least one relevant passage at progressively deeper retrieval cut-offs separates the two, since the value at depth 100 bounds what any reranker could ever achieve while the value at depth 10 reflects what the retriever actually delivers. Table 4.4 reports this coverage for the dense baseline.

Even at depth 100, the dense retriever failed to surface any relevant passage for roughly 10%

Table 4.4: Proportion of queries with at least one relevant passage at each retrieval depth.

Retrieval Depth	Coverage
Top-10	74.6%
Top-20	80.8%
Top-50	86.9%
Top-100	90.1%

of queries (coverage 90.1%), and for a further ∼15% it retrieved a relevant passage but ranked it outside the top-10 (coverage 74.6% at depth 10). The first figure reflects a genuine *recall ceiling*—relevant content the dense encoder never retrieves—while the second reflects a ranking problem. QE can address both: enriching the query representation improves ranking, while combining the dense retriever with a lexical retriever (hybrid fusion, Section 4.7) directly raises the recall ceiling. Both remedies are pursued in the sections that follow, and the coverage figures reported here become the reference points against which their contributions are measured.

4.2.4 Technique Selection Rationale

Manual inspection of the 20 worst-performing queries revealed recurring failure patterns: vocabulary mismatch (e.g., the Arabic term "آزوت" for nitrogen versus the more common "نيتروجين"), named entity variations (e.g., "إبن الهيثم" vs. "ابن الهيثم"), and diacritics sensitivity (e.g., "المَثَانةُ" vs. "المثانة"). Based on these findings, Query2Doc [6] was selected as the first technique because its pseudo-document generation mechanism directly addresses information poverty in short queries and vocabulary mismatch—the two dominant failure patterns identified. The complete rationale for this selection is documented in Section 3.4.

4.3 Query2Doc Results

The error analysis of the preceding section identified information poverty in short queries and vocabulary mismatch as the two dominant failure patterns, and Query2Doc was selected because its pseudo-document mechanism addresses both. The first experiments put that reasoning to the test using a single deliberately modest model, Qwen 2.5 3B Instruct, run against each retriever independently. Testing the two retrievers separately rather than in combination was

essential: it is the only way to observe whether a technique interacts differently with lexical and semantic matching, and the answer proved to be the most consequential finding of this phase.

4.3.1 Dense Retrieval

Dense retrieval was the first target because the error analysis had been conducted on the mDPR baseline, making it the setting in which the predicted benefit was most directly testable. If pseudo-documents genuinely relieve information poverty, the effect should appear most clearly where the query is consumed as a continuous embedding and additional context can only enrich that representation. Table 4.5 reports the outcome against the dense baseline.

Table 4.5: Query2Doc results with dense retrieval (mDPR). Improvement is computed relative to the dense baseline.

Metric	**Baseline**	**Query2Doc**	**Improvement**
NDCG@10	0.4993	0.5435	+8.9%
Recall@10	0.6156	0.6608	+7.3%
Recall@100	0.8407	0.8594	+2.2%
MRR	0.5328	0.5742	+7.8%

Query2Doc produced a substantial improvement of +8.9% in NDCG@10 over the dense baseline, representing the largest single-technique improvement achieved in the initial experiments. All metrics improved, with NDCG@10 and Recall@10 showing the largest gains. The smaller Recall@100 improvement (+2.2%) indicates that Query2Doc primarily improved *ranking* (moving relevant documents higher) rather than *discovery* (finding new relevant documents).

The query expansion statistics showed consistent enhancement: the median query length increased from 27 characters (original) to 250 characters (enhanced), representing an 8.45× expansion ratio. The LLM-generated pseudo-documents provided additional vocabulary and semantic context that improved the mDPR encoder's ability to match queries with relevant passages.

The total runtime was approximately 40 minutes for 2,896 queries on a T4 GPU, achieved through the batch processing optimisations described in Section 3.4.4.

4.3.2 BM25 Retrieval

The identical pseudo-documents were then supplied to the BM25S sparse retriever. Because the generation step is unchanged and only the matching function differs, any divergence between this subsection and the last is attributable to the retrieval paradigm itself rather than to the quality of the expansion. The expectation drawn from the original Query2Doc paper was a gain of the same order as the dense result; the measured outcome, reported in Table 4.6, ran firmly in the opposite direction.

Table 4.6: Query2Doc results with BM25 retrieval (BM25S). Negative values indicate performance degradation.

Metric	Baseline	Query2Doc	Change
NDCG@10	0.4621	0.4090	−11.5%
Recall@10	0.5964	0.5384	−9.7%
Recall@100	0.8577	0.8155	−4.9%
MRR	0.4836	0.4342	−10.2%

In stark contrast to the dense results, Query2Doc *degraded* BM25 performance across all metrics, with NDCG@10 declining by 11.5%. This result, while negative, provided a valuable insight into the interaction between QE techniques and retrieval paradigms.

4.3.2.1 Term Dilution Analysis

The performance degradation was attributed to **term dilution**: the BM25 scoring function (Equation 2.2) is sensitive to query length, as additional terms distribute the term-frequency weight across a larger vocabulary. When a short query such as "ما هي عاصمة السودان" ("What is the capital of Sudan," 4 tokens) was expanded to 200+ tokens, the importance of the original keywords "عاصمة" (capital) and "السودان" (Sudan) was diluted in the BM25 scoring.

As noted in Section 3.4.6, this experiment did not implement the query repetition strategy recommended by Wang et al. [6], where the original query is repeated $n = 5$ times before concatenation. This omission was the primary cause of the BM25 degradation. The term dilution phenomenon and its resolution are further examined in the model comparison experiments (Section 4.4.2), where models with Arabic-specific vocabulary were observed to partially overcome

this limitation.

4.3.3 Comparison with the Original Query2Doc Paper

Situating these two outcomes against the source publication clarifies which of them is anomalous. The original Query2Doc study [6] was conducted in English on MS MARCO with a 175B-parameter model and few-shot prompting, whereas this implementation operates in Arabic on MIRACL with a 3B model and zero-shot prompting—a substantially harder configuration on every axis. Table 4.7 sets the two side by side.

Table 4.7: Comparison between the original Query2Doc results (English, MS-MARCO) and this implementation (Arabic, MIRACL).

Configuration	**Original Paper**	**This Work**
Language	English	Arabic
Dataset	MS-MARCO	MIRACL Arabic
LLM	GPT-3 (175B)	Qwen 2.5 3B
Prompting	Few-shot (4 examples)	Zero-shot
Dense NDCG@10 improvement	+2–5%	+8.9%
Sparse NDCG@10 improvement	+3–7%	$-11.5\%^{*}$

*Without query repetition; the original paper uses $n = 5$ repetition for sparse retrieval.

The dense retrieval improvement (+8.9%) exceeded the original paper's reported range (+2–5%), despite using a model that is 58$\times$ smaller and zero-shot rather than few-shot prompting. The mDPR baseline used here was not fine-tuned on MIRACL, unlike the strong retrievers used in the original paper, which likely contributed to the larger observed improvement.

4.4 Model Comparison Results

The single-model experiments left an obvious question unanswered. A +8.9% dense gain and an -11.5% sparse loss were both obtained with one 3B model, and neither figure indicates how much of the effect belongs to Query2Doc as a technique and how much to Qwen 2.5 3B as a generator. Ten open-source LLMs spanning 2B to 8B parameters were therefore run through an identical pipeline, following the methodology of Section 3.5, with every component except

the generator held constant. The subsections below report the dense and sparse leaderboards in turn, followed by the one model whose failure was severe enough to warrant separate treatment.

4.4.1 Dense Retrieval Leaderboard

The single-model experiment established that Query2Doc helps dense retrieval, but not whether that benefit is a property of the technique or an artefact of the particular model generating the pseudo-documents. Holding the prompt, the retriever, the evaluation set, and the generation parameters fixed while varying only the LLM isolates the model's contribution. Table 4.8 ranks all ten models on the dense pipeline by NDCG@10, with ALLaM-7B listed separately for the reason given in Section 4.4.3.

Table 4.8: Dense retrieval (mDPR) results for all models, ranked by NDCG@10. The improvement column is relative to the no-enhancement dense baseline.

Rank	Model	Params	NDCG@10	Recall@10	MRR	ΔNDCG
—	Baseline (no QE)	—	0.4993	0.6156	0.5328	—
1	Aya Expanse 8B	8B	**0.6164**	**0.7256**	**0.6493**	+23.5%
2	Jais-2 8B	8B	0.6018	0.7161	0.6356	+20.5%
3	Qwen3-8B	8B	0.5958	0.7119	0.6278	+19.3%
4	Qwen 2.5 7B	7B	0.5813	0.6952	0.6134	+16.4%
5	Qwen3-4B	4B	0.5691	0.6824	0.6015	+14.0%
6	Qwen 2.5 3B	3B	0.5435	0.6608	0.5742	+8.9%
7	Gemma 3 4B	4B	0.5391	0.6443	0.5761	+8.0%
8	Falcon-H1 3B	3B	0.5359	0.6484	0.5681	+7.3%
9	SILMA 2B	2B	0.5177	0.6289	0.5508	+3.7%
Dropped model (see Section 4.4.3):						
—	ALLaM 7B	7B	0.2550	0.3335	0.2708	−48.9%

All nine successfully evaluated models improved over the dense baseline, with improvements ranging from +3.7% (SILMA 2B) to +23.5% (Aya Expanse 8B). The top three models—Aya Expanse 8B, Jais-2 8B, and Qwen3-8B—all achieved improvements exceeding +19%, demonstrating that Query2Doc with appropriately selected LLMs can substantially enhance Arabic dense retrieval. Figure 4.3 shows the nine models against the baseline, making the spread between them apparent.

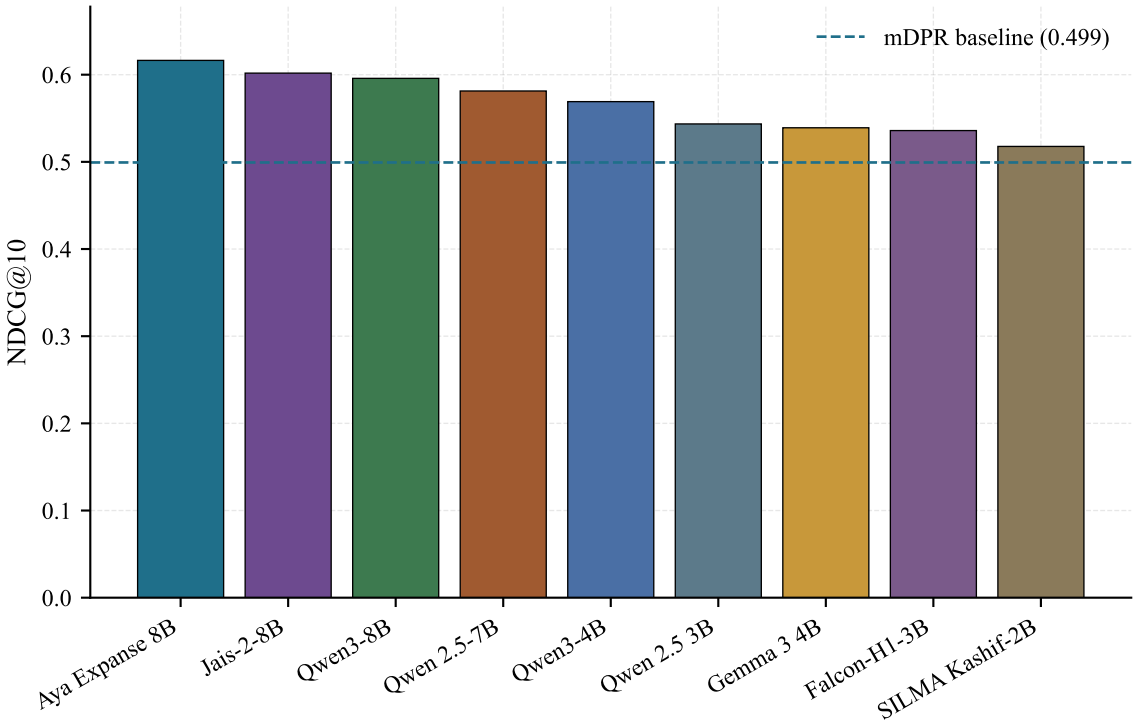

Figure 4.3: Dense retrieval NDCG@10 for the nine retained LLMs on the Query2Doc pipeline, sorted descending; the dropped ALLaM-7B is omitted. The dashed reference line marks the mDPR no-QE baseline (0.499).

4.4.2 BM25 Retrieval Results

Repeating the sweep on the sparse retriever tests whether the degradation observed with Qwen 2.5 3B in Section 4.3.2 is a general property of pseudo-document expansion under BM25 or something specific to that one model. If term dilution is intrinsic to the scoring function, every model should degrade; if instead it depends on the vocabulary a model produces, the outcome should vary with the model's tokeniser and training mix. Table 4.9 reports the same ten models scored on BM25S.

The BM25 results were markedly different from the dense results. Only three models improved over the BM25 baseline: Jais-2 8B (+10.8%), Aya Expanse 8B (+9.2%), and Qwen 2.5 7B (+1.3%). The remaining six models degraded BM25 performance, with Gemma 3 4B showing the largest decline (−25.4%).

The fact that two models—Jais-2 and Aya—achieved substantial BM25 improvements *without* implementing the query repetition strategy (Equation 3.2) is a notable finding. Jais-2's suc-

Table 4.9: BM25 retrieval (BM25S) results for all models, ranked by NDCG@10. The improvement column is relative to the no-enhancement BM25 baseline. Negative values indicate degradation.

Rank	Model	Params	NDCG@10	Recall@10	MRR	ΔNDCG
—	Baseline (no QE)	—	0.4621	0.5964	0.4836	—
1	Jais-2 8B	8B	**0.5122**	**0.6448**	**0.5397**	+10.8%
2	Aya Expanse 8B	8B	0.5046	0.6284	0.5377	+9.2%
3	Qwen 2.5 7B	7B	0.4682	0.6040	0.4905	+1.3%
4	Qwen3-8B	8B	0.4459	0.5806	0.4702	−3.5%
5	SILMA 2B	2B	0.4277	0.5550	0.4485	−7.4%
6	Qwen3-4B	4B	0.4145	0.5403	0.4415	−10.3%
7	Qwen 2.5 3B	3B	0.4090	0.5384	0.4342	−11.5%
8	Falcon-H1 3B	3B	0.4038	0.5311	0.4274	−12.6%
9	Gemma 3 4B	4B	0.3447	0.4532	0.3718	−25.4%

cess with BM25 is hypothesised to stem from its Arabic-centric vocabulary of 150,000 tokens (described in Section A.2.2), which produces pseudo-documents with vocabulary that closely matches the BM25 index terms. Similarly, Aya Expanse's multilingual training (23 languages with explicit Arabic support, Section A.3.6) appears to generate lexically precise expansions.

4.4.3 Dropped Models Analysis

One model, ALLaM-7B, was dropped from the final comparison due to a severe performance issue. Following Dr. Tahani's guidance that even negative results constitute valuable research contributions, the case is analysed in detail.

4.4.3.1 ALLaM-7B

ALLaM-7B (described in Section A.2.3) produced a catastrophic −48.9% decline in NDCG@10 (0.2550 vs. baseline 0.4993), performing *worse* than using no QE at all. Investigation revealed a sentencepiece tokeniser artefact: the Unicode character " " (used internally to mark word boundaries) leaked into the decoded pseudo-documents. These special characters contaminated the query embeddings, causing the mDPR encoder to produce degraded representations.

This failure was attributed to ALLaM's preview/alpha release status at the time of testing. The model had been accepted at ICLR 2025 [37] but was not yet in stable release, and the

tokeniser behaviour had not been fully validated for generation tasks.

4.5 Cross-Cutting Findings from the Model Comparison

This section synthesises the results of the model comparison reported above, identifying the patterns observed across models and their implications for the experiments that follow.

4.5.1 Model Size Correlates with Dense Retrieval Performance

A positive association was observed between model parameter count and dense retrieval improvement, though it must be interpreted cautiously: across the full model set, parameter count is confounded with architecture, training-data volume, and tokeniser design, so the cross-model trend is only suggestive. A cleaner test comes from the Qwen family, whose four models share a common architecture and differ chiefly in scale and generation. Within this family, parameter count and dense NDCG@10 are strongly associated (Pearson $r = 0.94$; Spearman $\rho = 1.00$ across the 3B, 4B, 7B, and 8B models), supporting the size–quality relationship once architecture is held constant. Figure 4.4 illustrates the broader relationship.

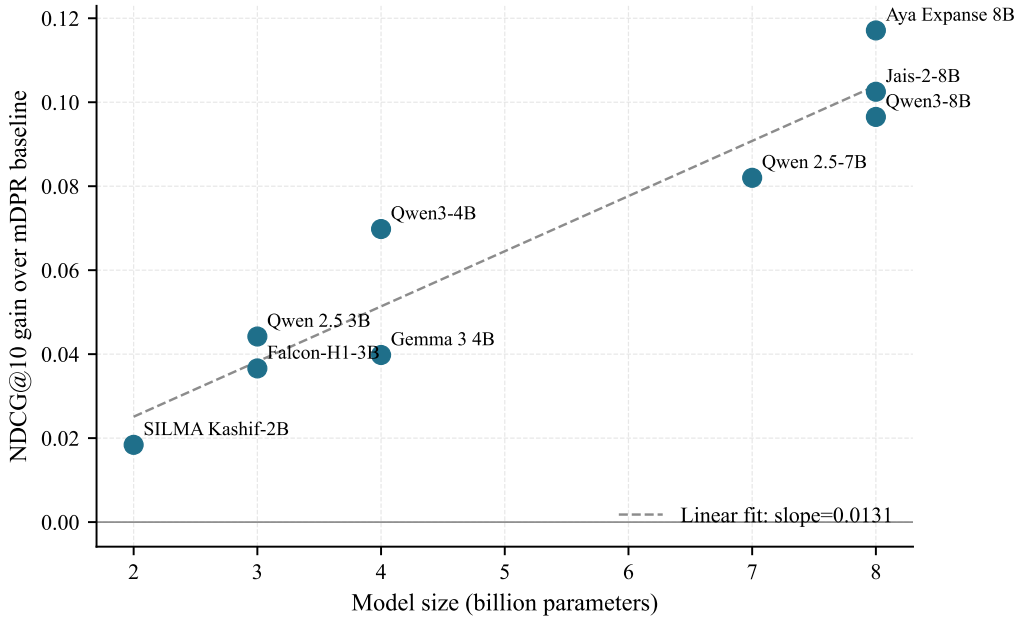

Figure 4.4: Relationship between model parameter count (billions) and gain over the mDPR baseline. A weak positive trend is visible across the full set, and the Qwen-family subset corroborates the size-quality correlation within a single architecture. ALLaM-7B is excluded as an outlier due to tokeniser artefacts.

Grouping models by size tier reveals a consistent pattern:

- **2–3B models:** +3.7% to +8.9% NDCG@10 improvement (SILMA, Qwen 2.5 3B, Falcon-H1)
- **4B models:** +8.0% to +14.0% (Gemma 3, Qwen3-4B)
- **7–8B models:** +16.4% to +23.5% (Qwen 2.5 7B, Qwen3-8B, Jais-2, Aya)

This suggests that larger models generate higher-quality pseudo-documents with richer vocabulary and more accurate factual content, leading to better-calibrated query representations in the dense embedding space.

4.5.2 Generational Improvement Within Model Families

Parameter count is only one of the axes along which the evaluated models differ. A second, harder to observe in a cross-model comparison, is the maturity of the training recipe itself.

The Qwen family isolates this variable unusually well: four of the ten models belong to it, spanning two generations at two comparable parameter scales, with architecture and prompting held constant. Any difference between generations at matched scale is therefore attributable to the training data and recipe rather than to capacity. Table 4.10 pairs the generations at each scale.

Table 4.10: Generational comparison within the Qwen model family.

Comparison	**Qwen 2.5**	**Qwen3**	Δ	**Training Data**
3–4B (Dense NDCG@10)	0.5435	0.5691	+4.7%	18T $\rightarrow$ 36T
7–8B (Dense NDCG@10)	0.5813	0.5958	+2.5%	18T $\rightarrow$ 36T

At both parameter scales, the newer Qwen3 generation outperformed Qwen 2.5, despite identical prompting and evaluation conditions. The improvement is attributed to Qwen3's doubled training data volume (36T vs. 18T tokens), expanded language coverage (119 vs. 29 languages including 8 Arabic dialects), and wider attention architecture (described in Sections A.3.3 and A.3.4).

4.5.3 Arabic Specialisation vs. Multilingual Training

An important finding was that Arabic-specialised models did not consistently outperform multilingual models. Falcon-H1-3B, despite holding the highest Arabic OALL benchmark score ($\sim$62%) at the 3B scale, was outperformed on dense NDCG@10 by both Qwen 2.5 3B (a multilingual model with a lower OALL score) and Qwen3-4B. Similarly, Jais-2-8B, while achieving the best BM25 results, was surpassed on dense retrieval by Aya Expanse 8B, a model designed for 23 languages without Arabic-specific architecture.

This suggests that **a model's Arabic-capability score on the OALL does not directly predict its query expansion quality**. Falcon-H1-3B holds the highest OALL score at its parameter scale, yet it is outperformed on retrieval by multilingual models with lower OALL scores. The skills measured by OALL (reading comprehension, QA, reasoning) differ from the ability to generate vocabulary-rich, lexically diverse pseudo-documents that improve retrieval; training-data volume and diversity appear to be more important factors for this specific task.

The notable exception is Jais-2's BM25 performance: its custom 150,000-token Arabic

vocabulary (Section A.2.2) produces expansions with term distributions that align closely with BM25 index terms, making it uniquely effective for sparse retrieval.

4.5.4 Dense vs. BM25 Behaviour with QE

The divergent response of dense and sparse retrieval to QE is one of the most practically significant findings of this research. Table 4.11 summarises the pattern.

Table 4.11: Number of models improving over baseline, by retriever type.

Outcome	Dense (mDPR)	BM25S
Models improving	9 / 9	3 / 9
Models degrading	0 / 9	6 / 9
Avg improvement	+12.3%	−5.5%
Best improvement	+23.5%	+10.8%
Worst degradation	+3.7%	−25.4%

Dense retrieval benefited universally from QE because the mDPR encoder operates in a continuous semantic space where additional context improves the query embedding representation. BM25, conversely, relies on exact term matching, where additional terms can dilute the weights of the original query keywords (Section 4.3.2.1).

This finding has direct implications for RAG system design: QE techniques must be adapted to the retrieval paradigm, and a strategy that works well for dense retrieval should not be assumed to transfer to sparse retrieval without modification.

4.5.5 Model Selection Summary

The preceding findings bear directly on a practical question: which model should a deployment actually use? No single model dominated on every axis, so the choice depends on which retriever is being served and what hardware is available. Consolidating the evidence from both leaderboards yields four profiles, the first of which determined the model carried forward into all subsequent experiments in this chapter.

a. **Strongest overall performance:** Aya Expanse 8B [38]. It achieved the highest dense

NDCG@10 (0.6164, +23.5%) and the second-highest BM25 NDCG@10 (0.5046, +9.2%), the only model with strong improvements on *both* retrieval paradigms.

b. **Strongest BM25 improvement:** Jais-2 8B [39]. It achieved the highest BM25 NDCG@10 (0.5122, +10.8%), benefiting from its Arabic-centric vocabulary for term-based matching.

c. **Best efficiency profile:** Qwen3-4B [40]. At 4B parameters with no quantisation required, it achieved +14.0% dense NDCG@10—nearly double the improvement of the 3B models—while fitting on a T4 GPU in FP16.

d. **Temperature 0.1** was found to be optimal for deterministic pseudo-document generation, yielding +2.5% improvement over temperature 0.7 in controlled testing (Section 3.5.3).

4.6 BM25 Query Repetition Results

Six of the nine models degraded BM25 performance, and Section 4.3.2.1 attributed this to term dilution rather than to any deficiency in the generated text. If that diagnosis is correct, the remedy need not involve changing the model at all: restoring weight to the original query terms should be sufficient. Query repetition does exactly this, prepending the query to its own expansion a number of times before scoring. The sweep described in Section 3.6 evaluates all nine models across eight repetition configurations—five fixed counts and three adaptive MuGI settings—and so tests both the diagnosis and its proposed remedy simultaneously.

The full nine-model by eight-configuration sweep is given as Table B.1 in Appendix B.1; Figure 4.5 plots its fixed-count columns, and Table 4.12 below reports each model's best configuration with the supplementary Recall and MRR values.

Five key observations emerged from the repetition sweep:

a. **Query repetition brought all nine models above the BM25 baseline.** At $n = 1$ (no repetition), only three of nine models exceeded the BM25 baseline of 0.4621 (Aya at 0.5046, Jais-2 at 0.5122, and Qwen 2.5 7B at 0.4682). At the optimal repetition configuration, all nine models exceeded the baseline, including all six that had initially fallen below it.

Table 4.12: BM25 retrieval metrics at each model's best repetition configuration. Δ NDCG is the improvement from $n = 1$ (no repetition) to the best configuration.

Model	Best Config	NDCG@10	Recall@10	Recall@100	MRR	ΔNDCG
Aya Expanse 8B	$\beta = 2$	0.5855	0.7128	0.9300	0.6165	+0.0808
Jais-2 8B	$\beta = 2$	0.5731	0.7075	0.9217	0.6004	+0.0610
Qwen 2.5 7B	$n = 5$	0.5358	0.6765	0.9105	0.5586	+0.0675
Qwen3-8B	$n = 7$	0.5328	0.6695	0.9064	0.5590	+0.0868
Gemma 3 4B	$n = 7$	0.5277	0.6640	0.9002	0.5551	**+0.1831**
Qwen3-4B	$n = 7$	0.5244	0.6617	0.8980	0.5500	+0.1098
Qwen 2.5 3B	$n = 5$	0.5185	0.6501	0.9031	0.5494	+0.1095
Falcon-H1 3B	$n = 10$	0.5113	0.6456	0.8927	0.5379	+0.1074
SILMA 2B	$n = 7$	0.4786	0.6141	0.8613	0.5019	+0.0509

b. **Optimal repetition varied across models.** The 8B models (Aya, Jais-2, Qwen3-8B) performed best with MuGI adaptive repetition ($\beta = 2$), which scales repetition proportionally to pseudo-document length. The 3–4B models plateaued at fixed $n = 5$–7. These differences likely reflect variation in pseudo-document length rather than model size per se.

c. **Beyond the optimum, performance decreased symmetrically.** Excessive repetition over-weighted the original query tokens and suppressed the useful expansion vocabulary. The sharpest decline was visible in the $\beta = 4$ and $\beta = 6$ columns for the 3–4B models.

d. **Aya Expanse 8B at $\beta = 2$ achieved 0.5855 NDCG@10**, improving BM25 by +26.7% over the baseline (0.4621) and recovering +0.0808 from the $n = 1$ result (0.5046). This confirmed that query repetition resolved the term-dilution degradation observed in the six models that initially underperformed the BM25 baseline.

e. **Gemma 3 4B showed the largest absolute recovery** ($\Delta = +0.1831$), rising from 0.3447 at $n = 1$ (the worst BM25 result of any model) to 0.5277 at $n = 7$. This model had generated the longest pseudo-documents on average, causing the most severe term dilution at $n = 1$.

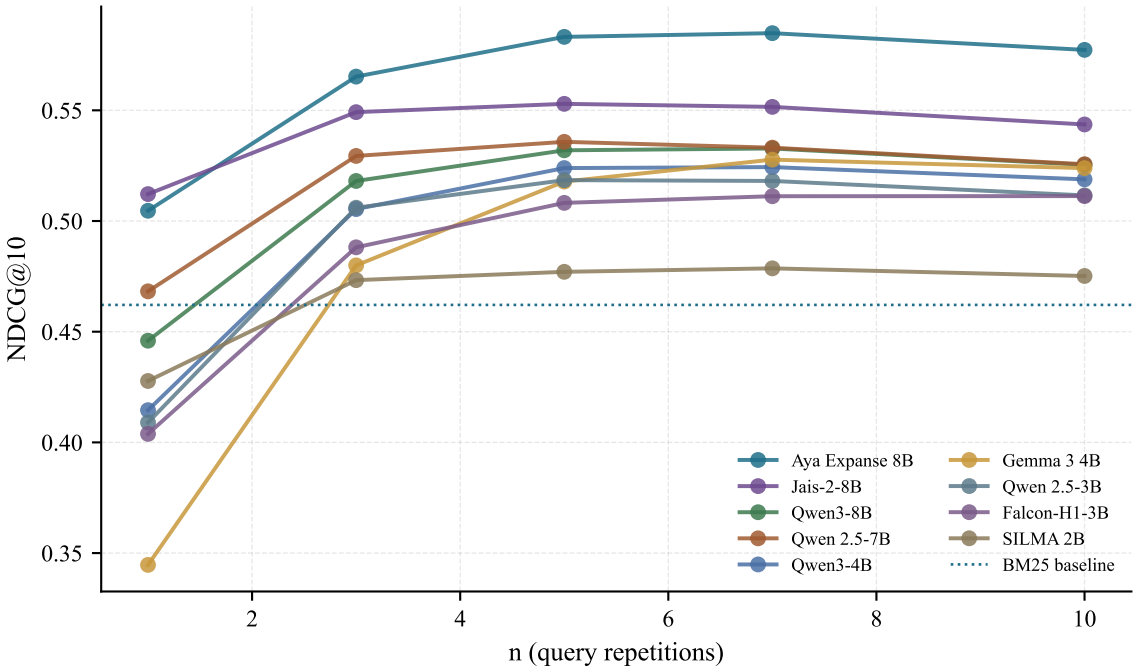

Figure 4.5: NDCG@10 on the BM25 retriever as a function of query-repetition count n for each LLM-generated pseudo-document. Six of nine models start below the BM25 baseline at n=1; all nine recover or exceed it at their optimum n.

Repetition repairs the sparse-side degradation, but it does not remove the underlying divergence between the two retrievers: the same pseudo-document that helps mDPR can still hurt BM25 before any repetition is applied. Figure 4.6 sets each model's dense gain against its sparse gain at n=1, and shows how few models improve both at once.

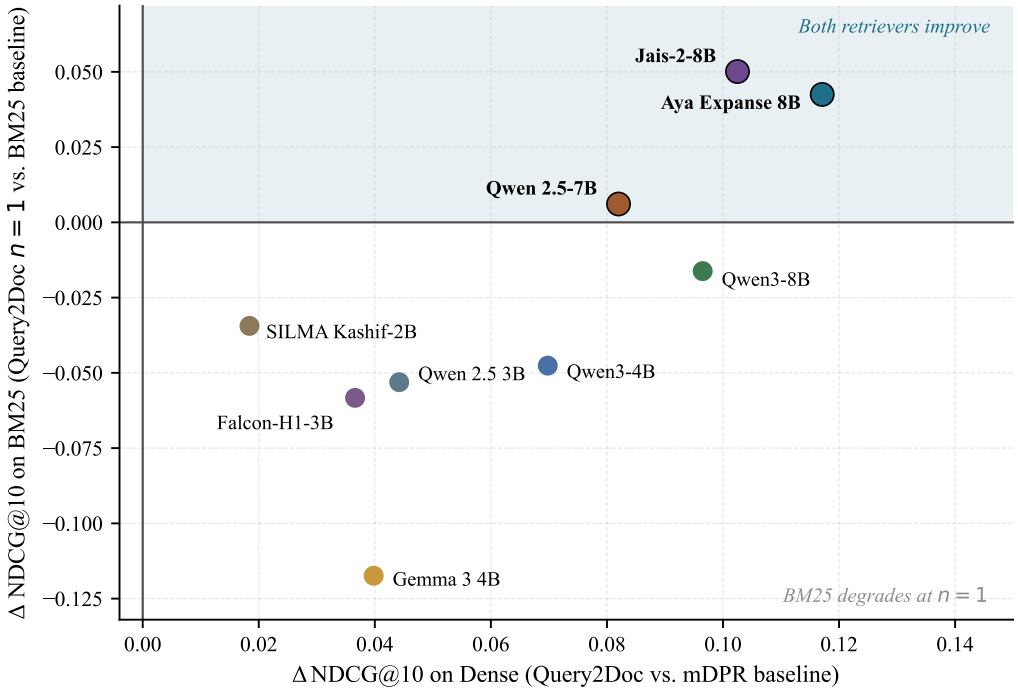

Figure 4.6: Per-model Query2Doc gain on the Dense retriever versus the BM25 retriever (at n=1). Only Aya Expanse 8B and Jais-2 8B sit in the both-positive quadrant; most other models help Dense but degrade BM25 at n=1.

4.7 Hybrid Retrieval Fusion Results

Every technique examined so far has operated on the query. The complementarity established in Section 4.1.1—mDPR ranking more accurately, BM25S recovering more relevant material overall—suggests an orthogonal route to the same goal: leave the query untouched and combine the two ranked lists instead. This matters for the interpretation of everything that follows, because a hybrid of two unenhanced retrievers is a far more demanding reference point than either retriever alone, and it is the bar any QE method must clear to justify its generation cost. The experiments described in Section 3.7 evaluate both fusion strategies: convex combination across a full weight sweep, and reciprocal rank fusion at two values of k. Table 4.13 reports the baselines, the best convex-combination setting and both RRF settings; the complete nine-point sweep is given in Appendix B.2.

Five key observations were drawn from the hybrid fusion results:

Table 4.13: Hybrid retrieval fusion results on the MIRACL Arabic dev set (2,896 queries).

Method	**NDCG@10**	**Recall@10**	**Recall@100**	**MRR**
BM25S alone	0.4621	0.5964	0.8577	0.4836
mDPR alone	0.4993	0.6156	0.8407	0.5328
Hybrid CC $\alpha = 0.5$	**0.6266**	0.7478	0.9458	**0.6577**
Hybrid RRF $k = 20$	**0.6267**	**0.7597**	**0.9466**	0.6517
Hybrid RRF $k = 60$	0.6230	0.7553	0.9466	0.6490

a. Hybrid fusion achieved a +35.6% improvement over BM25 alone and +25.5% over mDPR alone (NDCG@10), substantially outperforming either retriever in isolation and confirming the complementarity identified in Section 4.1.1.

b. RRF $k = 20$ (0.6267) and CC $\alpha = 0.5$ (0.6266) were numerically indistinguishable on NDCG@10. RRF had the edge on Recall@10 (+0.0119), while CC had the edge on MRR (+0.0060). Because RRF requires no hyperparameter tuning, it was adopted as the primary hybrid baseline for all subsequent experiments.

c. The CC sweep was smooth and unimodal, peaking at $\alpha = 0.5$ and degrading symmetrically toward either extreme, as plotted in Figure 4.7 and tabulated in full in Appendix B.2. At $\alpha = 0.9$ (almost pure Dense), the result (0.4996) was essentially the mDPR-alone baseline (0.4993); at $\alpha = 0.1$ (almost pure BM25), the result (0.5248) exceeded BM25 alone; a possible explanation is that even a small Dense component contributed to document reranking. This confirmed that the two retrievers contributed roughly equally.

d. RRF $k = 20$ slightly outperformed $k = 60$ (0.6267 vs. 0.6230 NDCG@10). The smaller k gives more weight to top-ranked items, which benefits NDCG@10 but does not affect Recall@100 (both 0.9466).

e. The hybrid RRF $k = 20$ result of 0.6267 NDCG@10 established the non-QE hybrid baseline that all subsequent QE methods were required to surpass.

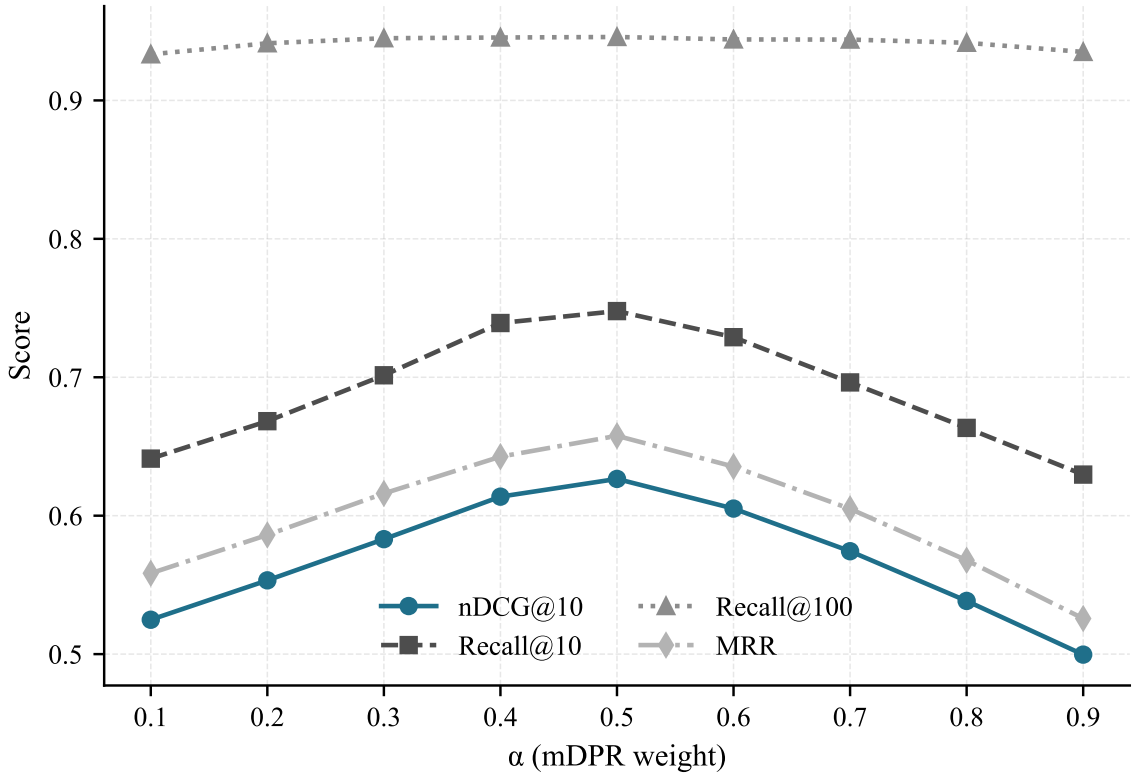

Figure 4.7: Convex-combination α sweep for the no-QE hybrid baseline, showing NDCG@10, Recall@{10,100} and MRR. NDCG@10 peaks near $\alpha = 0.5$; Recall@100 is largely insensitive to α.

4.8 CSQE Results

All the query enhancement reported so far has been *blind*: the LLM sees only the query and writes from parametric memory, with no access to the corpus being searched. That design has a specific weakness, visible in the regression cases examined later in this chapter—when the model does not know the answer it produces a fluent and confident fabrication, and the retriever is then steered away from the very document it was meant to find. Corpus-steered query expansion, implemented as described in Section 3.8, addresses this by grounding the expansion in passages the corpus actually contains. The results below first establish what CSQE achieves on each retriever, then decompose that effect into its corpus-grounded and blind components.

4.8.1 Main CSQE Results

The first question is whether corpus grounding delivers any advantage over the blind expansion it replaces, and whether that advantage behaves consistently across the two retrieval paradigms. Since blind QE had already proved to help dense retrieval while harming BM25 without repetition, CSQE was evaluated on both retrievers independently under the configuration fixed in Section 3.8. Table 4.14 places both results against their own baselines and against the no-QE hybrid.

Table 4.14: NDCG@10, Recall@10, Recall@100, and MRR for CSQE-enhanced retrieval on the MIRACL Arabic dev set. CSQE config: $k = 5$ first-pass, 2 corpus + 2 blind samples, $\alpha = 4$, Aya Expanse 8B.

Method	**NDCG@10**	**Recall@10**	**Recall@100**	**MRR**
BM25S alone	0.4621	0.5964	0.8577	0.4836
BM25+CSQE	**0.6157**	**0.7447**	**0.9422**	**0.6380**
mDPR alone	0.4993	0.6156	0.8407	0.5328
Dense+CSQE	0.5915	0.7073	0.8816	0.6225
Hybrid RRF $k = 20$ (no QE)	0.6267	0.7597	0.9466	0.6517

BM25+CSQE achieved 0.6157 NDCG@10, representing a +33.2% improvement over BM25 alone and, notably, nearly matching the no-QE hybrid baseline (0.6267) using only a single retriever. Dense+CSQE improved over mDPR alone by +18.5% but fell below BM25+CSQE. Dense+CSQE underperformed BM25+CSQE, indicating that the expanded query hurt rather than helped the Dense retriever in this configuration. This observation motivated the retriever-specific application experiments in Section 4.9.

4.8.2 Component Ablation

The CSQE configuration adopted here draws two corpus-grounded samples and two blind samples for every query, so the headline result above is a property of the mixture rather than of corpus grounding on its own. Establishing which component does the work—and whether the two are genuinely complementary or merely redundant—requires holding the total sample budget fixed at four and varying only their composition. Table 4.15 compares the balanced 2+2 configuration against the two extremes.

Table 4.15: BM25 retrieval metrics for corpus-only, blind-only, and combined CSQE expansion variants. All variants use $\alpha = 4$, Aya Expanse 8B, $k = 5$ first-pass.

Variant	NDCG@10	Recall@10	Recall@100	MRR
BM25 alone	0.4621	0.5964	0.8577	0.4836
013c: Corpus-only (4c+0b)	0.5381	0.6457	0.8790	0.5651
013d: Blind-only (0c+4b)	0.5752	0.7089	0.9201	0.6032
013: CSQE 2+2	**0.6157**	**0.7447**	**0.9422**	**0.6380**

The key finding is that corpus-grounded and blind expansion components are complementary: the combined 2+2 system (0.6157) exceeded both blind-only (0.5752) and corpus-only (0.5381) variants individually, a +0.0405 improvement over the stronger single component. Blind-only outperformed corpus-only on BM25 because blind generation produces full answer paragraphs with diverse Arabic term variants, which benefits BM25's vocabulary breadth scoring, while corpus-only extraction produces passage-level excerpts structurally similar to the original query. Corpus samples contribute by anchoring the expansion to attested Wikipedia vocabulary, reducing entity misidentification; blind samples diversify answer-space coverage beyond the first-pass retrieval window.

Table 4.16 presents the effect of the query repetition factor α on BM25+CSQE performance.

Table 4.16: Effect of query repetition factor α on CSQE BM25 retrieval NDCG@10.

α	BM25+CSQE NDCG@10
1	0.6095
2	0.6130
3	0.6154
4	**0.6157**

Performance improved monotonically from $\alpha = 1$ to $\alpha = 4$, but the total gain was minimal (+0.0062). At $\alpha = 1$, the system already captured 98.9% of the $\alpha = 4$ NDCG@10 result. The query weighting factor was therefore not a critical hyperparameter for this configuration.

4.9 CSQE with Hybrid Fusion

Two independent gains have now been established: hybrid fusion raises NDCG@10 to 0.6267 without touching the query, and CSQE raises BM25 to 0.6157 without a second retriever. Com-

bining them is the obvious next step, but it raises a question the literature does not settle—which retriever should receive the expanded query. The intuitive answer is both, on the reasoning that a better query benefits any retriever. The evidence of Section 4.8.1, where the Dense retriever gained less from CSQE than BM25 did, suggests otherwise. Section 4.9.1 therefore evaluates all three assignments of the expanded query to the two retrievers, and Section 4.9.2 consolidates the resulting system against every configuration reported earlier in this chapter.

4.9.1 Fusion Configuration Comparison

The three possible assignments—expanding the BM25 side only, the Dense side only, or both—were each evaluated under both fusion methods, following the methodology of Section 3.8.3. Because RRF combines ranks rather than scores, it rewards ranked lists that disagree with one another, which makes the assignment question genuinely open: an expansion that improves a retriever in isolation may still weaken the fused result if it makes the two lists converge. Table 4.17 reports all six combinations against the no-QE hybrid.

Table 4.17: Results for three CSQE application strategies in hybrid fusion. BM25-expanded: BM25 receives CSQE query, Dense receives original query. Dense-expanded: BM25 receives original query, Dense receives CSQE query. Both-expanded: both retrievers receive CSQE query.

Strategy	Fusion	NDCG@10	Recall@10	Recall@100	MRR
Hybrid RRF $k = 20$ (no QE)	RRF	0.6267	0.7597	0.9466	0.6517
Dense-expanded (BM25 raw + Dense+CSQE)	RRF	0.6474	0.7928	0.9571	0.6578
Dense-expanded (BM25 raw + Dense+CSQE)	CC $\alpha = 0.4$	0.6588	0.7851	0.9569	0.6777
Both-expanded (BM25+CSQE + Dense+CSQE)	RRF	0.6936	0.8290	0.9660	0.7037
Both-expanded (BM25+CSQE + Dense+CSQE)	CC $\alpha = 0.5$	0.6959	0.8249	0.9647	0.7079
BM25-expanded (BM25+CSQE + Dense raw)	CC $\alpha = 0.6$	0.7088	0.8302	0.9717	0.7268
BM25-expanded (BM25+CSQE + Dense raw)	**RRF**	**0.7137**	**0.8363**	**0.9734**	**0.7362**

BM25-expanded was the winning configuration despite giving the Dense retriever the original short query rather than the CSQE expansion. When CSQE is applied to both retrievers (Both-expanded, 0.6936), the two ranked lists become less divergent from each other—both are now based on expanded queries grounded in the same first-pass documents—which lowers the complementarity that RRF fusion relies upon. BM25-expanded preserves this complementarity: the BM25 run is enriched by CSQE vocabulary while the Dense run retains its independent semantic signal, making the two lists more complementary for fusion. A second, reinforcing

factor concerns the Dense encoder itself: mDPR was fine-tuned on MS MARCO, whose queries are short web-search strings [41], whereas a CSQE-expanded query is very long (mean ≈1,500 characters, against ≈30 for the original query). Feeding such a long, out-of-distribution input to the Dense encoder degrades its ranking quality, so withholding the expansion from the Dense side both preserves list divergence and avoids this length penalty, while the lexical BM25 retriever still benefits from the added vocabulary. This finding illustrates a **retriever-specific query representation principle**: each retriever should receive the query format that maximises its divergence from the other retriever's ranked list.

Table 4.18 presents the contribution of each CSQE component within BM25-expanded RRF.

Table 4.18: BM25-expanded RRF ablation: contribution of corpus and blind expansion components when fused with the original Dense run.

BM25 Expansion	**BM25-expanded RRF NDCG@10**
Corpus-only (4c+0b) + Dense orig	0.6616
Blind-only (0c+4b) + Dense orig	0.7082
CSQE 2+2 + Dense orig	**0.7137**

Table 4.19 presents the effect of the query repetition factor α on the best BM25-expanded RRF system.

Table 4.19: Effect of query repetition factor α on the best BM25-expanded RRF system. All values use the same CSQE 2c+2b expansion, varying only the α prepended to the BM25-side query.

α	**BM25-expanded RRF NDCG@10**
1	0.7123
2	0.7121
3	0.7130
4	**0.7137**

The α parameter was nearly flat for the final fused system: the $\alpha = 1$ result (0.7123) was only 0.0014 below $\alpha = 4$ (0.7137). This was consistent with Table 4.16 (BM25-alone α sweep): once CSQE expansion was combined with a strong dense run via RRF, the α parameter had negligible effect. The $\alpha = 4$ configuration was reported as the primary result because it matched the original CSQE experimental settings, but $\alpha = 1$ would be preferable for a production deployment (one fewer token per expanded query).

4.9.2 Overall System Progression

The BM25-expanded RRF configuration identified above is the final system of this thesis. Its standing is best judged not against a single baseline but against every intermediate configuration that preceded it, since each of those represents a design choice that could plausibly have been the endpoint: a stronger retriever, a better generator, blind expansion, fusion without expansion, or expansion without fusion. Quantifying the margin over each in turn shows how much of the total gain is attributable to which decision. Table 4.20 reports those margins.

Table 4.20: NDCG@10 improvements of BM25-expanded RRF over prior systems.

Comparison	**ΔNDCG@10**	**% change**
BM25-expanded RRF vs BM25 alone	+0.2516	+54.5%
BM25-expanded RRF vs mDPR alone	+0.2144	+42.9%
BM25-expanded RRF vs best blind BM25 QE (Aya $\beta = 2$)	+0.1282	+21.9%
BM25-expanded RRF vs best blind Dense QE (Aya 8B)	+0.0973	+15.8%
BM25-expanded RRF vs Hybrid RRF (no QE)	+0.0870	+13.9%
Both-expanded vs BM25-expanded	−0.0178	Dense+CSQE hurts in fusion
Dense-expanded vs Hybrid RRF	+0.0207	Weakest config still beats no-QE hybrid

All three CSQE fusion configurations exceeded the no-QE hybrid baseline, so the choice of assignment determines the size of the gain rather than whether one exists. The +0.0870 margin of BM25-expanded over that hybrid (+13.9%) is the most informative single number in the table, because both systems fuse the same two retrievers over the same corpus and differ only in whether the BM25 side receives a corpus-steered query. It therefore isolates the contribution of CSQE from the contribution of fusion. Figure 4.8 places this final margin in the context of the whole development sequence.

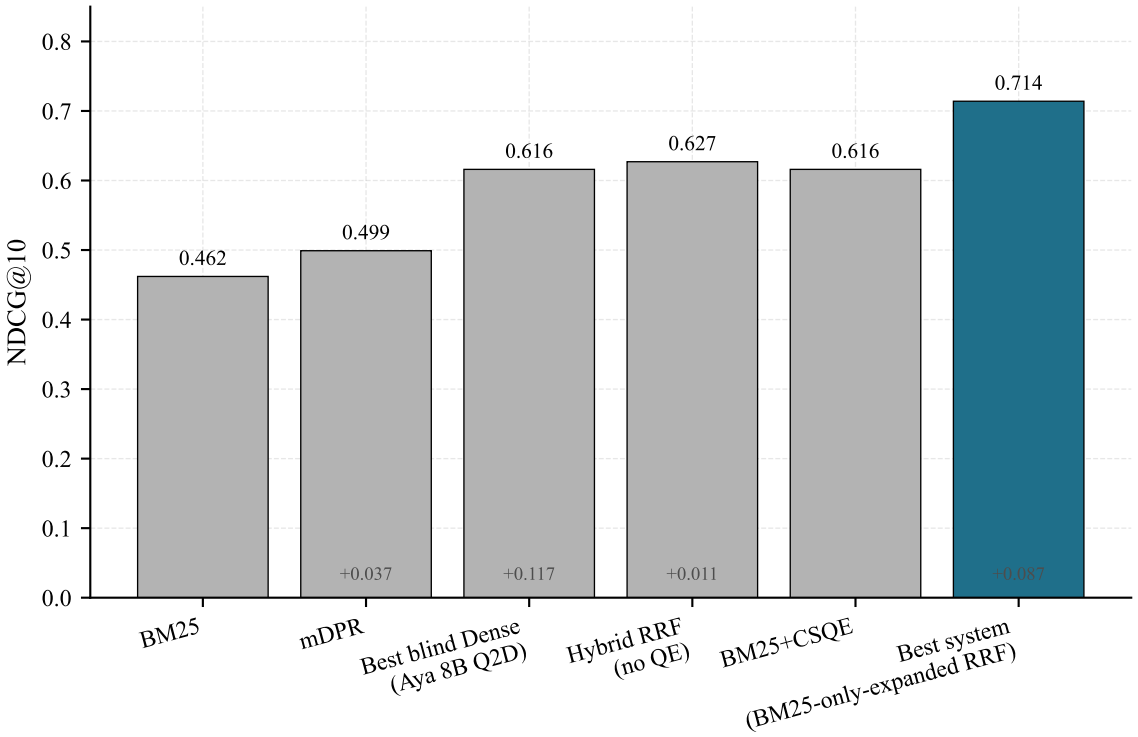

Figure 4.8: Progression of NDCG@10 from the BM25 baseline (0.462) through intermediate query-enhancement steps to the best system, BM25-only-expanded with RRF fusion, at 0.714. Annotations on each bar report the gain over the best previous configuration.

The progression demonstrated a consistent improvement through three stages of pipeline development. Blind QE improvements (Sections 4.3 and 4.6) confirmed that LLM-generated pseudo-documents are effective for Arabic information retrieval. The hybrid baseline (Section 4.7) established that lexical and semantic retrieval are complementary. CSQE (Section 4.8) introduced corpus grounding that outperformed blind QE on BM25. The final combination of corpus-steered expansion applied asymmetrically to hybrid retrieval (BM25-expanded RRF) achieved 0.7137 NDCG@10—a 54.5% improvement over BM25 alone and a 42.9% improvement over mDPR alone. Recall@100 = 0.9734 indicated that 97.3% of relevant documents appeared within the top 100 candidates, establishing a strong upper bound for downstream reranking.

A consolidated view of every experiment reported in this thesis, grouped by phase, is given in Table B.3 of Appendix B.

4.10 Per-Query Error Analysis

Corpus-level metrics establish that the final system is better, but not why, nor for which queries, nor at what cost. A mean improvement of +0.1890 NDCG@10 could equally describe a method that lifts every query slightly or one that transforms a subset while damaging others. Because the two profiles imply different deployment risks and different directions for future work, the per-query analysis described in Section 3.9 decomposes the aggregate gain along four dimensions: the overall win and loss distribution, the conditions that modulate the size of the gain, the mechanism behind the largest wins, and the causes of the regressions.

4.10.1 Win/Loss Distribution

A corpus-level gain of +0.1890 NDCG@10 is compatible with two very different underlying behaviours: a small uniform improvement spread evenly over every query, or a large improvement on some queries paid for partly by losses on others. The distinction matters for deployment, because a method that rescues many queries while damaging a few is not interchangeable with one that helps everything slightly. Comparing the final system against the blind baseline query by query separates these cases. Table 4.21 reports the resulting win, loss, and tie counts.

Table 4.21: Per-query comparison of the CSQE+Hybrid system (BM25-expanded RRF) against the Aya blind BM25 $n = 1$ baseline (0.5046 NDCG@10) on 2,896 MIRACL Arabic dev queries. The system's per-query-mean NDCG@10 is 0.6936; this is computed over individual queries and differs slightly from the corpus-level headline of 0.7137 reported in Table B.3, which is produced by the official pooled evaluation.

Condition	**Queries**	**%**
CSQE+Hybrid > Blind	1,646	56.8%
Tie	770	26.6%
CSQE+Hybrid < Blind	480	16.6%
Mean delta (CSQE − Blind)	—	**+0.1890**

The CSQE+Hybrid system improved on 56.8% of queries and regressed on 16.6%, with a mean per-query delta of +0.1890 NDCG@10. The 26.6% of ties were predominantly queries where both systems achieved either 0.0 (no relevant document retrieved by either) or 1.0 (perfectly retrieved). Figure 4.9 gives the full shape of that distribution, which the win-tie-loss

counts alone do not convey: the gains are concentrated in a long positive tail rather than spread evenly across the set.

A total of 258 queries (8.9%) were classified as failures (CSQE+Hybrid NDCG@10 $<$ 0.1). A corpus-membership check over the full 2,061,414-document MIRACL Arabic index confirmed that *every* one of these queries has at least one relevant passage present in the corpus: none are irretrievable, and there is no dataset-level ceiling or indexing gap. The failures are therefore genuine retrieval failures of two kinds. For 199 of the 258 queries the relevant passage is present but is missed by *all* methods, including the BM25 baseline—intrinsically hard queries for both lexical and semantic retrieval. The remaining 58 queries are retrievable by BM25 alone (43 of them with BM25 NDCG@10 $\geq$ 0.3) but are lost by the CSQE hybrid; these are genuine regressions in which expansion or fusion displaced a document that the bare query would otherwise have surfaced.

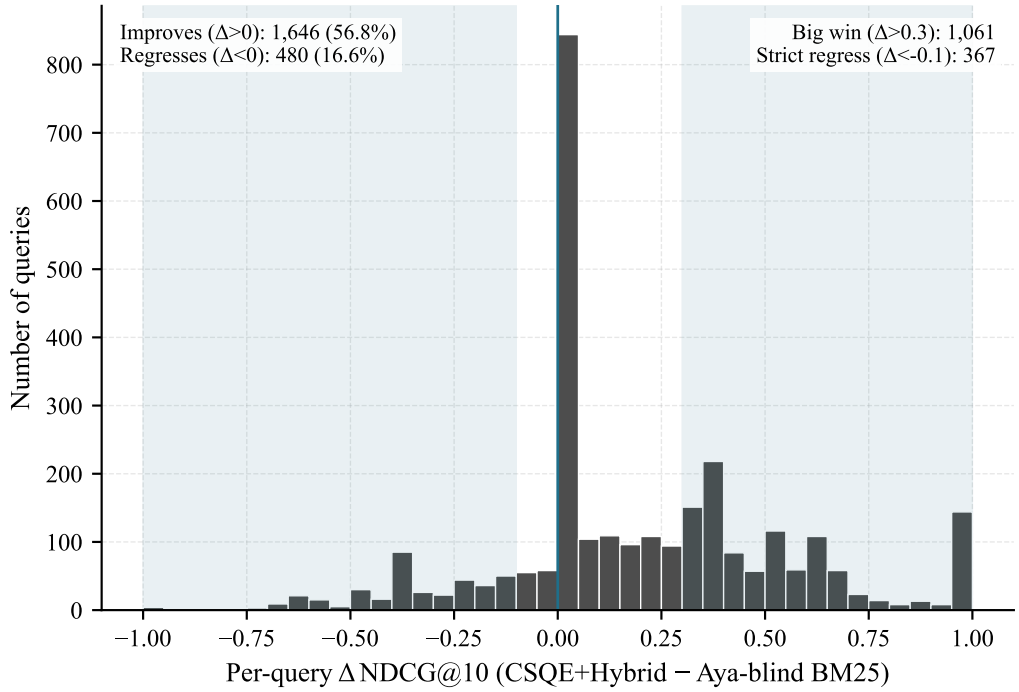

Figure 4.9: Distribution of the per-query difference Δ = NDCG@10(CSQE+Hybrid) − NDCG@10(Aya-blind BM25) across all 2,896 MIRACL Arabic dev queries. Shaded bands mark the Big-Win region ($\Delta > 0.3$) and the strict Regression region ($\Delta < -0.1$).

4.10.2 First-Pass Quality as the Largest Modulator

CSQE grounds its expansion in the passages returned by a BM25 first pass, so the quality of that first pass is a plausible determinant of the quality of the expansion it produces. If the mechanism works as intended, queries whose first pass already surfaces a relevant document should benefit more than those whose first pass does not, and the size of that difference measures how much the method depends on its own starting conditions. Partitioning the development set on whether the top-ranked first-pass document is relevant tests this directly, as shown in Figure 4.10.

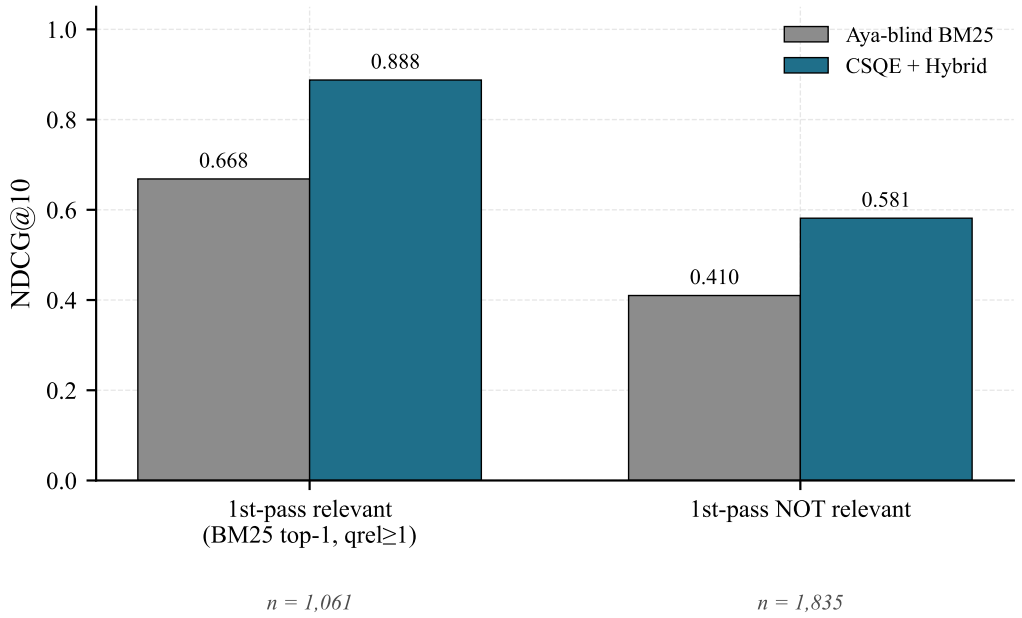

Figure 4.10: CSQE+Hybrid versus Aya-blind BM25 baseline conditioned on whether the BM25 first-pass top-1 document is relevant (qrel ≥ 1). The asymmetry between the relevant ($n = 1{,}061$) and not-relevant ($n = 1{,}835$) groups demonstrates first-pass quality as the largest behavioural modulator of CSQE.

When first-pass retrieval succeeded—1,061 queries, or 36.6% of the set—CSQE+Hybrid reached 0.8877 NDCG@10 against 0.6684 for blind QE, a gain of +0.2193 and effectively near-ceiling retrieval. For the remaining 1,835 queries the system still improved, from 0.4100 to 0.5814 (+0.1714), so a failed first pass degrades the method without disabling it. The gap of +0.3063 between the two groups' final scores exceeds the system's overall +0.1890 advantage over blind QE, which confirms that first-pass quality is the largest single modulator of CSQE effectiveness and identifies it as the most promising target for future improvement.

The same decomposition applied to query length tests a different claim. Section 4.2.2 es-

tablished that the shortest queries are the worst served by the dense baseline, and information poverty was the motivation for adopting expansion in the first place; if that reasoning is sound, the shortest queries should be the ones expansion helps most. Figure 4.11 reports the comparison across the three length bands.

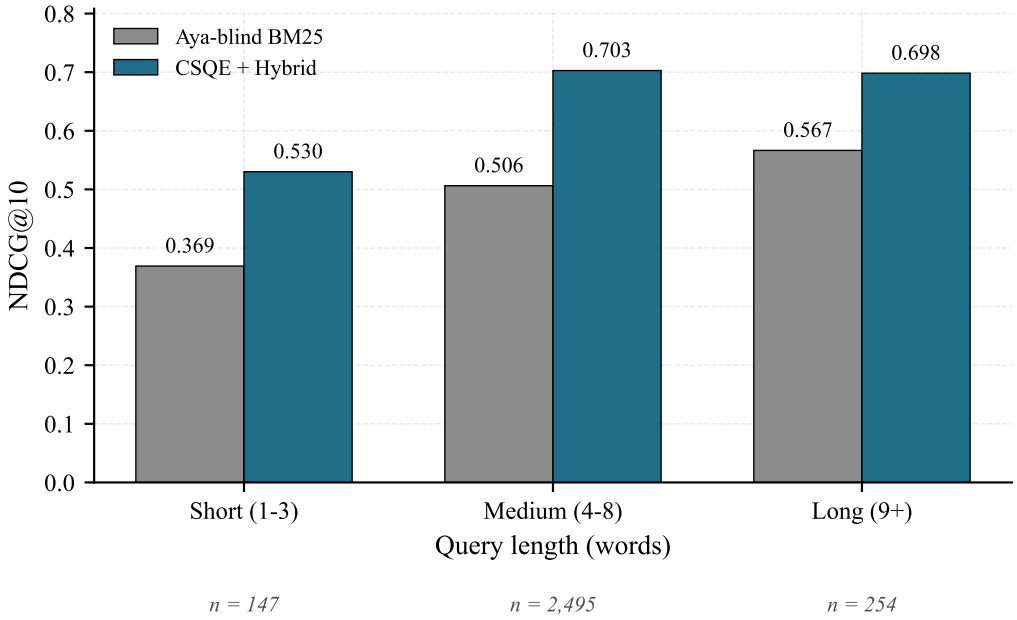

Figure 4.11: Per-bucket comparison of Aya-blind BM25 and CSQE+Hybrid using the 1–3 / 4–8 / 9+ word buckets. CSQE+Hybrid improves every bucket; short queries gain the most proportionally (+43.6%) and medium queries the most absolutely (+0.197).

Every bucket improved. Short queries (1–3 words, $n = 147$) rose from 0.369 to 0.530, a relative gain of +43.6% and the largest proportional improvement of the three, consistent with the information-poverty argument: these queries are the most underspecified and therefore have the most to gain from both added vocabulary and corpus grounding. Medium queries (4–8 words, $n = 2{,}495$) gained most in absolute terms, from 0.506 to 0.703 (+0.197), and because they constitute 86% of the set they dominate the corpus-level result. Long queries (9+ words, $n = 254$) improved least, from 0.567 to 0.698 (+0.132, +23.3%), since their greater intrinsic information content already mitigates the problem expansion is designed to solve. The technique therefore helps most precisely where the error analysis of Section 4.2 predicted it would.

4.10.3 Big Wins: The Corpus Grounding Effect

Among the 1,061 big-win queries ($\Delta > 0.3$), a recurring pattern was observed: in 463 the blind baseline scored 0.000 while CSQE recovered the query, and in 143 of these the recovery was complete (CSQE NDCG@10 = 1.000). In each such case, blind QE hallucinated a plausible but factually incorrect entity, while CSQE's first-pass corpus document anchored the expansion to the correct Wikipedia article.

One such case is the query ما هو الرباط المنصوري؟ ("What is al-Ribat al-Mansuri?"), where blind QE generated a passage about a surgical ligature while the corpus-grounded expansion correctly anchored the query to a Mamluk-era Sufi lodge endowed by Sultan al-Mansur Qalawun; CSQE scored NDCG@10 = 1.000 against the blind baseline's 0.000. Two further examples of the same pattern, together with the generated content in each case, are given in Table B.4 in Appendix B.4.

This pattern validated the corpus grounding hypothesis: for niche or ambiguous Arabic terms, blind QE confidently generated a common modern interpretation, while corpus grounding locked onto the actual Wikipedia entity. The impact on query expansion was direct: the corpus anchor supplied the correct Arabic vocabulary (e.g. رباط، مملوكي، قلاوون for the first example, rather than the surgical terms produced by blind generation) that both BM25 and Dense scoring relied upon. Notably, for all three of these examples the bare query alone—with no expansion at all—already retrieved the correct article; the blind pseudo-document therefore actively *poisoned* a query the retriever had already solved, whereas the corpus-grounded expansion stayed on target and preserved the hit.

4.10.4 Regression Analysis

The gains documented above are averages, and 367 queries moved in the opposite direction, losing more than 0.1 NDCG@10 relative to blind expansion. These cases are more diagnostically useful than the wins, because they mark the conditions under which corpus steering is the wrong choice and therefore define the limits of the method. Manual inspection of the full set, cross-referenced against each query's BM25 baseline score, showed that the losses are not

uniformly distributed but fall into three regimes separated by how well BM25 handled the query before any expansion was applied. Table 4.22 reports that classification.

Table 4.22: Classification of 367 regression queries (CSQE NDCG@10 $<$ Blind by > 0.1).

Type	Count	%	Description
A: Strong BM25 hurt	191	52%	BM25 baseline ≥ 0.3; CSQE expansion introduces off-topic vocabulary
B: Poisoned first-pass	131	36%	BM25 baseline < 0.1; first-pass retrieves irrelevant doc $\rightarrow$ LLM grounding on wrong content
C: Partial BM25	45	12%	BM25 baseline 0.1–0.3; mixed quality

Type A regressions (52%, 191 queries) occurred when BM25 already handled the query well (baseline NDCG@10 ≥ 0.3) and the CSQE expansion introduced Arabic vocabulary from the corpus document that diluted the original query terms. This was the same term-dilution effect observed in Section 4.3.2.1, but in reverse: here the expansion itself (rather than the pseudo-document) reduced BM25 precision for queries that were already well handled.

Type B regressions (36%, 131 queries) share a common mechanism: the query is short or ambiguous and its surface form collides with an Arabic homonym, personal name, or near-topic entity, so BM25's first pass retrieves the *wrong* entity. CSQE then grounds its expansion on those off-topic documents and the final query misses—even when the model's generation is otherwise clean—whereas blind expansion, which ignores the first pass, often answers correctly. The mode spans three sub-flavours. In a *tokenisation/dialect homonym* (e.g. "ماهو التطرف؟", "what is extremism?"), four of the five first-pass documents match the token "ماهو" and surface a southern-dialect article, a song, a novel, and an Iranian village rather than the extremism article, because "ماهو" collides with a dialectal phrase rather than the intended interrogative "ما هو". In a *personal-name homonym* (e.g. "متى ولد نجيب محفوظ؟"), the first pass retrieves a different "نجيب باشا محفوظ"—a physician, not the novelist. In a *wrong-entity / near-topic collision* (e.g. "من هو مصمم موقع ويكيبيديا؟"), the first pass returns an article on Wikipedia's ban in Turkey, leading the expansion to ground on an unrelated entity. In each case it is the first-pass error, not the generation, that drives the regression.

The remaining 12 per cent (Type C) fall in an intermediate band in which the BM25 baseline is neither strong nor absent, and no single mechanism dominates. The two dominant modes motivate two concrete design improvements: a *first-pass quality gate* that falls back to blind

expansion when the top-1 BM25 document overlaps poorly with the query (addressing Type B), and *asymmetric expansion weighting* that down-weights the expansion when BM25 is already strong (addressing Type A). Both are developed as recommendations in Chapter 5.

Chapter 5

Conclusion and Recommendations

This chapter summarises the key findings of the research, discusses the challenges encountered during the experimental work, and presents recommendations for future research directions in Arabic QE for RAG systems.

5.1 Conclusions

This thesis investigated the extent to which LLM-based QE—generated blindly or steered by the target corpus—can improve Arabic information retrieval across sparse, dense, and hybrid retrieval paradigms. The following conclusions were drawn from the experimental results.

1. **Baseline establishment and error analysis.** Dense (mDPR) and sparse (BM25S) retrieval baselines were successfully established on the MIRACL Arabic benchmark, confirming their complementary strengths: mDPR achieved superior ranking quality (NDCG@10 = 0.4993, MRR = 0.5328), while BM25S achieved broader retrieval coverage (Recall@100 = 0.8577). Error analysis on the dense baseline revealed that 34% of queries failed (NDCG@10 < 0.3), and coverage analysis showed that no relevant passage was surfaced within the top 100 for approximately one query in ten (coverage = 90.1% at rank 100), establishing a recall ceiling that ranking improvements alone could not lift. The analysis further revealed that the shortest queries underperformed, achieving only 72% of long-query performance (NDCG@10 = 0.345 vs. 0.476). Although performance did not increase monotonically with length (the overall length–performance correlation was negligible, $r \approx -0.01$), the specific weakness of very short queries validated the selection of query expansion as the primary enhancement technique. Manual inspection of the worst-performing queries additionally identified three recurring linguistic failure patterns—vocabulary mismatch between Arabic synonyms, orthographic variation in named entities, and diacritic

sensitivity—each of which operates at the level of surface form and is therefore addressable by query-side intervention (Section 4.2.4).

2. **Query2Doc transfers effectively to Arabic.** The Query2Doc technique, originally developed for English using GPT-3 (175B parameters) with few-shot prompting, was successfully adapted for Arabic zero-shot application using models as small as 3 billion parameters. The initial experiment with Qwen 2.5 3B achieved a +8.9% improvement in NDCG@10, exceeding the original paper's reported improvement range of +2–5% despite using a model 58 times smaller and without task-specific demonstrations. Practical execution on freely available cloud GPUs was achieved through a set of engineering optimisations—batched generation with left-padding, a 128-token generation limit, and half-precision inference with gradient computation disabled—which reduced pseudo-document generation over the full 2,896-query development set from an estimated eight hours of naïve sequential processing to approximately forty minutes per model on a T4 GPU (Section 3.4.4). Models whose native-precision footprint exceeded the available VRAM were additionally quantised to 4-bit NF4 (Section 3.5.5), and a two-notebook workflow separating query generation from retrieval evaluation allowed each set of enhanced queries to be reused across both retrieval paradigms without regeneration (Section 3.1.4).

3. **Comprehensive model comparison.** Ten openly available LLMs were evaluated using a standardised protocol across both dense and sparse retrieval. All nine viable models improved dense retrieval performance, with improvements ranging from +3.7% (SILMA Kashif-2B) to +23.5% (Aya Expanse 8B) in NDCG@10. Aya Expanse 8B was identified as the best overall model, achieving the highest dense improvement (+23.5%) and the second-highest BM25 improvement (+9.2%). Jais-2-8B was identified as the best model for sparse retrieval (+10.8% NDCG@10), uniquely benefiting from its 150,000-token Arabic-centric vocabulary. For resource-constrained environments, Qwen3-4B achieved +14.0% dense improvement while fitting on a T4 GPU in FP16 precision without quantisation.

4. **Patterns observed across model families.** Several cross-cutting patterns were identified from the model comparison. Larger models tended to produce better dense-retrieval expansions, although across the heterogeneous model set parameter count is confounded with architecture and training data; the cleanest evidence comes from the Qwen family,

where size and dense NDCG@10 are strongly associated once architecture is held constant (Spearman $\rho = 1.00$ across the 3B, 4B, 7B, and 8B models). Generational improvement within the Qwen family was also confirmed, with Qwen3 outperforming Qwen 2.5 at both the 3–4B and 7–8B scales. Notably, a model's Arabic-capability score on the OALL did not directly predict its query expansion quality; training-data volume and diversity were found to be more significant factors than Arabic-specific architecture or OALL ranking.

5. **Dense and sparse retrieval respond differently to QE.** The most practically significant finding was the divergent behaviour of dense and sparse retrieval under QE. Dense retrieval benefited universally (9 of 9 models improved), as additional semantic content improved the query embedding representation. BM25 was degraded by 6 of 9 models due to term dilution, where the pseudo-document vocabulary diluted the importance of original query keywords in the BM25 scoring function. Only models with strong Arabic vocabulary (Jais-2, Aya Expanse) or sufficient multilingual breadth (Qwen 2.5 7B) overcame this limitation. This finding has direct implications for RAG system design: QE strategies must be adapted to the retrieval paradigm, and dense retrieval systems are more robust recipients of LLM-generated query expansions.

6. **Query repetition resolves sparse retrieval degradation.** The BM25 term-dilution problem, initially identified for 6 of 9 models (Section 4.3.2.1), was fully resolved through query repetition. By prepending the original query before the pseudo-document, all nine models were brought above the BM25 baseline at their optimal repetition setting. The optimum was found to be model-dependent: the two strongest Arabic-capable models (Aya Expanse 8B and Jais-2 8B) peaked under adaptive repetition ($\beta = 2$), while the remaining seven models peaked at fixed counts of five to ten repetitions—differences attributed to variation in pseudo-document length rather than to model size (Section 4.6). Aya Expanse 8B with $\beta = 2$ achieved 0.5855 NDCG@10—a 26.7% improvement over the BM25 baseline and a substantial recovery from the 0.5046 result at $\beta = 1$ (Section 4.6).

7. **Hybrid retrieval establishes a strong non-QE ceiling.** Combining BM25S and mDPR through RRF ($k = 20$) achieved 0.6267 NDCG@10—a 35.6% improvement over BM25 alone—without any QE, confirming the complementarity of lexical and semantic retrieval observed in Section 4.1.1 (Section 4.7).

8. **Corpus-steered expansion validates the corpus grounding hypothesis.** The CSQE

pipeline, which grounds LLM expansion in first-pass retrieved documents, achieved 0.6157 NDCG@10 on BM25 alone—nearly matching the no-QE hybrid baseline and substantially outperforming blind Query2Doc. Applied to the dense retriever instead, the same expansion raised mDPR from 0.4993 to 0.5915 NDCG@10 (+18.5%), an improvement smaller than that obtained on BM25; the expanded query was therefore found to be better matched to lexical than to semantic retrieval, an asymmetry that was exploited in the fusion experiments (Section 4.8.1). Per-query error analysis confirmed the underlying mechanism: when first-pass retrieval succeeds, CSQE anchors the expansion to the correct Wikipedia entity, achieving near-ceiling performance (0.8877 NDCG@10 on first-pass-relevant queries). The combined system (BM25-expanded RRF: BM25+CSQE + Dense original query) achieved **0.7137 NDCG@10—a 54.5% improvement over BM25 alone and a 13.9% improvement over the no-QE hybrid baseline** (Sections 4.8 and 4.9).

9. **Corpus-grounded and blind expansion are complementary.** A component ablation isolated the two halves of the CSQE expansion. On BM25 alone, corpus-only expansion achieved 0.5381 NDCG@10 and blind-only expansion 0.5752, while the combined two-corpus-plus-two-blind configuration achieved 0.6157—an improvement of +0.0405 over the stronger single component, confirming that the two sources are not substitutes for one another (Section 4.8.2). The same ordering was preserved once the expanded BM25 run was fused with the original-query dense run (0.6616, 0.7082 and 0.7137 respectively), although the margin over blind-only narrowed to +0.0055, because the dense retriever supplies a semantic signal that does not depend on expansion quality and therefore absorbs much of the corpus component's marginal contribution (Section 4.9.1). The mechanism is interpreted as follows: the corpus samples anchor the expansion to vocabulary attested in the target corpus, reducing entity misidentification, while the blind samples widen answer-space coverage beyond the first-pass retrieval window.

10. **Retriever-specific query representation is critical.** A key finding is that applying CSQE asymmetrically—only to the BM25 retriever—outperformed both alternative placements: under the same RRF fusion, expanding both retrievers yielded 0.6936 NDCG@10 and expanding only the dense retriever yielded 0.6474, against 0.7137 for the sparse-only assignment (Table 4.17). Applying CSQE to both retrievers reduced the complementarity between their ranked lists, lowering the fusion ceiling; applying it only to BM25 preserved

the dense retriever's independent semantic signal. This retriever-specific representation principle has practical implications for similar hybrid retrieval pipelines (Section 4.9).

11. **Per-query analysis localises where the gains arise.** Measured against the blind-expansion baseline (Aya Expanse 8B, BM25, $n = 1$; 0.5046 NDCG@10), the final system improved 56.8% of the 2,896 development queries and regressed on 16.6%, for a mean per-query gain of +0.1890 NDCG@10 (Section 4.10.1). These gains were not distributed evenly. The shortest queries (1–3 words) gained the most in proportional terms (+43.6%), medium-length queries (4–8 words) the most in absolute terms (+0.197), and the longest queries (9+ words) the least (+0.132, or +23.3%)—confirming that the expansion compensates chiefly for the information poverty of underspecified queries, exactly the weakness identified in the baseline error analysis. First-pass retrieval quality was found to be the largest single modulator of that benefit: for the 36.6% of queries whose BM25 first pass returned a relevant document at rank 1, the system achieved 0.8877 NDCG@10, against 0.5814 for the remainder (Section 4.10.2).

12. **Overall.** LLM-based QE was found to be an effective and modular strategy for improving Arabic information retrieval under all three retrieval paradigms examined, provided that the form of the expansion is adapted to the paradigm. The largest gains were obtained when the expansion was steered by the target corpus and applied asymmetrically—to the sparse retriever alone—within a hybrid sparse–dense pipeline, which raised retrieval quality from a BM25 baseline of 0.4621 to 0.7137 NDCG@10, an improvement of 54.5%, and exceeded the strongest unenhanced hybrid baseline by 13.9%. This result was obtained with an openly available 8-billion-parameter model, supported by a comparison spanning openly available LLMs of 2–8 billion parameters, without API costs or dependence on proprietary models; the approach is therefore practical for Arabic RAG deployments, subject to the licence of the chosen generator—the best-performing model, Aya Expanse 8B, carries a non-commercial licence, while Apache-2.0 alternatives such as Jais-2-8B follow at a small cost in effectiveness (Table A.1).

5.2 Challenges

Several challenges were encountered during the course of this research.

1. **Resource constraints.** All experiments were conducted on Google Colab with NVIDIA T4 (15 GB) and A100 (40 GB) GPUs. This limited model sizes to 8 billion parameters (with 4-bit quantisation for larger models) and required significant engineering optimisation—batch processing, reduced token generation (128 tokens), and a two-notebook workflow separating query generation from retrieval evaluation—to achieve practical runtimes of approximately 40 minutes per full generation run.

2. **BM25 term dilution.** QE degraded BM25 performance for 6 of 9 models in the initial experiments. The query repetition strategy recommended by the original Query2Doc paper [6] ($n = 5$ repetitions of the original query before concatenation) was not implemented at that stage, as the focus was on establishing the baseline Query2Doc behaviour across models. This challenge was subsequently resolved through the query repetition technique (Section 4.6), which recovered BM25 performance for all six models that had initially degraded.

3. **Dropped model.** One of the ten models evaluated was unusable for Arabic QE: ALLaM-7B, a preview-stage model, exhibited a sentencepiece tokeniser artefact that leaked special characters into the pseudo-documents, producing a catastrophic -48.9% decline in NDCG@10. This failure highlights the risk of relying on preview-stage models—whose tokeniser behaviour may not yet be validated for generation—in Arabic natural language processing (NLP) tasks.

4. **Dataset scope.** The MIRACL Arabic benchmark contains exclusively MSA text. The effectiveness of QE for dialectal Arabic queries—Egyptian, Levantine, Gulf, Maghrebi, and other regional varieties—could not be evaluated within the scope of this work. Given the diglossic nature of Arabic, where users may formulate queries in dialect while knowledge bases are written in MSA, this represents a significant gap in the evaluation. A related limitation is that the MIRACL Arabic corpus provides no structural metadata—article boundaries, section headings, or passage hierarchy—beyond raw passage text, which prevented

the knowledge-base-aware, "chunking-aware" QE originally envisaged (Section 5.3).

5. **Single QE technique.** Only the Query2Doc technique (generative query expansion via pseudo-document concatenation) was evaluated. Other approaches—HyDE, GRF, query rewriting, and iterative refinement—were not compared. The results therefore reflect the potential of one specific technique rather than the broader landscape of QE methods.

6. **Baseline retriever strength.** The mDPR dense retriever used in this work was pre-trained on MS MARCO and not fine-tuned on MIRACL Arabic. The magnitude of improvement observed may therefore differ when QE is applied to retrieval models that have been specifically fine-tuned on Arabic data.

7. **First-pass quality dependence.** CSQE performance was found to be strongly conditioned on first-pass retrieval quality. When BM25 first-pass retrieval returned irrelevant documents (36% of regression cases), the LLM expansion was grounded on incorrect content, causing regressions that blind QE did not exhibit. Arabic homonym sensitivity in BM25 (e.g., ماهو retrieved as a dialectal phrase rather than a question prefix) was identified as the primary trigger (Section 4.10).

8. **Generator licensing.** The generator used for the corpus-steered pipeline, Aya Expanse 8B, is distributed under a CC-BY-NC-4.0 licence and may therefore not be used commercially. Apache-2.0 models of the same scale were evaluated in the model comparison, where Jais-2-8B trailed Aya Expanse by 2.4% on dense retrieval and 2.1% on sparse retrieval with repetition, and in fact led it before repetition was applied (Sections 4.4 and 4.6). The CSQE and fusion experiments, however, were conducted with Aya Expanse alone, so the cost of substituting a permissively licensed generator into the final pipeline was not measured directly.

5.3 Recommendations for Future Work

Based on the findings and limitations of this research, the following recommendations are proposed for future work. These recommendations are ordered from direct extensions of the current work to broader research directions.

1. **Knowledge-base-aware QE.** The current Query2Doc implementation generates pseudo-documents without any knowledge of the target corpus structure. Future work should investigate injecting structural information from the knowledge base—such as passage hierarchy, article boundaries, section headings, and metadata—into the QE prompt. This "chunking-aware" approach would enable the LLM to generate context-aware expansions tailored to the actual content organisation of the knowledge base, potentially bridging the gap between query-side and corpus-side optimisation.

2. **Stronger embedding models.** The effectiveness of QE should be evaluated with state-of-the-art Arabic-capable embedding models. BGE-M3 remains a strong, openly available multilingual retriever validated on MIRACL [33]; more recent encoders have since surpassed the older multilingual-E5-large, including the multilingual Qwen3-Embedding-8B [42] and the Arabic-centric Swan-Large [43], which outperforms multilingual-E5-large on most Arabic tasks. As these models are ranked primarily on general embedding benchmarks—the Massive Text Embedding Benchmark (MTEB), its multilingual extension (MMTEB), and Arabic Semantic Textual Similarity (STS)—rather than MIRACL Arabic retrieval specifically, a small validation run is advisable before adoption. This evaluation would establish whether QE provides additive improvement even over strong retrievers, or whether the benefits diminish as the baseline retriever becomes more capable.

3. **Dialectal Arabic evaluation.** The effectiveness of QE for dialectal Arabic queries should be evaluated using datasets that include regional Arabic varieties. The MIRACL Arabic benchmark contains exclusively MSA text, so the extent to which these findings generalise to queries formulated in Egyptian, Levantine, Gulf, or Maghrebi Arabic remains an open question.

4. **Few-shot and chain-of-thought prompting.** The current work used exclusively zero-shot prompting for pseudo-document generation. Few-shot prompting with curated Arabic demonstration examples may improve pseudo-document quality by providing the model with explicit patterns for Arabic query expansion. Chain-of-thought reasoning before expansion—where the model analyses the query's information needs before generating the pseudo-document—may further improve quality for complex or ambiguous queries.

5. **Multi-stage QE.** The current approach applies a single round of QE. Future work should investigate iterative refinement, where the results of a first retrieval pass inform a second

round of QE through a relevance feedback loop. This multi-stage approach could combine Query2Doc with re-ranking techniques, using the initially retrieved passages to generate more targeted query expansions.

6. **First-pass quality gate.** A quality filter should be implemented that checks lexical overlap between the top-1 BM25 document and the query before grounding the LLM expansion. When overlap falls below a threshold, the system should fall back to blind QE. This is expected to recover approximately 131 Type B regression queries identified in the error analysis (Section 4.10.4).

7. **Asymmetric expansion weighting.** For Type A regressions (strong BM25 queries), reducing the expansion weight α or using field-based BM25 indexing (original query in one field, expansion in another with lower boost) would prevent vocabulary dilution for queries that BM25 already handles well (Section 4.10.4).

8. **Permissively licensed generator for the final pipeline.** The CSQE and fusion experiments should be repeated with an Apache-2.0 generator such as Jais-2-8B, in order to quantify the effectiveness cost of removing the non-commercial licence constraint from the best-performing configuration.

9. **CSQE with stronger dense retrievers.** The current pipeline uses mDPR as a relatively weak dense baseline (Section 3.2.1). Testing CSQE+Hybrid with stronger dense retrievers such as BGE-M3 or Qwen3-Embedding-8B would assess whether the 54.5% BM25 improvement translates to similar gains over stronger dense retrievers, and whether the retriever-specific representation finding generalises.

Bibliography

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," arXiv.org, 2017. [Online]. Available: https://arxiv.org/abs/1706.03762

[2] M. Song and M. Zheng, "A survey of query optimization in large language models," arXiv.org, 2024. [Online]. Available: https://arxiv.org/abs/2412.17558

[3] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive nlp tasks," arXiv.org, 04 2021. [Online]. Available: https://arxiv.org/abs/2005.11401

[4] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[5] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, "Dense passage retrieval for open-domain question answering," arXiv.org, 2020. [Online]. Available: https://arxiv.org/abs/2004.04906

[6] L. Wang, N. Yang, and F. Wei, "Query2doc: Query expansion with large language models," arXiv.org, 10 2023. [Online]. Available: https://arxiv.org/abs/2303.07678

[7] S. Bruch, S. Gai, and A. Ingber, "An analysis of fusion functions for hybrid retrieval," *ACM Transactions on Information Systems*, vol. 42, no. 1, 2024, arXiv:2210.11934.

[8] L. Gao, X. Ma, J. Lin, and J. Callan, "Precise zero-shot dense retrieval without relevance labels," 12 2022. [Online]. Available: https://arxiv.org/pdf/2212.10496

[9] I. Mackie, S. Chatterjee, and J. Dalton, "Generative relevance feedback with large language models," in *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2023.

[10] C.-M. Chan, C. Xu, R. Yuan, H. Luo, W. Xue, Y. Guo, and J. Fu, "Rq-rag: Learning to refine queries for retrieval augmented generation," arXiv.org, 03 2024. [Online]. Available: https://arxiv.org/abs/2404.00610

[11] X. Ma, Y. Gong, P. He, H. Zhao, and N. Duan, "Query rewriting for retrieval-augmented large language models," arXiv.org, 10 2023. [Online]. Available: https://arxiv.org/abs/2305.14283

[12] H. S. Zheng, S. Mishra, X. Chen, H.-T. Cheng, E. H. Chi, Q. V. Le, and D. Zhou, "Take a step back: Evoking reasoning via abstraction in large language models," arXiv.org, 10 2023. [Online]. Available: https://arxiv.org/abs/2310.06117

[13] S. R. El-Beltagy and M. A. Abdallah, "Exploring retrieval augmented generation in arabic," arXiv.org, 2024. [Online]. Available: https://arxiv.org/abs/2408.07425

[14] X. Zhang, N. Thakur, O. Ogundepo, E. Kamalloo, D. Alfonso-Hermelo, X. Li, Q. Liu, M. Rezagholizadeh, and J. Lin, "Miracl: A multilingual retrieval dataset covering 18 diverse languages," arXiv.org, 2023. [Online]. Available: https://arxiv.org/abs/2210.09984

[15] A. Abdallah, M. Kasem, M. Abdalla, M. Mahmoud, M. Elkasaby, Y. Elbendary, and A. Jatowt, "ArabicaQA: A comprehensive dataset for arabic question answering," in *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '24)*, 2024, arXiv:2403.17848.

[16] J. H. Clark, E. Choi, M. Collins, D. Garrette, T. Kwiatkowski, V. Nikolaev, and J. Palomaki, "TyDi QA: A benchmark for information-seeking question answering in typologically diverse languages," *Transactions of the Association for Computational Linguistics*, vol. 8, pp. 454–470, 2020. [Online]. Available: https://aclanthology.org/2020.tacl-1.30/

[17] M. A. Hasan, M. Hasanain, F. Ahmad, S. R. Laskar, S. Upadhyay, V. N. Sukhadia, M. Kutlu, S. A. Chowdhury, and F. Alam, "NativQA: Multilingual culturally-aligned natural query for LLMs," arXiv.org, 2024. [Online]. Available: https://arxiv.org/abs/2407.09823

[18] P. Sen, A. F. Aji, and A. Saffari, “Mintaka: A complex, natural, and multilingual dataset for end-to-end question answering,” in *Proceedings of the 29th International Conference on Computational Linguistics (COLING)*, 2022. [Online]. Available: https://aclanthology.org/2022.coling-1.138/

[19] H. Mozannar, E. Maamary, K. El Hajal, and H. Hajj, “Neural arabic question answering,” in *Proceedings of the Fourth Arabic Natural Language Processing Workshop (WANLP)*, 2019, arXiv:1906.05394. [Online]. Available: https://aclanthology.org/W19-4612/

[20] X. H. Lü, “BM25S: An efficient python implementation of BM25,” 2024, software library. Accessed: July 29, 2026. [Online]. Available: https://github.com/xhluca/bm25s

[21] I. Mackie, S. Chatterjee, and J. Dalton, “Generative and pseudo-relevant feedback for sparse, dense and learned sparse retrieval,” arXiv.org, 2023. [Online]. Available: https://arxiv.org/abs/2305.07477

[22] Y. Lei, Y. Cao, T. Zhou, T. Shen, and A. Yates, “Corpus-steered query expansion with large language models,” in *Proceedings of the 18th Conference of the European Chapter of the ACL (EACL): Short Papers*, 2024, arXiv:2402.18031. [Online]. Available: https://aclanthology.org/2024.eacl-short.34

[23] L. Zhang, Y. Wu, Q. Yang, and J.-Y. Nie, “Exploring the best practices of query expansion with large language models,” in *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024. [Online]. Available: https://aclanthology.org/2024.findings-emnlp.103

[24] Y. Xia, J. Wu, S. Kim, T. Yu, R. A. Rossi, H. Wang, and J. McAuley, “Knowledge-aware query expansion with large language models for textual and relational retrieval,” in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the ACL (NAACL): Long Papers*, 2025. [Online]. Available: https://aclanthology.org/2025.naacl-long.216

[25] A. Yang, G. Penha, E. Palumbo, and H. Bouchard, “Aligned query expansion: Efficient query expansion for information retrieval through LLM alignment,” arXiv.org, 2025. [Online]. Available: https://arxiv.org/abs/2507.11042

[26] Y. Lei, T. Shen, and A. Yates, “ThinkQE: Query expansion via an evolving thinking process,” in *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025. [Online]. Available: https://aclanthology.org/2025.findings-emnlp.965

[27] Y. Zhang, P. Jia, D. Xu, Y. Wen, X. Li, Y. Wang, W. Zhang, X. Li, W. Gan, H. Guo, Y. Liu, and X. Zhao, “Personalize before retrieve: LLM-based personalized query expansion for user-centric retrieval,” arXiv.org, 2025. [Online]. Available: https://arxiv.org/abs/2510.08935

[28] Y. Yoon, J. Jung, S. Yoon, and K. Park, “Hypothetical documents or knowledge leakage? rethinking LLM-based query expansion,” arXiv.org, 2025. [Online]. Available: https://arxiv.org/abs/2504.14175

[29] O. Young, Y. Fan, R. Zhang, J. Guo, M. de Rijke, and X. Cheng, “GaQR: An efficient generation-augmented question rewriter,” in *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management (CIKM ’24)*, 2024, pp. 4228–4232.

[30] A. Bhusal, “Query augmentation for information retrieval (ir) using large language model (llm),” Master’s thesis, University of Alabama in Huntsville, 2024, Accessed: July 29, 2026. [Online]. Available: https://louis.uah.edu/uah-theses/663/

[31] K. Zhang, Z. Sun, W. Yu, X. Zang, K. Zheng, Y. Song, H. Li, and J. Xu, “Qe-rag: A robust retrieval-augmented generation benchmark for query entry errors,” arXiv.org, 2025. [Online]. Available: https://arxiv.org/abs/2504.04062

[32] O. Macmillan-Scott, R. Goworek, and E. B. Özyiğit, “Generative query expansion with multilingual LLMs for cross-lingual information retrieval,” arXiv.org, 2025. [Online]. Available: https://arxiv.org/abs/2511.19325

[33] J. Alsubhi, M. D. Alahmadi, A. Alhusayni, I. Aldailami, I. Hamdine, A. Shabana, Y. Iskandar, and S. Khayyat, “Optimizing rag pipelines for arabic: A systematic analysis of core components,” arXiv.org, 2025. [Online]. Available: https://arxiv.org/abs/2506.06339v1

[34] L. Liu and M. Zhang, “Exp4Fuse: A rank fusion framework for enhanced sparse retrieval using large language model-based query expansion,” in *Findings of the Association for*

Computational Linguistics: ACL 2025, 2025, arXiv:2506.04760. [Online]. Available: https://aclanthology.org/2025.findings-acl.9

[35] J. Lin, X. Ma, S.-C. Lin, J.-H. Yang, R. Pradeep, and R. Nogueira, "Pyserini: A python toolkit for reproducible information retrieval research with sparse and dense representations," arXiv.org, 2021. [Online]. Available: https://arxiv.org/abs/2102.10073

[36] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized language models," arXiv.org, 2023. [Online]. Available: https://arxiv.org/abs/2305.14314

[37] M. S. Bari, Y. Alnumay, N. A. Alzahrani, N. M. Alotaibi, H. A. Alyahya, S. AlRashed, F. A. Mirza, S. Z. Alsubaie, H. A. Alahmed, G. Alabduljabbar *et al.*, "ALLam: Large language models for arabic and english," in *The Thirteenth International Conference on Learning Representations*, 2025, Accessed: July 29, 2026. [Online]. Available: https://openreview.net/forum?id=MscdsFVZrN

[38] J. Dang, S. Singh, D. D'souza, A. Ahmadian *et al.*, "Aya expanse: Combining research breakthroughs for a new multilingual frontier," arXiv.org, 2024. [Online]. Available: https://arxiv.org/abs/2412.04261

[39] N. Sengupta, S. K. Sahu, B. Jia, S. Katipomu, H. Li, F. Koto, W. Marshall, G. Gosal, C. Liu, Z. Chen *et al.*, "Jais and jais-chat: Arabic-centric foundation and instruction-tuned open generative large language models," arXiv.org, 2023. [Online]. Available: https://arxiv.org/abs/2308.16149

[40] Qwen Team, "Qwen3 technical report," arXiv.org, 2025. [Online]. Available: https://arxiv.org/abs/2505.09388

[41] P. Bajaj, D. Campos, N. Craswell, L. Deng, J. Gao, X. Liu, R. Majumder, A. McNamara, B. Mitra, T. Nguyen, M. Rosenberg, X. Song, A. Stoica, S. Tiwary, and T. Wang, "MS MARCO: A human generated machine reading comprehension dataset," arXiv.org, 2016. [Online]. Available: https://arxiv.org/abs/1611.09268

[42] Qwen Team, "Qwen3 embedding: Advancing text embedding and reranking through foundation models," arXiv.org, 2025. [Online]. Available: https://arxiv.org/abs/2506.05176

[43] G. Bhatia, E. M. B. Nagoudi, A. El Mekki, F. Alwajih, and M. Abdul-Mageed, "Swan and ArabicMTEB: Dialect-aware, arabic-centric, cross-lingual, and cross-cultural embedding models and benchmarks," arXiv.org, 2024. [Online]. Available: https://arxiv.org/abs/2411.01192

[44] Technology Innovation Institute, "Falcon-h1: A family of hybrid-head language models," arXiv.org, 2025. [Online]. Available: https://arxiv.org/abs/2507.22448

[45] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," arXiv.org, 2024. [Online]. Available: https://arxiv.org/abs/2312.00752

[46] SILMA AI, "SILMA kashif-2b-instruct: Arabic RAG-optimized language model," 2024, model card. Accessed: July 29, 2026. [Online]. Available: https://huggingface.co/silma-ai/SILMA-Kashif-2B-Instruct-v1.0

[47] A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei *et al.*, "Qwen2.5 technical report," arXiv.org, 2024. [Online]. Available: https://arxiv.org/abs/2412.15115

[48] Gemma Team, Google DeepMind, "Gemma 3 technical report," arXiv.org, 2025. [Online]. Available: https://arxiv.org/abs/2503.19786

Appendix A

Model Details

This appendix gives the full description of each language model evaluated in this thesis, together with the comparative summary table. Section 2.4 of Chapter 2 introduces the two model groups and states the selection criteria; the detail is collected here so that the main text is not interrupted by ten model descriptions. Section A.4 documents the model-specific engineering workarounds referred to in Section 3.5.4.

A.1 Summary of Language Models

Table A.1 presents a comparative summary of all language models used in the experiments. The models span a range of parameter counts from 2 billion to 8 billion, encompassing Arabic-specialized and multilingual architectures.

Table A.1: Summary of language models used in the experiments.

Model	Developer	Params	Architecture	Arabic Focus	License
Arabic-Specialized					
Falcon-H1-3B	TII	3.15B	Hybrid Mamba2-Trans.	Primary	Open
Jais-2-8B	MBZUAI/Inception	8.09B	Decoder-only Trans.	Primary	Apache 2.0
ALLaM-7B	SDAIA	7.0B	LlamaForCausalLM	Primary	Apache 2.0
SILMA Kashif-2B	SILMA AI	2.0B	Decoder-only Trans.	Primary	Open
Multilingual					
Qwen 2.5 3B	Alibaba	3.09B	Decoder-only Trans.	29+ langs	Qwen Research
Qwen 2.5 7B	Alibaba	7.0B	Decoder-only Trans.	29+ langs	Apache 2.0
Qwen3-4B	Alibaba	4.0B	Decoder-only Trans.	119 langs	Apache 2.0
Qwen3-8B	Alibaba	8.0B	Decoder-only Trans.	119 langs	Apache 2.0
Gemma 3 4B	Google DeepMind	4.0B	Decoder-only Trans.	140+ langs	Gemma TOU
Aya Expanse 8B	Cohere Labs	8.0B	Decoder-only Trans.	101 langs	CC-BY-NC

A.2 Arabic-Specialized Models

A.2.1 Falcon-H1-Arabic-3B

Falcon-H1-Arabic-3B is a 3.15 billion parameter language model developed by the Technology Innovation Institute (TII), Abu Dhabi [44]. Unlike conventional Transformer-based LLMs, Falcon-H1 employs a hybrid Mamba2-Transformer architecture that combines State Space Model (SSM) layers [45] with standard multi-head attention layers and Swish-Gated Linear Unit (SwiGLU) activation functions. The model contains 32 layers with a hidden size of 2,560, utilizing 32 Mamba heads alongside 10 attention heads with Grouped Query Attention (GQA, 2 key-value heads).

The hybrid architecture was designed to combine the linear-time sequence processing efficiency of SSMs with the strong in-context learning capabilities of attention mechanisms. The model supports a context length of 128K tokens and was purpose-built for Arabic, listing it as a primary language alongside 17 additional languages.

On the Open Arabic LLM Leaderboard (OALL v2), Falcon-H1-Arabic-3B achieved an average score of approximately 62%, roughly 10 points ahead of competing models at the same parameter count. The model was released in May 2025 under an open license. In 16-bit floating-point (FP16) precision, the model requires approximately 10–11 GB of VRAM including SSM state buffers, fitting on both T4 and A100 GPUs without quantization.

A.2.2 Jais-2-8B

Jais-2-8B-Chat is an 8.09 billion parameter Arabic-centric language model developed collaboratively by Inception (G42), Mohamed bin Zayed University of Artificial Intelligence (MBZUAI), and Cerebras [39]. The model employs a standard decoder-only Transformer architecture with Rotary Position Embeddings (RoPE), squared Rectified Linear Unit (Squared-ReLU) activation, and LayerNorm. It contains 32 layers with a hidden size of 3,328 and 26 attention heads.

A distinguishing feature of Jais-2 is its Arabic-centric vocabulary of 150,272 tokens, trained

from scratch to optimize Arabic tokenization efficiency. The model was pre-trained on 2.6 trillion tokens—6.6 times more data than its predecessor Jais-1—with substantial Arabic and English representation. The instruction-tuned variant was refined through a three-stage alignment pipeline comprising large-scale SFT on Arabic and English prompt-response pairs, followed by DPO, and finally Group Relative Policy Optimization (GRPO).

On the AraGen-12-24 benchmark, Jais-2-8B-Chat achieved an overall 3C3H score of 58.64%, outperforming Fanar-1-9B and ALLaM-7B. The model was released in December 2024 under Apache 2.0 license (gated access). It requires 16-bit brain floating-point (BF16) precision due to Squared-ReLU activation overflow in FP16, using approximately 16.6 GB of VRAM on the A100.

A.2.3 ALLaM-7B

ALLaM-7B-Instruct-preview is a 7 billion parameter Arabic-English language model developed by the National Center for Artificial Intelligence (NCAI) at the Saudi Data and Artificial Intelligence Authority (SDAIA) [37]. Published at the International Conference on Learning Representations (ICLR) 2025, ALLaM employs the LlamaForCausalLM architecture with an expanded vocabulary of 64,000 tokens (doubled from Llama-2's 32K to accommodate Arabic).

The model was trained from scratch in two phases: (1) a foundation phase on 4 trillion English tokens, followed by (2) continued pre-training on 1.2 trillion mixed Arabic-English tokens at a 45%/45%/10% Arabic/English/code ratio, totaling 5.2 trillion training tokens. A substantial share of the Arabic pre-training data was derived from machine-translated English sources. Instruction tuning and DPO-based alignment were subsequently applied.

On the Arabic Massive Multitask Language Understanding (MMLU) benchmark, ALLaM achieved a 0-shot score of 67.78. The model was released in February 2025 under Apache 2.0 license. Notably, the publicly available version carries "preview" (alpha) status, indicating that it may not fully reflect the capabilities described in the published paper. The model requires approximately 14.5 GB of VRAM in FP16 on the A100.

A.2.4 SILMA Kashif-2B

SILMA Kashif-2B-Instruct is a 2 billion parameter language model developed by SILMA AI, an Arabic artificial intelligence (AI) startup, with a specific focus on Arabic RAG applications [46]. The model was optimized for extractive QA in Arabic, achieving a RAG Question Answering (RAGQA) benchmark score of 0.3478, which placed it at the top of the 3–9 billion parameter range on this benchmark.

The model supports a context length of 12K tokens and operates in FP16 precision, requiring approximately 4–5 GB of VRAM. Its small size makes it the most computationally efficient model in the comparison, suitable for deployment on consumer-grade hardware without quantization.

A.3 Multilingual Models

A.3.1 Qwen 2.5 3B

Qwen 2.5-3B-Instruct is a 3.09 billion parameter multilingual language model developed by the Qwen Team at Alibaba Cloud [47]. The model employs a standard decoder-only Transformer architecture with GQA (16 query heads, 2 key-value heads), Root Mean Square Normalization (RMSNorm), Sigmoid Linear Unit (SiLU) activation, and a vocabulary of 151,936 tokens. It was pre-trained on approximately 18 trillion tokens spanning 29 or more languages, with Arabic included as a supported language.

Qwen 2.5-3B-Instruct was released in September 2024 under the Qwen Research License (non-commercial use). The model requires approximately 6 GB of VRAM in FP16, fitting comfortably on the T4 GPU. In the context of this thesis, Qwen 2.5-3B served as the reference model for the first Query2Doc experiment, establishing the initial QE baseline.

A.3.2 Qwen 2.5 7B

Qwen 2.5-7B-Instruct is the 7 billion parameter variant in the same model family [47]. It shares the same architecture and tokenizer as the 3B variant but with a wider hidden dimension and more attention heads, resulting in stronger performance across multilingual benchmarks (MMLU: 74.2). The model requires 4-bit quantization to fit on the T4 GPU but operates in full FP16 precision on the A100. Pre-quantized versions are available from both the Qwen team and the Unsloth community.

A.3.3 Qwen3-4B

Qwen3-4B is a 4.0 billion parameter (3.6 billion non-embedding) next-generation language model from the Qwen Team [40]. Released in May 2025, it represents a significant architectural evolution over Qwen 2.5: the number of supported languages and dialects was expanded from 29 to 119 (with broader Arabic coverage), training data was doubled to approximately 36 trillion tokens, and attention capacity was increased to 32 query heads with 8 key-value heads.

A notable feature of Qwen3 is its built-in *thinking mode*, which generates internal reasoning within special tags before producing the final answer. For query expansion tasks, this thinking mode must be explicitly disabled to prevent reasoning artifacts from contaminating the generated pseudo-documents. The model also requires sampling-based decoding; greedy decoding causes performance degradation and repetitive output.

Qwen3-4B matches or exceeds Qwen 2.5-7B on many benchmarks despite having fewer parameters, achieving an MMMLU (multilingual MMLU) score of 71.42. It operates in FP16 precision requiring approximately 8.5 GB of VRAM.

A.3.4 Qwen3-8B

Qwen3-8B is the 8 billion parameter variant of the Qwen3 family [40], sharing the same architectural innovations as Qwen3-4B including 119-language support, thinking mode, and 36 trillion training tokens. As the larger variant, it provides stronger language understanding and

generation capabilities. Like Qwen3-4B, thinking tags must be stripped from generated output when used for query expansion. The model operates in FP16 on the A100 GPU.

A.3.5 Gemma 3 4B-IT

Gemma 3 4B-IT is a 4 billion parameter instruction-tuned language model developed by Google DeepMind [48]. Released in March 2025, it supports over 140 languages and features a context length of 128K tokens. The model employs a standard dense Transformer architecture with multimodal capabilities (vision and text), though only the text modality was used in this thesis.

Despite its broad language coverage, Gemma 3 exhibited the weakest Arabic performance among the tested models, scoring approximately 10 points below Falcon-H1-Arabic-3B on the OALL benchmark. It serves as a lower-bound comparison point in the model evaluation, representing a general-purpose multilingual model without Arabic-specific optimization.

A.3.6 Aya Expanse 8B

Aya Expanse 8B is an 8 billion parameter multilingual language model developed by Cohere Labs (CohereForAI), covering 23 languages with Arabic among the explicitly supported languages [38]. The model was trained using a data-centric approach that emphasizes high-quality multilingual training data, and its developers report that it outperforms comparable open-weight models, including Llama-3.1-8B-Instruct, on multilingual evaluations.

Aya Expanse employs a standard Transformer architecture and was loaded with 4-bit NormalFloat (NF4) quantization in the experiments due to its 8 billion parameter size. Despite the quantization, it demonstrated strong Arabic generation quality, suggesting that architecture and training data quality outweigh precision for this task. The model is distributed under a CC-BY-NC-4.0 (non-commercial) license.

A.4 Model-Specific Technical Issues

Several models required specific engineering workarounds before they could be run under the standardised evaluation protocol of Section 3.5.2. They are documented here for reproducibility.

a. **Falcon-H1-3B:** The hybrid Mamba2-Transformer architecture (described in Section A.2.1) contains a batching bug in the attention mask extension logic. When left-padded batches are used, the 4D causal attention mask is not extended as new tokens are generated, causing a shape mismatch. The workaround was to force `batch_size=1`, resulting in significantly longer generation times but correct outputs.

b. **Jais-2-8B:** This model requires BF16 precision rather than FP16 due to its Squared-ReLU activation function, which produces values that overflow FP16 range. Additionally, the `token_type_ids` parameter must be excluded from the model inputs, and the padding token must be explicitly set to the end-of-sequence token.

c. **Qwen3-4B and Qwen3-8B:** The Qwen3 generation includes "thinking" tokens (internal chain-of-thought reasoning) that must be stripped from the output. The parameter `enable_thinking=False` was passed during generation. Greedy decoding must not be used, as it causes infinite repetition loops.

d. **ALLaM-7B:** As a preview-stage model, ALLaM exhibited a sentencepiece tokenizer artefact where the Unicode character " " (lower one-eighth block) leaked into decoded outputs. This special character, used internally by the sentencepiece tokenizer to denote word boundaries, contaminated the pseudo-documents. The impact on retrieval performance is discussed in Section 4.4.3.

Appendix B

Extended Result Tables

This appendix collects the full result tables whose complete form is reference material rather than argument. In each case the main text carries the rows or the figure that support its claims, and the exhaustive version is reproduced here.

B.1 BM25 Query Repetition: Full Sweep

Table B.1 gives the complete nine-model by eight-configuration repetition sweep summarised in Section 4.6. Figure 4.5 plots the fixed-count columns and Table 4.12 reports every model's best configuration with all four metrics.

Table B.1: BM25 NDCG@10 for nine models under query repetition. Columns $n \in \{1, 3, 5, 7, 10\}$ are fixed-count repetition (Query2Doc style); columns $\beta \in \{2, 4, 6\}$ are MuGI adaptive repetition where n is computed per-query as $n = \max(1, \lfloor |d|/(|q| \cdot \beta) \rfloor)$. MIRACL Arabic dev (2,896 queries). Best value per model in bold.

Model	n=**1**	n=**3**	n=**5**	n=**7**	n=**10**	β=**2**	β=**4**	β=**6**
BM25 alone (no QE)	0.4621	—	—	—	—	—	—	—
Aya Expanse 8B	0.5046	0.5652	0.5832	0.5849	0.5773	**0.5855**	0.5515	0.5256
Jais-2 8B	0.5122	0.5492	0.5529	0.5516	0.5436	**0.5731**	0.5521	0.5350
Qwen3-8B	0.4459	0.5181	0.5319	**0.5328**	0.5254	0.5254	0.4841	0.4591
Qwen 2.5 7B	0.4682	0.5294	**0.5358**	0.5331	0.5257	0.5320	0.4977	0.4774
Qwen3-4B	0.4145	0.5054	0.5239	**0.5244**	0.5188	0.5177	0.4678	0.4347
Gemma 3 4B	0.3447	0.4800	0.5178	**0.5277**	0.5239	0.4915	0.4184	0.3694
Qwen 2.5 3B	0.4090	0.5060	**0.5185**	0.5181	0.5116	0.5046	0.4551	0.4253
Falcon-H1 3B	0.4038	0.4881	0.5082	0.5112	**0.5113**	0.4979	0.4561	0.4266
SILMA 2B	0.4277	0.4733	0.4770	**0.4786**	0.4752	0.4520	0.4312	0.4278

B.2 Hybrid Fusion: Full Convex-Combination Sweep

Table B.2 gives the complete convex-combination α-sweep together with both RRF configurations. Section 4.7 reports the baselines, the best CC setting and both RRF settings inline; Figure 4.7 plots all four metrics across the full sweep.

Table B.2: Hybrid retrieval fusion results on the MIRACL Arabic dev set (2,896 queries), full convex-combination α-sweep. CC scores are min-max normalised per query, with α controlling the Dense weight.

Method	**NDCG@10**	**Recall@10**	**Recall@100**	**MRR**
BM25S alone	0.4621	0.5964	0.8577	0.4836
mDPR alone	0.4993	0.6156	0.8407	0.5328
Hybrid CC $\alpha = 0.1$	0.5248	0.6412	0.9333	0.5584
Hybrid CC $\alpha = 0.2$	0.5533	0.6683	0.9413	0.5861
Hybrid CC $\alpha = 0.3$	0.5830	0.7014	0.9449	0.6161
Hybrid CC $\alpha = 0.4$	0.6137	0.7392	0.9454	0.6426
Hybrid CC $\alpha = 0.5$	**0.6266**	0.7478	0.9458	**0.6577**
Hybrid CC $\alpha = 0.6$	0.6051	0.7289	0.9440	0.6355
Hybrid CC $\alpha = 0.7$	0.5743	0.6963	0.9439	0.6049
Hybrid CC $\alpha = 0.8$	0.5384	0.6634	0.9416	0.5678
Hybrid CC $\alpha = 0.9$	0.4996	0.6297	0.9350	0.5257
Hybrid RRF $k = 20$	**0.6267**	**0.7597**	**0.9466**	0.6517
Hybrid RRF $k = 60$	0.6230	0.7553	0.9466	0.6490

B.3 Summary of All Experiments

The experiments were reported in Chapter 4 in the order they were conducted, each one motivated by the outcome of the last. Read that way the argument is cumulative but dispersed, and the relative standing of any two configurations is not always adjacent on the page. Table B.3 therefore gathers every experiment into a single view, grouped by phase, so that the full progression can be assessed at a glance now that the individual results and their reasoning have been established.

Table B.3: Summary of all experiments grouped by phase: baselines, initial Query2Doc experiments, and expanded experiments covering query repetition, hybrid fusion, and CSQE. Bold values indicate the best result for each metric across all experiments.

Model / Method	Retriever	NDCG@10	Recall@10	MRR	Status
Baselines					
None (baseline)	Dense	0.4993	0.6156	0.5328	Baseline
None (baseline)	BM25	0.4621	0.5964	0.4836	Baseline
None (baseline)	BM25+Dense (RRF)	0.6267	0.7597	0.6517	Best no-QE
Initial Query2Doc experiments					
Qwen 2.5 3B	Dense	0.5435	0.6608	0.5742	OK
Qwen 2.5 3B	BM25	0.4090	0.5384	0.4342	Degraded
Falcon-H1 3B	Dense	0.5359	0.6484	0.5681	OK
Jais-2 8B	Dense	0.6018	0.7161	0.6356	Good
Jais-2 8B	BM25	0.5122	0.6448	0.5397	Best BM25 (n=1)
Qwen3-4B	Dense	0.5691	0.6824	0.6015	Good
ALLaM 7B	Dense	0.2550	0.3335	0.2708	Dropped
Aya 8B	Dense	0.6164	0.7256	0.6493	Best Dense (blind)
Aya 8B	BM25	0.5046	0.6284	0.5377	Good
Expanded experiments					
Aya 8B ($\beta = 2$)	BM25	0.5855	0.7128	0.6165	Best BM25 (rep.)
Hybrid RRF $k = 20$	BM25+Dense	0.6267	0.7597	0.6517	Best no-QE hybrid
BM25+CSQE	BM25	0.6157	0.7447	0.6380	Good
Dense+CSQE	Dense	0.5915	0.7073	0.6225	Good
BM25-expanded RRF	**BM25+Dense**	**0.7137**	**0.8363**	**0.7362**	**Best overall**

B.4 Representative Big-Win Queries

Table B.4 gives the three worked examples of the corpus grounding effect discussed in Section 4.10.3. In all three cases CSQE achieved NDCG@10 = 1.000 while the blind baseline achieved 0.000.

Table B.4: Representative big-win queries illustrating the corpus grounding effect. In all three cases, CSQE achieved NDCG@10 = 1.000 while the blind baseline achieved 0.000.

Query (gloss)	**Blind QE generates**	**CSQE grounds to**	**CSQE**
ما هو الرباط المنصوري؟ (“What is al-Ribat al-Mansuri?”)	A surgical ligature/suture used in operations	A Mamluk-era Sufi lodge (*ribat*) endowed by Sultan al-Mansur Qalawun	1.000
ما هي الأسماء الخمسة؟ (“What are the Five Nouns?”)	A list of popular given names (Muhammad, Adam, Ibrahim, …)	The Arabic grammatical category of the Five Nouns (أب, أخ, حم, فو, ذو)	1.000
ما هو الفن الجزيري؟ (“What is Insular art?”)	Modern environmental / land art	Insular (Hiberno-Saxon) art of the post-Roman British Isles	1.000

Appendix C

Implementation Code

This appendix reproduces the code that implements the corpus-steered query expansion (CSQE) pipeline described in Section 3.8: the system prompts used for both the blind and the corpus-grounded generation branches, the routine that assembles the final expanded query, and the configuration that fixes every hyperparameter reported in Chapter 3. The listings are taken verbatim from the experiment notebooks, with diagnostic print statements and environment-specific paths removed for readability. The complete code base, including the retrieval, fusion and evaluation components, is available in the repository given in Section C.4.

C.1 Query Expansion System Prompts

Two system prompts are used. The first is the blind (Query2Doc-style) prompt of Section 3.4.3, which asks the model to write a passage answering the query from its own parametric knowledge. The second is the corpus-grounded CSQE prompt of Section 3.8, which instead casts the model as an information-retrieval assistant and asks it to extract key sentences from the passages returned by the first pass. Both require Arabic output.

Listing C.1: Blind and corpus-grounded system prompts.

```
BLIND_SYSTEM = (
    "You are asked to write a passage that answers the given query. "
    "Do not ask the user for further clarification. "
    "Respond in Arabic only."
)

CSQE_SYSTEM = (
    "You are an information retrieval assistant. "
    "You will examine retrieved documents and extract key sentences "
    "relevant to the query. "
    "The documents are in Arabic. Respond in Arabic only."
)
```

The corpus-grounded branch is additionally prefixed with a single English worked example, taken from Table 2 of the original CSQE paper [22]. This is the sole departure from the zero-shot protocol described in Section 3.4.2; Aya Expanse, being multilingual, transfers the pattern to Arabic queries while still producing Arabic output.

Listing C.2: The one-shot demonstration prepended to the corpus-grounded prompt.

```
CSQE_ONE_SHOT = (
    'Query: "how are some sharks warm blooded"\n'
    'Retrieved documents:\n'
    '1. Most sharks are cold-blooded. Some, like the Mako and the Great '
    'white shark, are partially warm-blooded (they are endotherms)...\n'
    '2. Are sharks cold-blooded or warm-blooded? Sharks have a reputation '
    'as cold-blooded...\n'
    '3. Great white sharks are some of the only warm blooded sharks...\n'
    'You will begin by examining the initially retrieved documents and '
    'identifying the ones that are relevant, even partially, to the query. '
    'Once the relevant documents are identified, you will extract the key '
    'sentences from each document that contribute to their relevance.\n'
    'Based on the query "how are some sharks warm blooded", I have examined '
    'the initially retrieved documents. Here are the relevant documents and '
    'the key sentences extracted from each:\n'
    'Document 1:\n'
    '"Most sharks are cold-blooded. Some, like the Mako and the Great white '
    'shark, are partially warm-blooded (they are endotherms)."\n'
    'Document 3:\n'
    '"Great white sharks are some of the only warm-blooded sharks."\n'
)
```

The two prompt builders below assemble the final user messages. The blind branch passes the query through unchanged, because its system prompt already carries the task; the corpus-grounded branch numbers the truncated first-pass passages and appends the extraction instruction.

Listing C.3: Prompt construction for the corpus-grounded and blind branches.

```
def build_csqe_prompt(query, retrieved_docs_truncated):
    """CSQE corpus-grounded prompt. The model should EXTRACT sentences
    from the retrieved documents, not generate freely."""
    docs_str = '\n'.join(
        f'{i+1}. {doc}' for i, doc in enumerate(retrieved_docs_truncated)
    )
    prompt = (
        f'{CSQE_ONE_SHOT}\n'
        f'Query: "{query}"\n'
        f'Retrieved documents:\n{docs_str}\n'
        f'You will begin by examining the initially retrieved documents '
        f'and identifying the ones that are relevant, even partially, to '
        f'the query. Once the relevant documents are identified, you will '
        f'extract the key sentences from each document that contribute to '
        f'their relevance. Respond in Arabic only.'
    )
```

```
    return prompt


def build_blind_prompt(query):
    """Blind (Query2Doc) prompt. The system prompt handles the task;
    the user message is just the query."""
    return query
```

C.2 Query Repetition and Expansion Assembly

The routine below is the core of the method. For a batch of queries it builds both prompt types, invokes the generator four times (twice corpus-grounded, twice blind), and assembles the final expanded query. The assembly step is the single line that implements the repetition factor α of Section 3.6: the original query is repeated α times and concatenated with all four generated passages, so that the original query terms are not diluted by the much longer expansion when the result is scored by BM25.

Listing C.4: CSQE batch enhancement and final query assembly.

```
def batch_enhance(self, qids, queries):
    """Full CSQE pipeline for a batch of queries.
    Four batched generation passes: corpus x2 + blind x2."""
    n = len(qids)
    corpus_bs = self.config['corpus_batch_size']
    blind_bs = self.config['blind_batch_size']
    max_corpus_len = self.config['max_corpus_prompt_length']
    max_blind_len = self.config['max_blind_prompt_length']

    # Step 1: build all prompts
    corpus_prompts, blind_prompts, all_retrieved = [], [], []
    for qid, query in zip(qids, queries):
        docs = self.get_retrieved_docs(qid) # BM25 first pass, truncated
        all_retrieved.append(docs)
        doc_texts = [d['text'] for d in docs if d['text']]
        corpus_prompts.append(build_csqe_prompt(query, doc_texts))
        blind_prompts.append(build_blind_prompt(query))

    # Step 2: corpus-grounded expansions (2 samples)
    corpus_samples_1 = self.batch_generate(
        CSQE_SYSTEM, corpus_prompts, corpus_bs,
        temperature=1.0, max_length=max_corpus_len)
    corpus_samples_2 = self.batch_generate(
        CSQE_SYSTEM, corpus_prompts, corpus_bs,
        temperature=1.0, max_length=max_corpus_len)

    # Step 3: blind expansions (2 samples)
```

```
    blind_samples_1 = self.batch_generate(
        BLIND_SYSTEM, blind_prompts, blind_bs,
        temperature=1.0, max_length=max_blind_len)
    blind_samples_2 = self.batch_generate(
        BLIND_SYSTEM, blind_prompts, blind_bs,
        temperature=1.0, max_length=max_blind_len)

    # Step 4: assemble the expanded query
    alpha = self.config['query_repetition']
    results = []
    for i in range(n):
        corpus_exps = [corpus_samples_1[i], corpus_samples_2[i]]
        blind_exps = [blind_samples_1[i], blind_samples_2[i]]
        all_exps = corpus_exps + blind_exps

        # Repeat the original query alpha times, then append all
        # four generated passages. This is the expanded query that
        # is passed to the BM25 second pass.
        final = (queries[i] + ' ') * alpha + ' '.join(all_exps)

        results.append({
            'qid': qids[i],
            'original': queries[i],
            'retrieved_docids': [d['docid'] for d in all_retrieved[i]],
            'corpus_expansions': corpus_exps,
            'blind_expansions': blind_exps,
            'enhanced': final,
        })
    return results
```

C.3 CSQE Configuration

The configuration below fixes every hyperparameter of the primary CSQE experiment reported in Section 4.8. The values correspond to those stated in Section 3.8: a first pass of $k = 5$ documents each truncated to 128 tokens, two corpus-grounded and two blind samples, a repetition factor of $\alpha = 4$, and a generation temperature of 1.0 chosen to give diversity across the four samples.

Listing C.5: Configuration for the primary CSQE experiment.

```
CONFIG = {
    # BM25 first pass
    'top_k_docs': 5, # top-5 carry most of the signal and
                             # halve the corpus prompt length
    'doc_truncation_tokens': 128, # truncate each retrieved doc

    # LLM generation
```

```
    'model_name': 'CohereForAI/aya-expanse-8b',
    'temperature': 1.0, # diversity across multiple samples
    'max_new_tokens': 128,
    'top_p': 0.9,
    'num_corpus_samples': 2, # corpus-grounded expansions
    'num_blind_samples': 2, # blind (Query2Doc-style) expansions

    # Query construction: alpha = num_corpus + num_blind = 4
    'query_repetition': 4,

    # Batching (cross-query batching on the A100)
    'corpus_batch_size': 8, # long prompts (~800-1000 tokens)
    'blind_batch_size': 32, # short prompts (~60-80 tokens)
    'max_corpus_prompt_length': 2048,
    'max_blind_prompt_length': 512,
    'macro_batch_size': 200, # queries per checkpoint cycle
}
```

C.4 Source Code Repository

The complete implementation is publicly available at:

`https://github.com/Osmanoor/arabic-rag-query-enhancement`

The repository contains the components not reproduced above, including the BM25S and mDPR retrieval wrappers, the reciprocal rank fusion and convex combination implementations used for the hybrid experiments of Section 4.7, the `pytrec_eval` evaluation harness that produced every metric reported in Chapter 4, the per-model query generation notebooks, and the ablation and error-analysis notebooks.